WORLDBUILDING WITH INKHAVEN

Building the World with Inkhaven

A Worldbuilder's Process — from a Single Seed to a Living World

Vladimir Ulogov

2026 · examples assume Inkhaven 3.0.0 or newer

Contents

Before You Begin

This is a book about building a world you can believe in — and then writing inside it. A place with a sky that makes sense, land that the weather actually shapes, rivers that run where rivers would run, cities that stand where people would build them. A world with a past that predates page one, peoples who did not spring into being the moment your hero met them, and a calendar that says what season it is when your character looks out the window.

You will build it inside **Inkhaven** — a writing tool with a **World Simulation** built in. You will not draw a map by hand or fill a binder with notes that quietly contradict each other by chapter twelve. Instead you will set a few starting conditions — a star, a planet, a seed — and let a small, patient engine grow the rest, consistently, every time. Then you will deepen that world with a history and a people, and finally carry it to your desk, so the world is **present while you write** rather than filed away in a drawer.

Who this book is for

This guide assumes **no prior knowledge** — of worldbuilding, or of Inkhaven.

You do not need to have built a world before. You do not need to know what a biome is, or how a river system forms, or why a coastline ends up where it does. Every idea is introduced the first time it is needed, in a marked box like this one:

> TERM **Worldbuilding**
>
> The craft of inventing a setting — its geography, history, peoples, and rules — thoroughly and consistently enough that a reader (or a player, or you, six months later) never catches it contradicting itself. Good worldbuilding is less about how much you invent than about how well the pieces hold together.

You do not need to be a novelist. A world serves anyone who tells stories in a setting — a novelist, a game-master preparing a campaign, a screenwriter, a designer. Where this book says "your story," read it as "your project"; the craft is the same.

And you do not need to be a programmer. You will type a few short commands and edit one small text file, and the book walks you through every character of it.

How this book talks to you

Because you are learning a craft and not just a tool, the asides in this book come in five clearly-marked kinds. You will see each of them often, so meet them now.

> TERM **Term**
>
> A boxed definition, like the two above. Every piece of jargon is defined in one of these the first time it appears — so you can always read straight through without looking anything up.

> NOTE
>
> A **Note** is a practical remark about how Inkhaven behaves — a command, a default, a place a thing is saved. When you are actually at the keyboard, the Notes are what you act on.

> INSIGHT
>
> An **Insight** is a worldbuilding principle — the "why" beneath the "how", the thing worth remembering long after you have forgotten which command did what. The Insights are the real subject of this book.

> ASK YOURSELF
>
> An **Ask Yourself** box poses a worldbuilding question for you to answer about **your own** world before you build it. Worldbuilding is mostly the art of asking good questions in the right order; these boxes are that order, made explicit.

> **PITFALL**
>
> A **Pitfall** is a common mistake and how to step around it — usually a way that worlds quietly become inconsistent, which is the one thing good worldbuilding exists to prevent.

> **TRY IT**
>
> A **Try It** box is a short exercise at the keyboard: a command to run, a value to change, a result to look at. You will learn this far faster by doing it than by reading about it, so do them.

What you will be able to do

By the last page you will be able to:

- grow a complete physical world — sky, land, climate, rivers, and settlements — from a handful of choices, and understand **why** each part came out the way it did;
- give that world a past: an age of founding, migrations, the rise and fall of realms, dated on the world's own calendar;
- give it a people: nations, cultures, beliefs, and the sketch of a language each one speaks;
- decide, deliberately, what you let the world invent and what you declare by your own hand — and keep the two from ever contradicting each other;
- and write **against** that world: with the season and weather of a scene at your cursor, a check that your riders cannot cross a continent in a day, and the world's places and calendar flowing straight into your manuscript.

Not a set of tools — one environment

It is easy to meet Inkhaven as a **toolbox**: a world compiler here, a map renderer there, a place to keep character notes, a checker for your prose. Each of those is real, and this book will teach you to use them. But if a toolbox is all you see, you will use these the way you used the binder and the spreadsheet before them — side by side, and quietly drifting apart by chapter twelve. Inkhaven is built on the opposite

premise. Its parts are not a set of instruments laid out on a bench; they are one **environment**, and every part knows about the others.

> TERM **System book**
>
> Inkhaven keeps certain kinds of project-wide material in dedicated **system books** that live alongside your manuscript and can be read, searched, and cited from it. Some you fill **by hand** — **Characters** for your cast, **Places** for your settings, **Artefacts** for the objects that matter to your story — and some the tools maintain for you, like the **Timeline** of your story's events and the **World** book that holds your compiled world. They are all the same kind of thing, and they all speak to the same manuscript.

The characters, places, and artefacts of your world are **yours to author**, first and always. You open the Characters book and write your protagonist; you open Places and set down the city your story opens in; you open Artefacts and record the sword, the letter, the stolen crown. These entries are not outputs of the simulation — they are your own worldbuilding, kept where the rest of Inkhaven can see them. The World Simulation is one **contributor** to that shared world, never its author: it can **propose** a settlement into Places, or a realm's ruler into Characters, for you to accept or refuse — but the books are yours, the entries are yours, and most of what fills them you will have written by hand. The world never touches the Artefacts book at all; the objects of your story are wholly your own.

Because these books are shared, the connections cost you nothing. A place you tag in your prose is the same place the world gave a climate and a position on the map. A character you wrote is the same character an event on the Timeline is dated around. The season at your cursor is read from the same astronomy that raised your mountains. Nothing has to be kept in step by hand, because nothing lives in its own corner to begin with.

> INSIGHT
>
> A toolbox asks you to remember which tool holds which truth, and to reconcile them yourself. An environment holds **one** truth and lets every tool read it. That is the difference this book is really teaching: not a set of commands, but a place where your world, your cast, your map, and your manuscript are all the same world, seen from different windows.

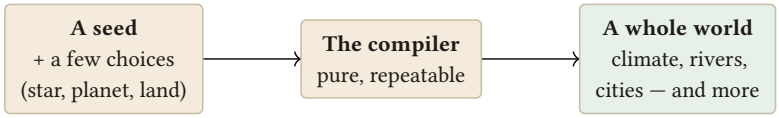
So the world you build here is not a separate program bolted onto your writing. It feeds the same Places you tag in your prose, the same Timeline your scenes are dated on, the same cast and objects you invented — and, as you will see in Part VI, the very paragraph you are typing.

INSIGHT

A setting is only worth the trouble if it **touches the page**. The measure of a built world is not how thick the appendix is — it is how often the world saves you from a contradiction, hands you a detail, or tells you what the weather is while you write. Keep that test in mind through every chapter.

The one idea to carry with you

If you remember nothing else from this introduction, remember this picture. It is the spine of the whole book:



You do not draw the world. You set its starting conditions, and the same seed always grows the same world.

You do not build a world the way you draw a map — placing every mountain by hand and hoping the rivers you sketch would really flow downhill. You set the starting conditions, and the world **follows** from them. Give the engine a star and a planet and a number to seed its choices, and it works out the seasons, raises the mountains, runs the rain down them into rivers, and settles people where the land would support them — the same way, every time you ask.

TERM **Seed**

A single number that fixes all the "random" choices a world involves — where exactly the coastlines fall, which valley grows the first city. The same seed always produces the same world, so your world is **reproducible**: you can share the short definition, and anyone gets your world back, cities and all.

Everything in this book is a way of working with that idea: choosing the starting conditions well, reading what grew from them, deepening it with time and people, and finally writing against it.

What you will need

Inkhaven, installed and open on a project — a folder with a manuscript in it. That is all. Nothing here requires an account, a subscription, or a paid connection. The world compiler is fully deterministic and runs offline; the few places where the world reaches out to the rest of Inkhaven (a language model to help name things, an external map tool to render a picture) are optional and clearly marked where they appear.

How to read this book

Read it front to back the first time. Part I builds the foundation — why a world is worth building, the idea of a world as a system, and your very first compiled world — and everything after it assumes those three chapters. Part II grows the physical world one layer at a time; Parts III and IV give it a past and a people; Part V is about the line between what you invent and what you declare; and Part VI brings the whole world to your writing desk. Part VII builds one world end to end, so you can see the whole process in motion.

Commands you type look like `realworld compile`. Every one of them is a subcommand of Inkhaven: the full command is `inkhaven realworld compile`, and the book abbreviates it to `realworld …` once Chapter 3 has made that plain. Keys you press look like `Ctrl+B W`. When a command shows you something, the book tells you what you would see and why it matters — but it draws the **shape** of the work with diagrams rather than screenshots, because screenshots age and the ideas beneath them do not.

Turn the page, and let us talk about why you would build a world at all.

WHAT YOU LEARNED

- This book teaches a **process** for building a believable, consistent world and writing inside it — assuming no prior knowledge of worldbuilding or Inkhaven.
- Its asides come in five marked kinds — **Note, Insight, Ask Yourself, Pitfall, Try It** — plus boxed **Term** definitions for every piece of jargon.
- You do not draw a world; you set its starting conditions and a **seed**, and a deterministic compiler grows the rest — the same way every time.
- Inkhaven is not a toolbox but one **environment**: your **Characters**, **Places**, and **Artefacts** books are yours to author by hand, and the World Simulation is one contributor that **proposes** into them — never their author. A built world is only worth it if it **touches the page**.

PART I

What a World Is

CHAPTER 1

1

Why Build a World

You could write your whole story without ever building a world. Many fine stories are set in a place the author holds loosely in mind — a city that is mostly a mood, a countryside that is mostly weather. If your story lives close to the characters and rarely looks up at the wider setting, that may be enough, and you should not feel obliged to do more.

But most stories that reach for a setting eventually lean on it. A river has to be crossed. A journey has to take a plausible number of days. It has to be winter in chapter three and still winter, at the same latitude, when the characters struggle home two chapters later. The moment your setting has to **bear weight** — to be consulted, measured against, returned to — you are no longer merely imagining it.

You are building it, whether you meant to or not. This book is about building it **on purpose**, so that it holds.

The binder that drifts

The oldest way to build a world is the binder: a folder of notes, invented as the story needs them. It works beautifully at first. Then, somewhere around chapter twelve, the binder betrays you. The port city you described as three days' ride from the capital is, according to a note you wrote in chapter four, on the far coast — a month away. The harvest festival falls in autumn on one page and in spring on another. Nobody put those contradictions there deliberately. They crept in because a binder has no memory of its own; it only remembers what you last wrote in it, and you cannot hold a whole world in your head without it quietly drifting.

A built world does not drift, because it is not a pile of notes. It is a single consistent object that every fact is drawn from. Ask it twice how far the port is from the capital, and it answers the same both times — not because you were disciplined, but because there is only one world for it to answer from.

> TERM **Canon**
>
> The set of facts that are **true** in your world — the ones your story must not contradict. In a binder, canon is whatever you can find and remember. In a built world, canon is what the world actually contains: a single source you can ask, rather than a memory you must keep.

> TERM **Consistency**
>
> The quality of a world never contradicting itself: every fact agreeing with every other, across every chapter and every corner of the map. Consistency is not the same as detail. A world can be sparsely imagined and perfectly consistent, or richly imagined and hopelessly at odds with itself. It is the agreement of the parts, not the number of them, that a reader feels.

Three things a built world gives you

Set against the drifting binder, a built world offers three payoffs. They are the reason to do any of this at all.

Consistency — it never contradicts itself

This is the first and plainest gift. Distances stay fixed. Seasons stay in order. The coastline is in the same place on page four hundred as it was on page four. You are freed from the exhausting, invisible labour of keeping a whole world straight in your head, because the world keeps itself straight. Every question you put to it is answered from the same body of fact, so the answers cannot disagree.

Discovery — it hands you facts you did not plan

This is the payoff nobody expects the first time. When you build a world properly, it contains far more than you deliberately put in — and some of what it contains is exactly what your story needed. You reach the moment your travellers must cross the mountains, and you find the world already has a river valley cutting through them, a natural pass where a road would obviously run. You did not place it there for the plot. It is there because that is where a river of that size, in that terrain, would carve its way — and now the plot has a crossing that feels inevitable rather than convenient.

> **INSIGHT**
>
> A built world does not only **store** what you decide; it **tells you things you never decided**. The most useful worldbuilding is not the part you plan — it is the part the world hands back to you, already consistent with everything else, precisely when the story reaches for it.

Groundedness — details that feel true because they are true

The third gift is the hardest to fake and the easiest to feel. A grounded detail is one that is not merely asserted but **caused** — the fishing town is poor not because you decided it should be atmospheric, but because it sits on a cold, harbourless stretch of coast where little grows and few ships call. A reader rarely works out the cause consciously. They simply feel that the place is real, because its details are consequences of one another rather than a scatter of invented facts. Groundedness is what you get when consistency runs all the way down: when nothing in the world is arbitrary, because everything follows from something else.

Answering questions before the story asks them

Here is the most useful way to think about the whole craft. Worldbuilding is the work of **answering the world's questions before your story asks them**. What is the climate where this scene is set? How long is a year here, and what do people call its seasons? Where would a city of this size actually stand? Your story will ask these questions — some of them on the page, some only in a reader's quiet sense of whether things add up. If you have already answered them, in a way that agrees with all your other answers, the story never stumbles. If you have not, you answer them on the fly, under pressure, and that is precisely how the binder starts to drift.

> **ASK YOURSELF**
>
> Before you build anything, ask the one question this whole book turns on: what does **your** story actually need from its world? A single besieged city needs almost nothing beyond its walls. An age-spanning migration across a continent needs climate, distance, and centuries. Name what your story leans on, because that — and not a hunger for detail — is what tells you how much world to build.

> **PITFALL**
>
> The opposite mistake to the drifting binder is **over-building**: inventing a thousand years of dynastic history, a full pantheon, and the grammar of six languages for a story that never leaves one valley in one summer. Detail your story never touches is not richness; it is unread notes that cost you time and, worse, invite contradictions with the parts that **do** reach the page. Build what the story needs, plus a modest margin for discovery — no more. The chapters that follow will keep returning you to this restraint.

The good news is that building a world need not mean inventing a thousand disconnected facts by hand. As the next chapter shows, you can set a few starting conditions and let most of the world **follow** — consistently, and with more inside it than you put in. That is where the discovery comes from, and it is the idea the rest of Part I is built on.

WHAT YOU LEARNED

- A world becomes worth building the moment your story has to **lean** on it — when distances, seasons, and places must be consulted rather than merely imagined.
- A built world offers three things a binder cannot: **consistency** (it never contradicts itself), **discovery** (it hands you facts you never planned), and **groundedness** (details that feel true because they are consequences of one another).
- Worldbuilding is the craft of answering the world's questions — climate, distance, where a city would stand — **before** your story asks them, so the story never stumbles.
- Build only what your story needs, plus a small margin. **Over-building** — inventing what the story never uses — wastes effort and invites the very contradictions good worldbuilding exists to prevent.

CHAPTER 2

2

A World Is a System

The last chapter promised that most of a world can **follow** from a few choices rather than being invented fact by fact. This chapter is about how — and the idea underneath it is the most important one in the book. A world, built well, is not a collection. It is a **system**: a small set of starting conditions, and a chain of consequences that fall out of them on their own.

Emergence: you set the physics, the world falls out

Think about how a real coastline comes to be shaped the way it is. Nobody chose it. It is the result of rock and rain and sea acting on each other over ages — an outcome,

not a decision. The same is true of almost everything in a convincing setting: where deserts lie, which rivers are navigable, why the great cities grew where they did. These are not facts to be authored one by one. They are **consequences**.

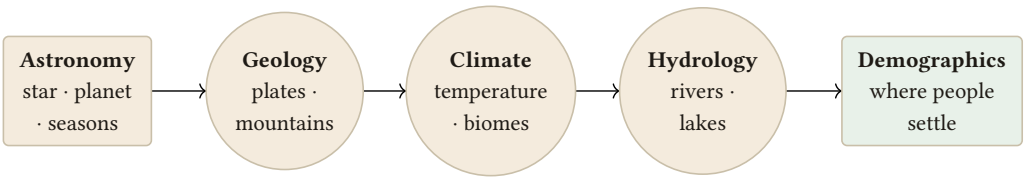
> TERM **Emergence**
>
> When complex, specific, believable detail arises on its own from a few simple rules and starting conditions — rather than being placed by hand. A coastline, a climate, a pattern of cities: each **emerges** from the conditions beneath it. Emergence is what lets a small definition grow into a large, coherent world.

This is the exact opposite of drawing a world by hand. When you draw, every feature costs you a decision, and every decision is a chance to contradict another. When you let the world emerge, you make only the handful of decisions that truly belong to you — the size of the star, the tilt of the planet — and the rest arrives already consistent, because it was **computed** from those decisions rather than guessed alongside them.

> INSIGHT
>
> Drawing a world places every detail by hand and hopes the pieces agree. Emergence sets a few conditions and lets the details **derive** from them, so they cannot help but agree. This is the whole reason a built world stays consistent where a binder drifts: consistency is not maintained, it is **produced** — a property of how the world is made, not a discipline you must keep.

The chain of layers

The physics of a world does not emerge all at once. It comes in **layers**, each one computed from the layers before it. This is the spine of the physical world, and you will spend all of Part II inside it:



Each layer is a pure consequence of the ones before it: the sun shapes the climate, the climate carves the rivers, the rivers decide where the cities stand.

> TERM **Layer**
>
> One stage of the world's physical derivation — astronomy, geology, climate, hydrology, or demographics. Each layer takes the finished layers before it as its input and produces the next set of facts. You will meet them one at a time; for now, what matters is the **order**, because the order is the causation.

Read the chain as a story of cause. **Astronomy** comes first: a star, a planet, an orbit, a tilt — and from these, the length of a year and the swing of the seasons. **Geology** raises the land: plates collide, mountains rise, a sea level settles. **Climate** is then forced by the two above it — the sun deciding how much heat each latitude receives, the mountains deciding where the rain falls and where it never does — and out of that come temperature, wind, and biomes. **Hydrology** follows the climate: rain has to go somewhere, so it runs downhill into rivers, pools into lakes, and drains whole watersheds. And **demographics** comes last, because people settle where the earlier layers made it possible to live — at a river's mouth, in a fertile valley, where water and land and climate agree.

Nothing in this chain is arbitrary. Each layer is a **pure consequence** of the ones before it: change the tilt of the planet and the seasons shift, which moves the climate bands, which redraws the rivers, which relocates the cities. That is emergence made concrete — and it is why a world built this way is grounded all the way down, exactly as the last chapter promised.

The same seed, the same world

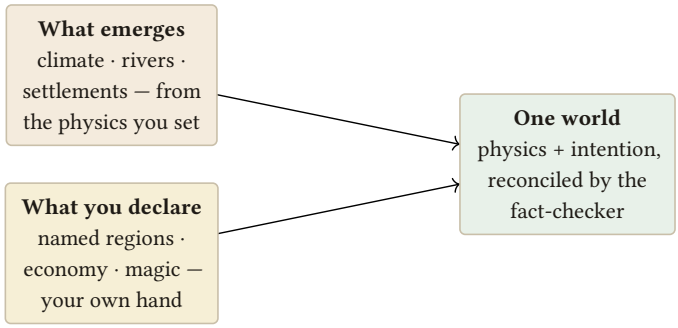
For this to be trustworthy, it must be **repeatable**. If the same starting conditions gave you a different world each time you asked, none of the consistency would be worth anything. They do not. The system is deterministic, anchored by the seed you met in the introduction.

> **TERM Deterministic**
>
> Producing exactly the same result every time, given the same input. The world compiler is deterministic: the same definition, with the same **seed**, always grows precisely the same world — the same coastlines, the same rivers, the same cities in the same valleys. Nothing is left to chance that you did not choose.

Determinism is what turns a world from a happening into an object. Because the same seed always yields the same world, your entire setting is captured by a short, shareable definition: hand someone your seed and your choices, and they get your world back, whole. And when you deliberately **change** the seed, you are not patching one detail — you are asking the whole system for a different world grown by the same laws. Every world it can give you is internally consistent; the seed simply chooses which of them you get.

The two hands

Emergence is powerful, but it is not the whole of worldbuilding — and it would be a poor tool if it were. Some things should never be left to physics. The name of a city is not a consequence of rainfall. Your world's economy, and the rules of its magic, are **yours** to declare. So a built world is made with two hands:



A world has two hands: the physics that falls out on its own, and the names and rules you set by intention. Both are honoured.

With one hand you set the **physics** and let the world emerge — climate, rivers, where people settle. With the other you **declare** what physics has no opinion about — the names on the map, the shape of the economy, the exceptions your

magic makes to the ordinary rules. Neither hand overrules the other. The emerged world gives your declarations somewhere true to stand; your declarations give the emerged world meaning and intent. Later chapters return to this line often — where exactly it falls, and how Inkhaven keeps the two from ever contradicting each other — because knowing what to let emerge and what to declare is much of the craft.

ASK YOURSELF

Look at your own world and sort it into the two hands. Which parts do you **care** to control by name — a particular ruling dynasty, a sacred mountain, the tongue a people speaks? And which parts would you be content to let **emerge**, and perhaps be surprised by — where the deserts fall, which river the capital sits on? There is no wrong answer, but there is a wasteful one: controlling by hand what you could have let the world grow for you, consistently and for free.

WHAT YOU LEARNED

- A world is a **system**: a few starting conditions and a chain of consequences that **emerge** from them on their own — the opposite of drawing every detail by hand.
- The physical world derives in **layers** — astronomy, geology, climate, hydrology, demographics — each a pure consequence of the ones before it, so consistency is produced rather than maintained.
- The compiler is **deterministic**: the same definition and **seed** always grow the same world, which makes your whole setting a short, shareable, reproducible object.
- A world is built with **two hands** — what you let **emerge** from physics, and what you **declare** by intention (names, economy, magic). Much of the craft is knowing which is which.

CHAPTER 3

3

Your First World

Enough principle. In this chapter you will make a world — a real one, compiled from a single small file, with a sky and land and rivers and cities in it — in about four commands. You will not understand every layer yet; Part II is for that. The aim here is only to see the whole machine turn over once, from a definition to pages you can read, so that everything after this has something concrete to stand on.

Every world command belongs to Inkhaven itself. `realworld` is not a separate program — it is the **part of Inkhaven that builds worlds**, one of its subcommands, run from your project folder.

NOTE

Read every `realworld` in this book as `inkhaven realworld`. The full command you type at the terminal always begins with `inkhaven` — for example, the scaffolding command just below is really:

```
inkhaven realworld new myworld
```

From here on the book writes only the `realworld …` part, for brevity; the `inkhaven` in front is always understood. (Inside Inkhaven's own command palette you drop it too.)

Scaffolding a world

You do not write `world.hjson` from a blank page. Ask `realworld` to lay down a starter for you:

```
realworld new myworld
```

This writes a `world.hjson` describing a plain, Earth-like world — a single yellow star, a planet much like our own, a familiar calendar — ready to compile as-is and ready for you to edit.

TERM **world.hjson**

The single file that defines your world. HJSON is a relaxed form of JSON that tolerates comments and unquoted keys, so the file stays readable by a human. It holds your world's `name`, its `seed`, its `primary_language`, and blocks of physical and declared detail — the **whole** definition of your world lives here, and nowhere else.

> **NOTE**
>
> `world.hjson` is written to your **project root** — the top folder of the project you have open in Inkhaven, alongside your manuscript. There is one per project. When any `realworld` command speaks of "the world," it means this file and what compiles from it.

Open it, and the shape is easy to read even before you know every field:

```
{
  name: "My World"
  seed: 0x5eed
  primary_language: "Common"

  astronomy: {
    star:   { class: "G", luminosity_solar: 1.0, age_gyr: 4.6 }
    planet: { mass_earth: 1.0, radius_earth: 1.0, axial_tilt_deg:
23.4,
              day_length_hours: 24, rotation_direction:
"prograde" }
    orbit:  { semi_major_axis_au: 1.0, eccentricity: 0.017,
year_length_days: 365 }
  }
}
```

Three things at the top name the world: a `name`, a `seed` — the number from the introduction that fixes every "random" choice, here written in hexadecimal — and a `primary_language` for the names it will generate. Then the `astronomy` block, the one required layer, sets the physics the rest of the world emerges from: the star's class and brightness, the planet's mass and tilt and day, the orbit that decides the length of the year. Change nothing, and you have a workable Earth-like world. Change the tilt, and you have set a different climate in motion — but that is Part II's pleasure, not this chapter's.

Checking it holds together

Before compiling the whole thing, ask whether the definition is even valid:

```
realworld validate
```

This compiles every layer in turn and reports each one `ok`, or stops at the first value it cannot make sense of — a negative year, a calendar whose months do not add up. It is the quickest way to catch a broken edit, and worth running whenever you change the file.

Compiling the world

Now grow it. With no arguments, `realworld compile` runs the entire layer chain — astronomy through demographics — and holds the finished world in hand:

```
realworld compile
```

> TERM **Compile**
>
> To run the world's layers and produce the finished world from `world.hjson`. A bare `realworld compile` (or `compile all`) computes the **whole** world; you can also compile a single layer by name while you experiment. Compiling is pure and deterministic: the same file always compiles to the same world.

Compiling gives you the world, but it does not yet write anything you can sit and read. For that, materialise it:

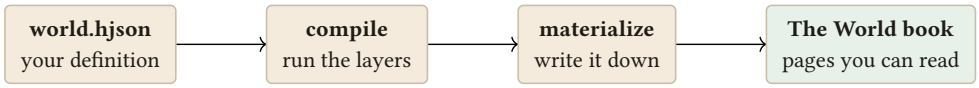
```
realworld compile --materialize
```

> TERM **Materialize**
>
> To write the compiled world into readable pages — chapter by chapter, layer by layer — inside Inkhaven's **World book**. Compiling produces the world in memory; materialising commits it to the page, so you and Inkhaven can read it, search it, and cite it from your manuscript.

TERM **The World book**

The read-only **system book** that holds your materialised world: a chapter per layer — the sky, the land, the climate, the waters, the peoples. It sits alongside your manuscript like the Places and Timeline books, and is rewritten each time you materialise, so it is always a faithful picture of the current world rather than a set of notes that can drift.

Here is the whole path you have just walked, from definition to readable pages:



Compiling turns a small definition into a full world; materialising writes that world into readable pages you keep alongside your manuscript.

If you would rather glance at the definition than read the full book, `realworld show` prints the world's current shape to your terminal — a fast way to confirm what seed and star you are working with.

TRY IT

Do the whole loop once, now. Run `realworld new myworld` to scaffold a world, then `realworld compile` to grow it, then `realworld compile --materialize` to write it into the World book. Finally, press `Ctrl+B W` in the editor to open the read-only **World overview** and page through the sky, land, climate, waters, and peoples the seed grew for you. You wrote a handful of lines; read how much world came back.

You don't have to take the first world

The seed you were handed grew **one** world — but it is one of billions, and there is nothing sacred about it. If its coastlines or its balance of land and sea are not what your story wants, you do not edit them by hand; you audition a few more and keep the one you like. `realworld variants` does exactly that: it grows several candidate worlds from consecutive seeds and prints a one-line summary of each, so you can compare them at a glance.

```
realworld variants --count 6
```

Each row reports what that seed grew — how many continents, how much of the surface is sea, how many settlements and people, the dominant biome, the largest realm — beside the seed itself. When one of them is the world you want, you adopt it by writing that number into `world.hjson` and compiling as usual:

EDIT · world.hjson

```
seed: 1715007
```

NOTE

This is the same discipline the rest of the book will keep returning to: **the world proposes, you choose.** `realworld variants` proposes whole worlds; later, `realworld history`, `polities`, and the rest propose the pieces inside one. Nothing is committed until you pick it — auditioning six worlds changes nothing on disk until you set the seed and compile.

TRY IT

Run `realworld variants --count 6` and read the six worlds. Notice how different a single number makes them — one seed a scatter of islands, the next a single broad continent. Pick the one that most looks like the stage your story wants, set its seed in `world.hjson`, and recompile. You have just chosen a world.

What you now have

Take a moment with what just happened. From a file you could read in a minute — a star, a planet, a seed — a full physical world emerged: a calendar and seasons, continents and mountains, climate bands and biomes, rivers and lakes, and cities standing where the land supports them. You placed none of it by hand. You set the conditions, and the system did the rest, the same way it will every time you ask.

That is the end of Part I. You now understand **why** a world is worth building, that a world is a **system** whose detail emerges from a few choices, and — as of this chapter — you have a real, compiled world of your own to open with `Ctrl+B W`. In Part II you will go into that world one layer at a time, starting with the sky, and learn to shape each layer on purpose. The machine has turned over once; now you learn to drive it.

> **NOTE**
>
> Prefer to build by **asking** rather than by hand-editing `world.hjson`? There is a full-screen interactive companion — `inkhaven worldbuilder` — that walks you through the same choices in an interview, scores the world live, and draws its map in the terminal. It is a front-end to everything in this book; every edit it makes lands in the same `world.hjson`. Chapter 20 is its complete guide. You can keep reading here and meet it at the end, or open it now and follow along.

> **WHAT YOU LEARNED**
>
> - `realworld new <name>` scaffolds a starter `world.hjson` in your project root — an Earth-like world with a `name`, `seed`, `primary_language`, and a required `astronomy` block.
> - `realworld validate` compiles every layer and reports each `ok`, catching a broken value before you rely on it.
> - `realworld compile` grows the **whole** world; `--materialize` writes it as readable, always-current chapters into the **World book**; `realworld show` prints the definition.
> - `Ctrl+B W` opens the read-only **World overview**. You end Part I with a real compiled world of your own, ready to explore layer by layer.

PART II

The Physical World

CHAPTER 4

4

The Sky

Every world begins with a sky. Not because the sky is where a reader looks first — most never look up at all — but because everything a reader **does** notice hangs from it. The length of a year, the turn of the seasons, the tides that time a harbour, the very difference between a temperate coast and a frozen one: all of it descends from a star, a planet, and the tilt of that planet as it swings around the star. In Inkhaven the world is grown one layer at a time, and the first layer — the one every later layer leans on — is **astronomy**. This is the root. Get the sky right and the land, the weather, and the rivers have a firm thing to grow from. Get it careless and every layer above it inherits the carelessness.

> **INSIGHT**
>
> The sky is the root of the world. Axial tilt drives the seasons; the seasons drive the climate; the climate carves the rivers and decides where people can live. When you set the astronomy, you are not describing a backdrop — you are setting the initial conditions from which the whole physical world falls out. Everything downstream inherits whatever you decide up here.

What the astronomy block holds

The whole sky lives in one block of your `world.hjson`. It reads almost like a form — a star, the planet that orbits it, the shape of that orbit, any moons, and the calendar your people keep. Here is the shape of it:

```
astronomy: {
  star:   { class: "G2V", luminosity_solar: 1.0, age_gyr: 4.6 }
  planet: {
    mass_earth: 1.0, radius_earth: 1.0,
    axial_tilt_deg: 23.4,
    day_length_hours: 24.0,
    rotation_direction: "prograde"
  }
  orbit:  { semi_major_axis_au: 1.0, eccentricity: 0.017,
year_length_days: 365.25 }
  moons: [
    { name: "Luna", mass_lunar: 1.0, period_days: 27.3 }
  ]
  calendar: {
    months: 12, month_length_days: 30, weekdays: 7,
    month_names: ["Frostmoon", "Thawmoon", "..."],
    new_year_aligns_to: "spring_equinox"
  }
}
```

You do not have to fill every field to something exotic. Copy Earth's numbers, as above, and you have a familiar sky you can trust while you learn — a real starting point, not a placeholder. Change one number at a time and watch what moves. Swap the Sun for a cooler, redder star and the whole energy budget shifts:

EDIT · world.hjson

```
astronomy: {
  star: { class: "K", luminosity_solar: 0.6 }
}
```

TERM **Axial tilt**

The angle between a planet's spin axis and the line straight up from its orbit — Earth's is about 23.4°. It is the single most consequential number in the sky, because it is **why there are seasons at all**: as the planet circles its star, the tilt leans first one hemisphere and then the other toward the light, so the same place gets more sun in summer and less in winter.

What the sky computes

Compiling the astronomy layer does more than tidy your numbers back to you. From the orbit and the planet's day, Inkhaven works out how long a year actually is **in that world's own days** — the `year_length_days` of the orbit divided by the `day_length_hours` of the planet — because a story is dated in days, not in some abstract fraction of an orbit. A world with an eighteen-hour day and a long slow orbit can run to five hundred days a year, and every calendar you build must answer to that number.

From the orbit and the tilt it finds the **four season markers** — the two equinoxes and the two solstices — the four moments that pin the year in place.

TERM **Solstice / Equinox**

The four turning points of the year. At an **equinox** the tilt leans neither hemisphere toward the star, so day and night are equal the world over — these are the balance points of spring and autumn. At a **solstice** one hemisphere leans most toward the star (its longest, brightest day) while the other leans away (its shortest) — the peaks of summer and the depths of winter.

From the tilt it also decides how **strong** the seasons are. This is the quiet power of that one field: a world with almost no tilt has a mild, monotonous year — perpetual near-equinox, little difference between the solstices — while a sharply tilted world has ferocious summers and brutal winters, because far more of the star's light lands on the leaning hemisphere. Push the tilt well past Earth's to feel it:

EDIT · world.hjson

```
astronomy: {
  planet: { axial_tilt_deg: 35 }
}
```

That quantity has a name.

TERM **Insolation**

The amount of a star's light and heat falling on a given patch of ground over a given time — literally, incoming solar radiation. It varies with latitude (the poles catch the light at a glancing angle, the tropics head-on) and with the season (the tilt swings each hemisphere toward and away from the star). Insolation is the raw energy budget the climate layer will spend; the sky is where it is first computed.

The compiler works out insolation for each latitude, which is precisely the number the **climate** layer will pick up in the next chapter and turn into temperature and rain. The sky hands the land its energy; the land does the rest.

Moons, synodic periods, and tides

If you gave your world moons, the sky computes their **synodic periods** and the tides they raise. A world with three moons has a busy, layered sky and complex tides; a world with none has still seas and dark nights. Hang a second, smaller moon on a faster orbit and the tides gain a second beat:

EDIT · world.hjson

```
astronomy: {
  moons: [
    { name: "Pale", mass_lunar: 1.0, period_days: 27.3 }
    { name: "Ember", mass_lunar: 0.4, period_days: 9.1 }
  ]
}
```

TERM **Synodic period**

The time a moon takes to return to the same phase as seen from the ground — new moon to new moon. It differs from the moon's raw orbital period because the planet is itself moving around the star meanwhile. The synodic period is the one your characters would actually keep time by, so it is the one worth knowing: it is the rhythm of a lunar month.

The calendar-consistency check

Last, the sky checks your **calendar** against the orbit it just computed. You declared a year — some number of months of some number of days. The compiler knows the true year length in planet-days. If the two disagree, it says so. A calendar is a human artefact laid over an astronomical fact, and the check keeps the artefact honest.

PITFALL

The commonest sky mistake is declaring a calendar that quietly contradicts the orbit — twelve months of thirty days (a 360-day year) sitting on top of an orbit that really runs 365.25 days, with nothing to reconcile the missing five and a quarter. The consistency check flags exactly this. Either adjust your months, or decide deliberately that your people's calendar **drifts** against the seasons — a genuine, story-rich choice — but do not leave the contradiction there by accident.

Compiling the sky, and the calendar bridge

To grow just this layer and read what it found, compile it alone:

```
realworld compile --layer astronomy
```

You will see the year in planet-days, the four season markers, the per-latitude insolation, the moons' synodic periods, and the verdict of the calendar check. This is the whole sky, computed and laid out, before any land exists to stand under it.

> **NOTE**
>
> The sky is also the source of your story's **calendar**. Run `realworld calendar` and Inkhaven derives a story-Timeline calendar from the astronomy — months, weekdays, the alignment of the new year to a season marker — as a set of lines you can adopt into `timeline.calendar`. The world proposes the calendar; you adopt it. Nothing reaches your Timeline until you choose to let it.

That bridge matters: it means the date at the top of a scene and the season your character sees out the window are computed from the **same** sky. They cannot drift apart, because they share a root.

> **ASK YOURSELF**
>
> How alien is your sky? One sun or two? A single familiar moon, or three that cross the night at different speeds? A brisk year, or a long slow one where a child might see only a handful of summers before growing up? You do not have to answer exotically — but answer **deliberately**, because every choice here is a choice the whole world below will have to live with.

TRY IT

Open your `world.hjson` and change one number: `axial_tilt_deg`. Set it near `0` and recompile with `realworld compile --layer astronomy` — watch the seasons flatten toward a single endless mild one. Now set it to `40` and recompile — watch the summers and winters pull violently apart. You have just felt, in one field, the lever that drives the entire climate of your world.

WHAT YOU LEARNED

- The **astronomy** layer is the **root** of the physical world — the first layer, and the one every later layer depends on.
- The `astronomy` block holds a **star**, a **planet** (crucially its `axial_tilt_deg`), an **orbit**, any **moons**, and a **calendar**.
- Compiling it yields the year in planet-days, the four **season markers**, per-latitude **insolation**, moon **synodic periods** and tides, and a **calendar-consistency check**.
- **Axial tilt** is the master lever: it sets how strong the seasons are, and so — through climate — shapes everything downstream.
- `realworld calendar` derives a story-Timeline calendar from the same sky, so your dates and your seasons can never drift apart — and you adopt it by choice.

CHAPTER 5

5

The Land

Under the sky lies the land, and the land is the layer where most people expect to do their worldbuilding — hunched over a map, drawing coastlines, penciling in a mountain range because the story wants a hard place to cross. Inkhaven asks you to work the other way. The **geology** layer does not ask you to draw. It asks you for a seed, and from that seed it grows a whole planet's crust: the plates that carry the continents, the ranges thrown up where those plates collide, the sea level that decides how much of it stands above water, the minerals in the rock, and the elevation of every point on the surface. You do not place the mountains. You set the conditions, and you **read** the mountains that emerged.

INSIGHT

You are not drawing coastlines. You set the seed and read what emerged — the same discipline as the sky, one layer down. Your job is to choose good starting conditions and then to **listen** to the land they produced, not to overrule it line by line. And when you genuinely must control the shape — a specific continent your story cannot do without — you do not reach for a pencil. You hand the layer a map.

Two ways to make a world's land

The geology layer will take its shape from one of two sources. By default it generates the land from your world's `seed` — the single number that fixes every "random" choice. The same seed always raises the same continents, so your world is reproducible: change the seed and you get a different planet entirely; keep it and the land is fixed forever.

TERM **Tectonic plate**

A large rigid slab of a planet's outer shell that moves, slowly, over the hotter material beneath. Where plates pull apart, oceans open; where they collide, crust crumples upward into mountain ranges. Inkhaven models plates because they are the **cause** beneath the scenery — the reason a range runs where it does rather than a line you drew because it looked good.

TERM **Continent**

A large connected mass of land standing above the sea. In Inkhaven a continent is not something you outline — it is what remains above `sea_level` once the plates have moved, the ranges have risen, and the water has found its level. You choose the conditions; the coastline is the consequence.

You need not take the seed's land exactly as it comes, either. A `generated` block lets you **steer** the generation without abandoning it — how many plates and continents to start from, how tall the mountains rise, and where the sea sits:

EDIT · world.hjson

```
geology: {
  generated: {
    plates: 9
    continents: 3
    mountain_orogeny: "ancient"
    sea_level: 0.45
    notable_minerals: ["iron", "silver", "obsidian"]
  }
}
```

Every key is optional; each nudges the same emergent process rather than replacing it. `mountain_orogeny: "ancient"` wears the ranges down to old, low hills; a higher `sea_level` drowns more of the map; `notable_minerals` names what the ground holds, which the economy fact-checker later uses to catch a claim about a metal your world does not have.

The second source is for when the emergent land is not what your story needs at all. Instead of the seed, you can hand the layer a real heightmap — a **DEM** — and it will build the continents from your image rather than from noise. This is the bring-your-own-map door: the whole of Inkhaven's downstream machinery — climate, rivers, cities — running on **your** chosen shape of land.

TERM **DEM (heightmap)**

A **digital elevation model** — an image in which the brightness of each pixel encodes the height of the ground there: black for the deepest sea floor, white for the highest peak, greys for everything between. It is the standard way real geographers store terrain, and the standard way to hand Inkhaven a shape you drew, scanned, or exported from another tool. Point `geology.dem` at one and the layer reads your land instead of inventing it.

To use it, add a `dem` block to your geology block. At its simplest it is just a `path` to the image:

EDIT · world.hjson

```
geology: {
  dem: { path: "maps/my-continent.png" }
}
```

The `dem` block also carries the `scale_km_per_pixel` that turns pixels into real ground — so that later, when a rider crosses your map, the distance is true — and the `sea_level_pixel_value`, the pixel brightness (0–255) Inkhaven reads as the shoreline: anything at or below it is sea.

EDIT · world.hjson

```
geology: {
  dem: { path: "heightmap.png", scale_km_per_pixel: 5.0,
sea_level_pixel_value: 40 }
}
```

NOTE

The `dem` path is relative to your project, not to wherever you happen to be standing in the terminal — `maps/my-continent.png` means the `maps` folder beside your `world.hjson`. Keep the image in the project and the world stays portable: the definition and the map travel together.

Producing a heightmap

You do not need special software or any artistic skill to make a DEM — it is an ordinary greyscale image, and a free image editor is enough. Before the recipe, three facts about how Inkhaven reads the file, so nothing about it is a guess:

- **Any common image format works** — PNG, JPEG, TIFF, BMP. Prefer **PNG**: it is lossless, so it will not smear your coastlines the way JPEG can.
- **The size does not matter.** Inkhaven resamples your image onto its own grid, so a 512×512 or 1024×512 picture is plenty; you do not need to match any exact dimension.
- **Brightness is height, and it is read relatively.** Inkhaven finds the darkest and brightest pixels in your image and stretches that range to fit — your darkest pixel

becomes the deepest sea floor, your brightest the highest peak. So use the **whole** range from black to white; the absolute grey values do not matter, only which areas are darker than which.

The easiest road: generate a DEM with plakat

Inkhaven has a companion tool, **plakat**, that can grow a heightmap for you from a single sentence of description — no drawing, no image editor, and no graphics card. It is the same tool `realworld map` uses to paint a world map, and its terrain engine can write out exactly the greyscale DEM the land layer wants, **deterministically**: the same description and seed always produce the same heightmap. If you already have plakat installed, start here.

> TERM **plakat**
>
> Inkhaven's companion image-and-map tool. Among its features, `plakat map` turns a prose world description into a fantasy map through a geometry engine (terrain → rivers → coastline → biomes), and can dump any layer of that engine — the tectonic **heightmap** included — as a PNG. The heightmap step is pure and offline (no model, no GPU); only the prose-to-spec parse uses a language model.

Here is the whole recipe, start to finish.

TRY IT

1. Describe the land and parse it to a reusable spec. This one step uses a language model to turn your sentence into a `MapSpec` file. Pick a `--map-scale` that matches how much of the world you are drawing — `region` for a continent, `inland-sea` or `hemisphere` for a whole world:

```
plakat map "a broad continent with a mountain spine down its
west coast, a wide
  river valley opening to the east, and an inland sea in the
north" \
  --map-scale region --map-dump-spec world.spec.json
```

2. Dump the heightmap from that spec — deterministic, offline, no model. The `--seed` fixes the terrain; change it for a different continent from the same description:

```
plakat map --map-spec world.spec.json --seed 7 --map-dump-
heightmap heightmap.png
```

You now have `heightmap.png`: a greyscale DEM, brighter where the land is higher — the same convention Inkhaven reads.

3. Put it in your project and point the world at it. Move the PNG into a `maps` folder beside your `world.hjson`:

EDIT · world.hjson

```
geology: {
  dem: { path: "maps/heightmap.png" }
}
```

TRY IT

4. Compile the land and look. Run `realworld compile --layer geology` and read what came up — the continents, the ranges, the coastlines — all grown over the terrain plakat handed you. From here, every downstream layer (climate, rivers, cities) follows from **your** land.

> **NOTE**
>
> Because a plakat spec + seed is byte-stable, your world stays reproducible: commit `world.spec.json` alongside `world.hjson` and anyone can regenerate the exact same `heightmap.png`. You can skip step 1 entirely if you are offline — hand-write a small `MapSpec` JSON, or reuse a committed one — since only the prose parse needs a model; the heightmap itself is pure geometry. And while you are there, plakat can dump the matching `--map-dump-rivers` and `--map-dump-coast` layers, a useful preview of what Inkhaven will grow.

Drawing one by hand, step by step

This is the most direct road — you get exactly the continents you imagined. Using **GIMP** (free, from `gimp.org`; the same steps work almost unchanged in Krita or Photoshop):

- **Make a new greyscale image.** `File → New`, set the size to `1024 × 512` pixels, then `Image → Mode → Grayscale`.
- **Fill it with black** — this is your open ocean. `Edit → Fill with FG Color` with the foreground set to black.
- **Paint your land in white.** Take a soft-edged brush, set the colour to white and the brush opacity low (about 20%), and paint where you want land. Build the brightness up in passes: one pass for coastal lowland (dim grey), more passes stacked in the interior for hills, brightest of all along the spines where you want mountains. Think of the brush as **raising** the ground each time you stroke.
- **Smooth the slopes.** `Filters → Blur → Gaussian Blur` with a radius of about `20` pixels. This is the important step (see the Pitfall): it turns your painted patches into gentle grades that rivers can run down.
- **Export it.** `File → Export As…`, name it `my-continent.png`, and save it into a `maps` folder next to your `world.hjson`.

Then point the world at it, exactly as shown above:

> **EDIT · world.hjson**

```
geology: {
  dem: { path: "maps/my-continent.png" }
}
```

Compile, and Inkhaven runs the entire climate-and-rivers machine over **your** land.

Where the coastline falls

The simplest way to set the sea is to **not** set it: leave `sea_level_pixel_value` out, and Inkhaven puts the shoreline at 40% up your height range — the lowest 40% of the land becomes ocean. To get more sea, paint more of your image dark; for more land, paint more of it bright. That trial-and-error is usually all you need. (If you want to place the coast at an exact brightness, `sea_level_pixel_value` takes a number from `0` to `65535`, where `0` is black and `65535` is white; everything at or below it is sea.)

Three shortcuts

If drawing is not your strength, three roads hand you a heightmap ready-made:

- **Generate one.** Free terrain tools — **Wilbur**, or Blender's built-in **A.N.T. Landscape** generator (and the paid **World Machine** and **Gaea**) — grow realistic erosion, ranges, and valleys from noise and export a greyscale heightmap directly. Good when you want plausible terrain without placing every ridge.
- **Borrow the real Earth.** The website `tangrams.github.io/heightmapper` lets you pan to any real region and download its terrain as a greyscale PNG; **QGIS** (free) can do the same over public SRTM elevation data. A quiet way to give a fantasy map the bones of a real coastline.
- **Ask a text-to-image generator.** Tools like ChatGPT / DALL·E, the free Bing / Microsoft Image Creator, Google Gemini, Midjourney, or Stable Diffusion can draw a heightmap straight from a written description — the fastest road of all if you can describe the shape you want but not draw it. It needs a good prompt and a little clean-up; both are below.

Prompting an image generator for a heightmap

The trick is to ask for a **heightmap** explicitly and to fix the convention (brighter = higher), because a picture that merely "looks like a map" will not do. Prompts along these lines work well:

```
A top-down greyscale heightmap (digital elevation model) of a
fantasy
continent. Black ocean; dark-grey coastal lowlands rising to
light-grey
hills and pure-white mountain ranges. Smooth, soft gradients. No
text, no
labels, no borders, no compass, no shading or hillshade —
```

```
greyscale
elevation only.
```

For a specific shape, describe it: **"a large landmass in the north-west and a long island chain curving to the south-east, a wide bay on the east coast."**

> **NOTE**
>
> Two clean-up steps make an AI image usable, because these tools do not produce a true DEM on their own:
>
> - **Desaturate it.** Generators output full colour even when they look grey. Open the image and convert it to greyscale (`Image → Mode → Grayscale` in GIMP), so brightness alone carries the height.
> - **Blur it.** AI heightmaps often have hard edges and stray texture. Apply a Gaussian Blur (radius 15–25) to smooth the slopes, exactly as for a hand-drawn map, and crop out any leftover frame or label.
>
> One more watch-point: some models **invert** the convention and paint mountains dark. If your continents come out as oceans, invert the image (`Colors → Invert`) so that bright is high again.

> **PITFALL**
>
> A crisp, high-contrast image with hard edges makes for cliff-walled, unnatural terrain and confused rivers. Real land is smooth: always blur your heightmap (the Gaussian Blur step above), keep the transitions gradual, and let the sea meet the land along a soft grey coast, not a black-to-white wall. If your rivers come out strange, the usual cure is **more blur**.

Compiling the land

Grow just this layer and read what came up:

```
realworld compile --layer geology
```

You will see the plates it settled on, the continents that stood above the sea, the mountain ranges where plates met, the sea level, the minerals distributed through

the crust, and the elevation statistics of the whole surface. Whether the land came from your seed or from a DEM, the report reads the same — the rest of the world neither knows nor cares which door you came in by.

> NOTE
>
> When you **materialize** the world — the full `realworld compile --materialize` from the introduction — the geology layer writes an actual heightmap image into the World system book alongside the prose. Even a seed-grown world hands you a picture of its terrain to keep, look at, and hand to a mapmaker.

Why the land is a root, not a decoration

Geology is the second physical layer, and like the sky it is a **root**: the climate layer reads its elevation to decide where the air cools and the rain falls, and the hydrology layer reads its slopes to run rivers downhill. A mountain range is not scenery. It is a rain shadow, a watershed, a wall that a climate will pile weather against on one side and starve of it on the other. When you change the land — a new seed, a different DEM — you are not repainting a backdrop; you are re-deciding where it rains and where the rivers run, two chapters from now.

> ASK YOURSELF
>
> Does your world's **shape** matter to your story, or only its **climate**? For many tales the answer is climate: you need a frozen north and a burning south and a temperate middle, but the exact silhouette of the coast is the story's to discover — so let the seed decide it. For others the map itself is a character: a particular strait, a sacred mountain, an island that must sit just so. Know which you are writing, because it tells you whether to trust the seed or reach for a DEM.

TRY IT

Change the `seed` in your `world.hjson` by a single digit and run `realworld compile --layer geology` again. The continents are not nudged — they are **different continents**, a whole new arrangement of land and sea from one altered number. Try three or four seeds and keep the world whose shape you like best. This is worldbuilding by audition: you are not drawing the land, you are choosing which grown land to keep.

EDIT · world.hjson

```
seed: 0x5eed2
```

WHAT YOU LEARNED

- The **geology** layer grows the land from your `seed` — plates, continents, mountain ranges, sea level, minerals, and elevation — the same land every time for a given seed.
- You do not draw coastlines; you set conditions and **read** what emerged. To control the shape directly, hand the layer a **DEM** via `geology.dem`.
- A **DEM (heightmap)** is an image encoding terrain height — the bring-your-own-map door — and its path is relative to your **project**.
- `realworld compile --layer geology` reports the land; a full `--materialize` writes an actual **heightmap image** into the World book.
- Land is a **root**, not decoration: climate and rivers are grown from its elevation and slopes, so changing the land re-decides the weather and the waters downstream.

CHAPTER 6

6

Weather and Water

You have a sky and you have land. The star climbs and sets, the seasons turn, the plates have shoved up mountains and drowned basins under a sea. What you do not yet have is weather — and without weather, land is only a relief map. In this chapter two more layers grow on top of the ones you have already built. First **climate**: where the world is warm and where it is cold, where the rain falls and where it never does. Then **hydrology**: what that rain does once it lands — the rivers it gathers into, the lakes it fills, the valleys it makes worth living in.

Neither layer is drawn. Both are consequences. That is the whole lesson of this chapter, so hold it from the first line: you did not decide where the desert goes. The

sun and the mountains decided, and the compiler merely worked out what they had already settled.

Climate: sunlight, then shelter

Climate is the marriage of two things you already have. From the astronomy comes **sunlight** — how much of it reaches each band of latitude, which the sky layer worked out for you as insolation. From the geology comes **shelter** — the mountains that stand in the way of the wind and wring the water out of it before it can travel further.

Start with the sun. Insolation is not spread evenly over a planet: the equator faces the star almost head-on and bakes; the poles catch the same light at a long, cold slant and it spreads thin. So temperature falls, in a smooth and utterly predictable way, from a hot middle to two frozen ends. You did not paint that gradient. It falls straight out of the geometry of a round world lit from one side — the same geometry that gave you the seasons.

Now add the mountains, and the tidy picture of "warm middle, cold ends" breaks into something far more interesting.

> TERM **Rain shadow**
>
> The dry country on the far side of a mountain range. Wind carrying moist air is forced upward as it crosses the range; rising air cools, cannot hold its water, and drops it as rain or snow on the near, **windward** slope. By the time the air spills down the far, **leeward** side it is wrung dry — so the land in the mountains' lee is starved of rain. A great many of the world's deserts sit in exactly such a shadow, and yours will too.

This is why a desert can sit a short ride from a green and dripping forest: the forest drinks the rain on the windward slope, and the desert lives in the shadow behind the peaks. Temperature from the sun, moisture from the winds and the mountains — put the two together across every cell of the world and you have climate.

The twelve biomes

The compiler does not hand you back a fog of numbers. It reads the temperature and the rainfall of each cell and names the **kind** of place they make.

> **TERM** **Biome**
>
> A category of living landscape defined by its climate — its temperature and its rainfall together — and by the community of life that climate supports. A biome is not a place but a **type** of place: "hot desert" and "taiga" describe land you would recognise anywhere on the world, on any continent, wherever the same warmth and the same rain happen to meet.

Inkhaven's climate layer sorts the world into twelve of them. Eleven cover the land, and one is the water that surrounds it:

- **ice_cap** and **tundra** — the frozen and near-frozen ends of the world;
- **taiga** and **temperate_forest** — the cold and the mild woodlands;
- **temperate_grassland** and **mediterranean** — open plains, and the dry-summer coasts of the middle latitudes;
- **cold_desert** and **hot_desert** — the rain-starved lands, whether frozen or baking;
- **savanna**, **tropical_seasonal**, and **tropical_rainforest** — the warm belt, by rising order of rain, from grassland with scattered trees to unbroken jungle;
- and **ocean** — everything below the sea line, the biome that frames all the rest.

The compiler groups the cells that share a biome into named **zones**, so you get back not a pixel map but a legible geography: a rainforest belt here, a band of cold desert there, the tundra fringing the ice. When you run the layer, you can read the world's weather as a list of places, not a spreadsheet.

> **NOTE**
>
> `realworld compile --layer climate` runs just this layer and reports the temperature, rainfall, winds, and biome zones it derived. It needs the astronomy and the geology beneath it; if you have those, the climate follows with no further input from you.

INSIGHT

Climate is not painted onto a world — it is **deduced** from it. The poles are cold because the light reaches them at a slant; the equator is warm because it faces the star; the deserts sit behind mountains because the rain fell on the windward slope. Every band of biome is the physics of sunlight and shelter, made visible. When your climate surprises you, it is not being arbitrary — it is telling you something true about the sky and the land you chose.

TRY IT

Open `world.hjson` and nudge `star.luminosity_solar` upward — say from `1.0` to `1.15` — then run `realworld compile --layer climate`. A brighter star pours more energy into the same latitudes: the warm belt widens, the ice retreats toward the poles, and more water evaporates to fall as rain. You have just made a warmer, wetter world without touching a single biome by hand.

EDIT · world.hjson

```
astronomy: {
  star: { luminosity_solar: 1.3 }
}
```

Hydrology: what the rain does next

The rain has fallen. Now follow it downhill.

Water does one honest thing: it flows to the lowest ground it can reach. The hydrology layer takes the elevation from your geology and the rainfall from your climate and simply lets gravity finish the job. The rule it uses is deliberately simple, and it is worth knowing by name.

> **NOTE**
>
> The compiler routes water by the **D8** rule: each cell on the grid looks at its eight neighbours and sends its water to the single lowest one. Follow those little arrows downhill from every cell and they braid together — a thousand trickles joining into streams, streams into rivers — until the water reaches the sea or a basin it cannot climb out of.

Trace those flows and a river network draws itself. Where many cells drain into one, you get a river; where a river has nowhere lower to go — a hollow with no outlet — the water pools into a lake. And every river gathers its water from a definite patch of land.

> **TERM** **Watershed**
>
> All the land that drains into one river or lake — every slope whose rain, by the downhill rule, ends up in the same water. Watersheds are the world's natural divisions: a ridge line is not just scenery, it is the boundary between two of them, sending rain one way to one river and the other way to another. Realms and roads and rivalries tend to follow these divides, because water does.

Once the water has found its paths, some places along them are plainly better to live in than others — and the compiler marks them. It flags the good sites as **settlement priors**: a **river_mouth**, where a river meets the sea and trade and fresh water and fish all come together; a **confluence**, where two rivers join and the traffic of both passes; a **fertile_valley**, where a river has laid down good soil across a sheltered floor. It does not build anything there yet. It only notices that people would.

> **NOTE**
>
> `realworld compile --layer hydrology` runs this layer on top of the geology and climate beneath it, and reports the rivers, lakes, watersheds, and the settlement sites it found. Those flagged sites are the seed the next chapter grows cities from.

PITFALL

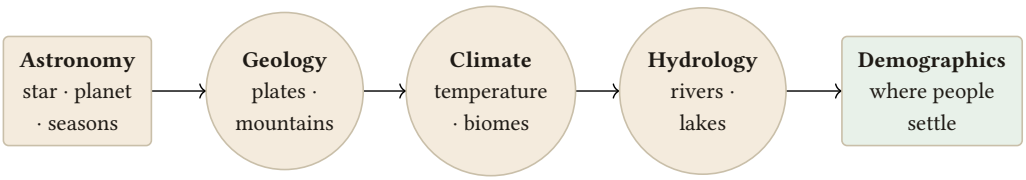
The commonest way a fantasy map betrays itself is a desert lying right against a rainforest with nothing to explain the seam. Real climate does not switch like a light. If two wildly different biomes share a border, the world owes you a **reason** standing between them — a mountain range casting a rain shadow, or a cold ocean current chilling one coast while the other bakes. Let the compiler derive your climate and this never happens by accident; if you later override a biome by hand, ask what physical thing draws the line, or a careful reader will ask it for you.

ASK YOURSELF

Look at the climate and rivers your world grew, and ask what its weather **does** to the stories you mean to tell there. Do seasonal monsoons rule the calendar of a farming people? Does one realm suffer a winter that never quite ends? Has a rising sea drowned a coast, leaving a city half in the water? Weather is not backdrop — it is pressure. The most memorable settings are the ones whose climate the characters cannot ignore.

The physical picture so far

Step back and look at the chain. The star set the temperatures; the plates raised the mountains; the mountains and the sun together made the climate; and the climate's rain, running down the geology's slopes, made the rivers and lakes and marked the places worth settling.



Each layer is a pure consequence of the ones before it: the sun shapes the climate, the climate carves the rivers, the rivers decide where the cities stand.

Every layer in that chain is a pure consequence of the ones before it. You have now built four of the five, and the fifth is already half-decided: the hydrology has quietly pointed at every river mouth and fertile valley where a town would want to stand. In the next chapter you let people arrive and take them up.

Declaring a river's course

The procedural rivers run on their own — the D8 rule braids them out of the slopes whether you say anything or not. But sometimes a particular river matters to your story, and you want it to run **just there**: from a spring you have named to a mouth on a coast you have already imagined. You can **assert** a course by adding a named river to a `hydrology` block, giving it a `from` cell and a `to` cell, and the world will check that the course you drew is one water could actually take.

EDIT · world.hjson

```
hydrology: {
  rivers: [
    { name: "The Aldermere", from: [12, 4], to: [40, 30] }
  ]
}
```

The `from` and `to` are grid cells — a source high in the land and a mouth where the river ends. You are not switching the procedural rivers off; they still run. You are pinning one named course on top of them and asking the world to vouch for it.

NOTE

The world runs a plausibility check on the course you declare. It warns if the source sits **below** the mouth — a river that would have to run uphill — or if the mouth is above the shoreline and so reaches no sea or lake. The warning does not stop the compile, and the procedural rivers run regardless; it simply tells you the course you asserted quarrels with the land beneath it, so you can move a cell or reconsider where your Aldermere truly rises.

WHAT YOU LEARNED

- **Climate** is derived, not drawn: sunlight by latitude sets temperature, and mountains casting **rain shadows** set where the rain falls — together they sort the world into **biomes**.
- The twelve biomes run from **ice_cap** and **tundra** through the temperate and dry lands to **tropical_rainforest**, with **ocean** framing them all; the compiler groups them into named zones you can read.
- **Hydrology** lets the rain run downhill by the **D8** rule into rivers and lakes, divides the land into **watersheds**, and flags the good settlement sites — **river_mouth**, **confluence**, **fertile_valley**.
- Run them with `realworld compile --layer climate` and `--layer hydrology`; each grows on the layers beneath it with no further input from you.
- A desert beside a rainforest needs a **reason** between them — a rain shadow or a current — or the world has quietly contradicted itself.

CHAPTER 7

7

Where People Settle

Here is the layer that turns a landscape into a world someone lives in. You have a sky, land, a climate, and a network of rivers running down to the sea. Now people arrive — and the question this chapter answers is not **where do you want your cities** but **where would people actually build them**. The two answers are rarely the same, and the difference is the whole point.

This is the **demographics** layer, and it is the last of the five physical layers. When it finishes, the physical world is complete: everything from the star's brightness to the population of a river-mouth port has followed, step by step, from the handful of conditions you set at the beginning.

People go where the land lets them

No one founds a city out of nothing. A settlement needs water to drink and to carry goods, soil that will grow more food than its people eat, and a position worth defending or trading from. Every one of those needs is something your earlier layers already worked out. The climate said where crops can grow. The hydrology marked exactly the sites where water and soil and traffic meet — the river mouths, the confluences, the fertile valleys. The demographics layer does not go looking for good ground. It was handed the good ground already.

> TERM **Demographics**
>
> The distribution of a world's people across its land — how many live where, in settlements of what size, doing what kinds of work. In Inkhaven, demographics is not a thing you author city by city; it is **derived** from the climate and hydrology beneath it, so that population follows the land's ability to feed and water it, the way real population always has.

The governing idea is that land can only support so many people, and the layer respects that ceiling everywhere.

> TERM **Carrying capacity**
>
> The number of people a given stretch of land can sustain, set by its climate and water — how much food it can grow, how reliably the rain comes, how easily goods can move across it. A rainforest delta and a cold desert do not have the same carrying capacity, and so they do not grow the same cities. The compiler reads capacity from the layers beneath it and lets settlements grow only as large as their ground allows.

So a fertile river valley in the warm belt can carry a great city; a patch of tundra can carry a hunting village and no more. You did not assign those sizes. The climate and the rivers assigned them, and the demographics layer merely counted.

A hierarchy, not a scatter

Real settlement is not a random sprinkle of dots. A country has a few great cities, more middling towns, and a great many small villages — and the sizes follow a startlingly regular pattern.

> TERM **Rank-size hierarchy**
>
> The empirical regularity that, across a region, a settlement's population tends to fall off in step with its rank: the second city is roughly half the largest, the third roughly a third, and so on down to the smallest hamlet. It means a believable world has **one** or two dominant centres, a modest tier of towns, and a broad base of villages — never a dozen equal metropolises, and never a capital with nothing between it and the wilderness.

Inkhaven's demographics layer builds exactly this shape. It ranks the settlement sites the hydrology flagged, seats the largest population where the land can best support it, and steps the rest down through **city**, **town**, and **village** in the familiar falling curve. The result reads like a real country: a handful of names you would know, a scattering you might, and a long tail of places too small to mark. To each settlement it also attaches **role archetypes** — the kinds of people a place of that size and setting would hold, a river port trading differently from an upland market town — so the settlement list is already peopled, not just counted.

> NOTE
>
> `realworld compile --layer demographics` runs this layer on top of the climate and hydrology beneath it, and reports the settlements it grew: each one's class (city, town, or village), its population, its site, and its role archetypes. It invents no new ground — every settlement stands on a site the hydrology already marked.

INSIGHT

Cities are not **placed** — they **grow**. By the time this layer runs, the choosing is already done: the hydrology pointed at every river mouth and fertile valley, and the climate said how many mouths could feed a city. Demographics only takes those sites, ranks them, and lets population settle onto them in the shape real populations take. When you see where your largest city landed, you are not seeing a decision you made here — you are seeing the sum of every layer beneath it, come to its natural conclusion.

TRY IT

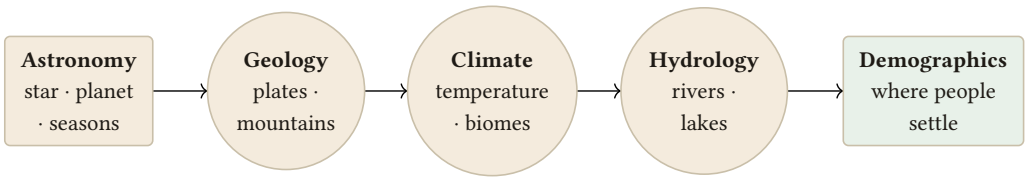
Run `realworld compile --layer demographics` and read the settlement list from the top. Notice the shape of it: the sharp drop from the first name to the second, the widening tier of towns, the long tail of villages. Trace the largest few back to their sites — you will find them sitting on the river mouths and fertile valleys the hydrology flagged a chapter ago. Nothing here was placed by hand, and yet it reads like a country.

ASK YOURSELF

Look at where your civilization clustered — and, just as hard, at where it did **not**. Every world has empty quarters: a desert interior no city will ever hold, a frozen north with a single lonely outpost, a mountain wall with people crowded on one side and nobody on the other. Those blanks are not failures of the map. They are where the frontier stories live, the crossings that cost something, the places a character disappears into. Where does your world leave room to be lost?

The physical world is complete

This is a threshold, so mark it. With demographics grown, all five physical layers are in place, and each one fell out of the last:



Each layer is a pure consequence of the ones before it: the sun shapes the climate, the climate carves the rivers, the rivers decide where the cities stand.

The star set the seasons; the plates raised the land; the sun and the mountains made the climate; the climate's rain carved the rivers; and the rivers decided where the people stand. From a seed and a few choices you now have a **place** — a whole physical world, warm where it should be warm, wet where it should be wet, peopled where the land invites people and empty where it forbids them, and consistent in every corner because none of it was guessed.

What that place does not yet have is **time** and a **people** in the fuller sense — a past that predates page one, and nations and cultures and tongues who carry it. Those are the work of the parts ahead: Part III gives your world a history, and Part IV gives it civilizations. The land is built. Now you fill it with lives.

WHAT YOU LEARNED

- **Demographics** is the last physical layer, derived from climate and hydrology: people settle where the land can feed and water them, never in a random scatter.
- **Carrying capacity** caps how large a settlement can grow from its climate and water; a fertile delta carries a city, a tundra a village.
- Settlements fall into a **rank-size hierarchy** — a few cities, more towns, many villages, with role archetypes — grown from the sites the hydrology already flagged.
- Cities are not placed but **grown**: run `realworld compile --layer demographics` and the largest centres land on the river mouths and valleys chosen a layer earlier.
- With this layer done the **physical** world is complete; Parts III and IV give it time and people.

PART III

Giving the World a Past

CHAPTER 8

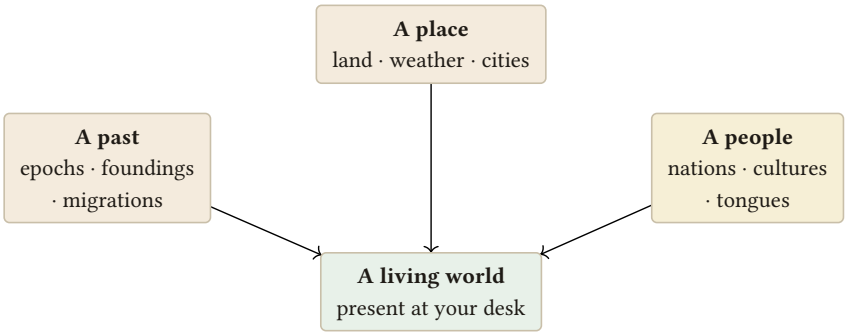
8

Giving the World a Past

Until now you have built a **place**: a sky, a coastline, weather that behaves, rivers that run downhill, cities that stand where people would build them. It is a fine place. But if a reader walked into it today, they would feel something was missing, even if they could not name it. The cities have no ruins under them. The oldest port has no story about why it is the oldest. Nothing has ever **happened** here. A place with no yesterday is a stage set — convincing from the front row, hollow the moment anyone opens a door at the back.

This chapter opens the **time dimension** of your world. It is the first of the parts that turn a map into a history: not more geography, but a past that predates page one — an age of founding, an age of growth, a rise and fall of realms, people moving

from one land to another and taking their quarrels with them. And you will not invent it from a blank page. Your world already contains its own history, folded into where the cities sit and how large they grew. You only have to read it out.



A place is only the beginning. A world becomes alive when it also has a past, a people — and a presence while you write.

The past is already implied

You have not written a single date, yet your world already tells you which of its settlements is oldest. Think about how the demographic layer worked: cities grew where the land supported them — river mouths, fertile valleys, the safe confluence of two waters. The very best sites were claimed first, because they are the sites people reach for first. So the largest, best-sited city is almost certainly the **oldest** one; the modest town on the marginal river arrived later, when the prime ground was already taken.

That is a chronology hiding in plain sight. The `realworld history` command reads it. It sorts your settlements by how good their site is and how large they grew, infers a **founding order** from that ranking, and lays those foundings out along a single axis of time — measured backward from now.

NOTE

The past is **generated** first: `realworld history` reads it out of the world you have already compiled, taking no input but the settlements themselves. To change what it infers, change what it grows from — a different `seed`, or a land that settles its cities differently, tells a different story. But you may also **declare** events of your own, in a `history:` block: they merge into the generated chronology, sort into their epoch by year, and — where they name a Place you have accepted — adopt onto the story Timeline alongside the inferred ones. The world still takes the generated past as its base; your declarations are added to it, not swapped in for it. (The one date you **do** always set is the world's calendar, back in the `astronomy` block — that is what the years below are counted in.)

TERM **Chronology**

The ordering of events along time — what came before what, and how long ago. Your world's chronology is inferred from its settlements: the better a city's site and the larger it grew, the earlier it was likely founded. The command turns that ranking into dated events on the world's own calendar, the one you derived from its astronomy back in the calendar chapter.

INSIGHT

A world without a past is a stage set; give it one and the present acquires weight. The same tower is a different thing when it is nine hundred years old, when it was raised on the ashes of the realm before it, when it is the last building of a city three others have since surpassed. History is not backstory filed away — it is the mass that makes the present feel like it **cost** something to arrive at.

Naming the ages

A list of foundings dated in raw years is a spreadsheet, not a history. What makes a past feel like a past is that it comes in **eras** — stretches of time with a character

of their own, so that "the Founding Age" already tells you something before you have named a single event in it. So `history` divides your world's inferred past into three epochs.

> TERM **Epoch**
>
> A named stretch of a world's history with its own character. This command produces three, in order from oldest to most recent: the **Founding Age**, when the first and best-sited cities were established; the **Age of Expansion**, when people spread to the lesser sites and realms grew to their present reach; and the **Present Age**, the recent past that leads up to now. Every generated event falls into one of the three.

> TERM **The present (year 0)**
>
> The instant your story's "now" sits at. All of the world's history is dated in years **before the present** — the founding of the oldest city might land at year 900, a realm's fall at year 210 — so that year 0 is always "when the reader arrives." Anchoring on the present, rather than an absolute epoch, keeps the history readable: a number is immediately a distance into the past.

What happened, not just when

Foundings alone are a skeleton. Onto it, `history` hangs **events** — the things a historian would actually recount. Two kinds fall out naturally from what your world already contains. First, the **rise and fall of realms**: where clusters of cities imply a polity (the next chapter's subject), the command dates its rise in one epoch and, for some, its fall in a later one — the older the world, the more it has buried. Second, **migrations between biomes**: a people leaving the cold grassland for the temperate forest, the desert's edge for the river valley, each dated and given a direction. Movements like these are the engine of real history — a drought empties one region and fills another — and they leave exactly the kind of hook a story reaches for.

ASK YOURSELF

What happened before page one? Something did. Which realm fell, and is its fall still resented? Who arrived from somewhere else, and are they still called newcomers by the people who were already here? Name the one event, before your story opens, whose consequences your protagonist lives inside — then see whether the generated history offers you something close to it.

TRY IT

Run `realworld history`. Read it top to bottom: the three epochs, the founding dates of your cities, the realms that rose and the ones that fell, the migrations and their directions. Do not adopt anything yet — just read it as a chronicle of the world you have been building, and notice which single line makes you want to write a scene.

Reading it, keeping it, adopting it

`realworld history --json` prints the whole chronology as structured data, for tools and scripts. `realworld history --materialize` writes it down as a History chapter in your World book, so the epochs and events live beside the rest of the compiled world, readable and searchable. Both of those record the world's **proposal** — they do not touch your manuscript.

Adoption is the separate, deliberate step. Alongside its chronicle, `history` prints ready-made command lines — `inkhaven event add …`, one per event — that place a chosen event onto your story's **Timeline**, the system book your scenes are dated against. You copy the lines for the events you want, and only those. The fall of the old realm, if it matters to your plot; the great migration, if your people are its descendants; nothing you do not need.

> **NOTE**
>
> The world proposes; you adopt. `history` never writes an event onto your Timeline on its own — it hands you the `inkhaven event add …` lines and stops. You decide which foundings, falls, and migrations are real for **your** story, and paste in only those. The chronicle is a generous draft of the past; the author edits it down to the history the book actually needs.

A last point of craft: the history is inferred, not decreed. If your plot needs the youngest city to be the ancient one — a colony that outgrew its motherland, a capital deliberately founded on empty ground — override it. Rename an epoch, redate a fall, drop a migration that does not fit. The generated past is a strong first draft precisely because it is consistent with the geography; but, as always, the author has the last word over which yesterday the world remembers.

Declaring your own events

Inference is generous, but it cannot know the one event your book is built on — the treaty, the landing, the betrayal that has no settlement to imply it. So you may write events straight into `world.hjson`, in a `history:` block, and the world will fold them into the chronology it generates:

> **EDIT · world.hjson**

```
history: {
  events: [
    {
      year: -1200
      title: "The First Landing"
      epoch: "Founding Age"
      places: ["Karthage"]
      description: "The seafarers make landfall."
    }
  ]
}
```

Each field earns its place. The `year` is dated the same way the generated events are — years **before the present**, so `-1200` is twelve centuries ago (the sign is simply

the direction into the past). The `epoch` is optional: name it and the event files under that age; leave it out and the world infers the epoch from the year, dropping the event into whichever of the three ages the date falls in. The `places` list links the event to Places you have already accepted — that link is what lets a declared event adopt onto the Timeline, exactly as a generated one does; an event with no place stays in the chronicle but has nowhere to sit on the story's dated axis. The `title` and `description` are yours to phrase as a historian would.

> **NOTE**
>
> A declaration is checked for plausibility, not obeyed blindly. The world warns if a `year` lands **after** the present — an event that has not happened yet — or so far back that it predates recorded history, and it warns if you name an `epoch` that does not contain the `year` you gave (an event you filed under the Founding Age but dated to last week). The warning does not delete your event; it tells you the past you wrote does not line up with the past the world knows, and leaves the reconciling to you.

The past as a rising tide

`realworld history` gives you the events — the foundings, the rises and falls, the migrations — dated on the world's own calendar. But sometimes what you want is not the events but the **shape** of the past: how big the world was as it grew, how many realms stood at once, when the tide of settlement was rising and when it turned. For that there is `realworld chronicle`:

```
inkhaven realworld chronicle
```

It reads the same compiled history, but instead of a list of events it reports, at the close of each epoch, **how far the world had grown by then** — how many settlements of each kind, how many people they held, how many realms were standing. You watch the Founding Age hold a handful of towns, the Age of Expansion swell with cities, and the Present Age settle into the world your story opens in, a realm or two having risen and waned along the way.

NOTE

The chronicle invents nothing. It is the **same** deterministic history, read as a running total rather than a timeline — a presentation, not a simulation. The world does not model growth year by year; it shows you the state its own chronology already implies at each turning of the age. When you need a felt sense of scale over time — for a prologue, a founding myth, a fallen empire in the backstory — the chronicle is where you read it.

WHAT YOU LEARNED

- A world with no past is a stage set; a **chronology** — inferred from where the cities sit and how large they grew — gives the present its weight.
- `realworld history` reads the founding order out of your settlements, divides the past into three **epochs** (Founding Age, Age of Expansion, Present Age), and dates everything in years **before the present (year 0)**.
- It generates events a historian would recount — the **rise and fall of realms** and **migrations between biomes** — on the world's own calendar.
- `--json` and `--materialize` record the proposal; the printed `inkhaven event add …` lines let you **adopt** chosen events onto the story Timeline — you pick which yesterdays are real.

PART IV

Giving the World a People

CHAPTER 9

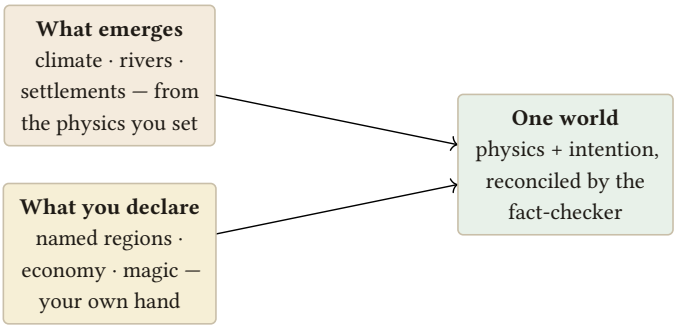
9

Nations

Your world now has a place and a past. This part gives it a **people** — and the first thing a people needs is not a language or a religion but a **map of power**. Who rules this stretch of coast? Whose banner flies over that cluster of river towns? Where does one realm end and the next begin, and along which border do the two of them glare at each other? Before your world has a single named character, it has **powers** — and they are already latent in the map you built.

Look again at your settlements. They are not scattered evenly. They gather: a great port with a ring of smaller towns leaning on it, a fertile basin with its market city and the villages that feed it. Those clusters are not an accident of the rendering — they are where the land concentrates people. And a concentration of people

around a dominant city is very nearly the definition of a realm. The `realworld polities` command reads those clusters and draws the nations your geography already implies.



A world has two hands: the physics that falls out on its own, and the names and rules you set by intention. Both are honoured.

From cities to countries

`polities` works outward from the biggest cities. It finds the largest settlements — the natural seats of power — and gathers the smaller settlements around each one into a group, by which capital they sit nearest and lean toward. Each group becomes a nation.

> TERM **Polity**
>
> An organised body of people under one rule — a kingdom, a republic, a city-state, a confederation. In this command a polity is a **cluster** of settlements gathered around one dominant city. The word stays deliberately neutral about the **form** of rule: the command gives you the shape of the power (these cities, together, under that seat); you decide whether it is a crown, a council, or a merchant league.

> TERM **Capital**
>
> The dominant settlement a polity forms around — the largest city in the cluster, the seat its towns and villages lean toward. The command picks the capital first, by size, and then builds the realm around it. When you rename a polity, its capital's name is usually the place to start: realms are very often named for their chief city, or their chief city for the realm.

For each polity the command gives you three things. A **generated name**, so the realm is something you can say aloud rather than "the cluster around city 4." A **summed population** — every settlement in the cluster added up — so you know at a glance whether this is a great power or a minor one. And its **sphere of influence**: the reach of the capital, the ground its pull covers.

> TERM **Sphere of influence**
>
> The territory a capital's pull effectively covers — the settlements that fall to it rather than to a rival seat. Spheres are how the command decides where one realm ends and the next begins: a town on the seam between two capitals is claimed by the nearer, stronger pull. Where two spheres press against each other is exactly where you will want a contested border in the story.

Who hates whom

A collection of nations sitting politely side by side is a map, not a politics. What turns neighbours into a **world** is that they have **opinions** about one another. So `polities` seeds each pair of realms with a relation — **allied**, **rival**, or **neutral** — deterministically from the world seed, so the same world always has the same web of friendships and grudges.

These relations are not arbitrary decoration; they are the raw material of every conflict your story might use. An alliance is a promise that can be broken at the worst moment. A rivalry is a war waiting for a pretext, or a cold trade dispute, or a marriage that would end it. Even neutrality is a fact with weight: the realm that stays out is the one both sides are courting. Read the relations as a list of the tensions your world hands you for free.

INSIGHT

A nation is not a coloured shape on a map. It is settlements **plus a story of who rules whom and who resents it** — the population that gives it weight, the capital that gives it a will, and the relations that give it something to want and something to fear. Draw the borders and you have a map; name the grudges and you have a plot.

ASK YOURSELF

Who are the powers of your world — and who hates whom, and **why**? Not "these two are rivals," but the reason: an old war never quite settled, a river both need, a throne two families both claim, a faith one exported and the other refused. The command hands you **that** they are rivals; the richest work you will do is deciding the **because**.

TRY IT

Run `realworld polities`. Read the realms first — their names, their capitals, their populations — and get a feel for which are the great powers and which are the minnows. Then read the relations: the allies, the rivals, the neutrals. Find the one rivalry that most makes you want to know its history, and write a sentence explaining why those two realms cannot stand each other.

A first draft of a political map

Keep this grounded. The realms `polities` gives you are not a decree about your world's politics — they are the political map your **geography** implies, drawn consistently from where the cities actually are. That makes them a strong first draft and a poor final word.

So treat them as clay. The generated names are placeholders for names in your own tongue — and when you reach the culture chapter, the language each people speaks will give you far better ones. The borders follow population, but your story may need a realm that punches above its size, a capital in exile, a federation the

map would have split in two. Redraw them. Merge two clusters the history joined; split one a civil war tore apart.

> **NOTE**
>
> The world proposes the realms; you rule them. Nations are **generated** first: `realworld polities` reads them out of where your cities already sit and seeds their relations from the world seed, and it writes nothing into your manuscript. But you may also **declare** realms of your own, in a `nations:` block. Each declared realm pins a named country to a capital cell; the world clusters the rest of the settlements around your seats just as it does around the generated ones, and where a declaration and the inference disagree, your declared **names** and **relations** win. You still rename, redraw, and rewrite the grudges as the story needs, exactly as you accept or reject the places the world proposes. The map is the world's; the politics is yours.

Declaring your own nations

Redrawing the generated realms by hand is one way to get the politics you want. Declaring them outright is another, and it is the one to reach for when a realm matters enough to fix in the world itself — a capital your plot has already chosen, an old enmity the story turns on. You write realms into `world.hjson`, in a `nations:` block, and the world builds around them:

EDIT · world.hjson

```
nations: [
  {
    name: "Karon"
    capital: [48, 19]
    relations: [
      { with: "Serai", stance: "rival" }
    ]
  }
]
```

The `capital` is a **map cell** — an `[x, y]` coordinate on the world grid, not a city name — and the world seats the realm at the settlement nearest that cell, then clusters the surrounding towns around it exactly as `polities` does for a generated seat. The `relations` list overrides the seeded web pair by pair: each `{ with, stance }` names another realm and the stance this one takes toward it — `rival`, `allied`, `neutral` — replacing whatever the seed rolled for that pair. Everything you do not declare is still inferred, so a single named realm can sit inside a map of otherwise generated neighbours.

> **NOTE**
>
> The coordinate is checked against the land. If a `capital` cell sits far from any settlement — out in the wilderness, where no city stands to be its seat — the world warns you: a realm needs people to rule, and a capital in empty country is almost always a coordinate typed a few cells wrong. As ever the warning does not overrule you; it points at the seat with no city under it and lets you move it.

Trade follows the peace

Relations are not only grudges; they are also the shape of commerce. Realms that are **not** rivals trade with their nearest neighbours, and `realworld trade` reads that straight out of the map:

```
inkhaven realworld trade
```

Each realm links to the closest few realms it is not at odds with; the route is a land road if the capitals sit inland, a sea lane if they sit on a coast. Rivals never trade — a border of suspicion is a border with no road across it. What comes back is the commercial geography of your world: which realms are bound by exchange and which are cut off, before you have written a single caravan or embargo.

NOTE

The trade layer is **connectivity, not economics**. It tells you that Karon and Serai are linked by a sea lane and that neither trades with the realm they both fear — it does not invent prices, goods, or volumes, which are yours to write. When you render the map with `realworld map`, the routes are drawn as roads between the realm capitals, so the web of trade is there to see.

WHAT YOU LEARNED

- Your settlements already cluster around dominant cities; `realworld polities` reads those clusters into **nations**, each a **polity** built around its **capital**.
- Each realm gets a generated name, a summed population, and a **sphere of influence** — and where two spheres meet is where a contested border belongs.
- The command seeds every pair of realms with a relation — **allied, rival, or neutral** — deterministically, handing you the tensions your story can use.
- A nation is settlements **plus a story of who rules whom and who resents it**. The realms emerge from where the cities are; the author renames and re-draws as the story demands.
- `realworld trade` turns the relations into a **trade network** — each realm linked to its nearest non-rivals by land road or sea lane, drawn on the map. Connectivity, not economics: which realms are bound, and which are cut off.

CHAPTER 10

10

Cultures and Tongues

In the last chapter you clustered your settlements into nations — polities with capitals, populations, and seeded relations of alliance and rivalry. But a nation is not only a border on a map and a count of heads. A people has a **character**: things it holds sacred, a way of carrying itself that its homeland taught it, and a tongue in which it names its children and its towns. Two realms with identical populations can be nothing alike — a desert khanate and a forest confederacy are different peoples before either of them has done a single thing in your story.

The command that gives each of your polities that character is `realworld culture`. It reads the nations you already have and, for each one, proposes three things at once: an **ethos** drawn from the land its capital stands on, a **belief** its

people live by, and a **language profile** — a compact sketch of the tongue they speak. Together these turn a shaded region into a people you could sit down and share a meal with.

> TERM **Culture**
>
> The shared character of a people — how they see the world, what they honour, and how they speak. In the World system a culture belongs to a **polity**: one nation, one culture, generated from where that nation lives and grown into something you can name and describe. It is the human layer that sits atop the physical world, the way demographics sits atop climate.

An ethos grows from the ground

The first thing `realworld culture` gives a people is an **ethos**, and it does not pull it from nowhere. It reads the biome of the capital — the land your largest city actually stands on — and lets that land shape the temperament. This is the oldest idea in worldbuilding, made mechanical: people are marked by the country that fed them.

> TERM **Ethos**
>
> The characteristic temperament and values of a people — how they meet the world. A desert people, schooled by scarcity, may prize hospitality, endurance, and the sacred trust of water shared; a forest people, hemmed in and sheltered, may prize kinship, patience, and a wariness of what the trees hide. The ethos is the biome's fingerprint on the human heart.

Because the ethos follows from the capital's biome, it is not arbitrary and it is not detachable. Move a nation's heart from the savanna to the taiga and its ethos would move with it. That is the point: your peoples differ from one another for the **same reason your climates do** — they grew in different places. You built those places in Part II without thinking about anyone living in them; now the land pays you back by telling you who its people became.

INSIGHT

A desert people and a forest people are not two colours on a legend — they are two answers to the question "what does this land ask of the people who live here?" Let the biome you already built decide the ethos, and your peoples will differ the way real peoples do: because their homelands differed first.

A belief they live by

On top of the ethos, `realworld culture` proposes a **belief** — the frame through which a people explains the world and orders its life. It might be reverence for ancestors, a pantheon tied to the seasons your astronomy already fixed, a single distant sky-god, a cult of the great river that feeds the capital, or a philosophy with no gods in it at all. The belief gives you the thing your characters swear by, build temples to, go to war over, and quietly stop believing in — the material of a hundred scenes.

ASK YOURSELF

For each of your peoples: what do they hold **sacred** — and can you trace it back to their homeland? A people of the river mouth might worship the flood; a people of the long dark winter might reckon time by the return of the light. If a belief could belong to any people anywhere, it is not yet rooted. Ask what **this** land would teach **these** people to fear and to honour.

NOTE

Every world has the same handful of social roles — someone farms, someone fights, someone tends the sacred — but each people **names** them differently, and `realworld culture` now shows you how. The common "priest" becomes a **keeper of the founding dead** in one realm and a **keeper of the sky-pantheon** in another; the "warrior" wears the realm's ethos, the "farmer" the realm's ground — an ice-tiller here, a temperate-tiller there. They are the world's roles, spoken in each realm's own tongue, ready to drop into a scene.

A language profile — a proposal, not a language

The third thing a culture carries is a **language profile**: a short typological sketch of the tongue the people speak. You will see it written in a compact line like `SOV · agglutinative · tonal` — three coordinates that place a language in the space of all human languages.

> TERM **Language typology**
>
> A description of a language by its structural type rather than its words, along three axes: **word order** (the default arrangement of Subject, Object, and Verb — SOV, SVO, VSO, and so on), **morphology** (how words are built — **isolating**, one morpheme per word; **agglutinative**, morphemes glued in transparent chains; **fusional**, morphemes fused so one ending carries many meanings), and **sound** (the phonological flavour — **tonal**, where pitch distinguishes words, versus non-tonal, and the general texture of the consonants and vowels). Three coordinates, and you already know a great deal about how a tongue would feel.

Here is the crucial thing, and it is the reason this chapter exists at all: the language profile is **not a language**. It is a **proposal** — a specification, a brief handed to a builder. `realworld culture` also gives you a small **naming sample**, a handful of names in the proposed style so you can hear its music, but it does not invent a phonology, a lexicon, or a grammar. It tells you what kind of tongue this people speaks. It does not speak it.

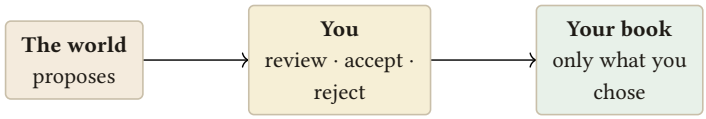
> NOTE
>
> The naming sample is illustrative, not canonical. It shows you the flavour of the profile — the shape of the syllables, whether the tongue runs to long agglutinative chains or short isolating stems — so you can decide whether that is the language you want this people to have before you build it for real.

The sample is one name; a realm has many towns. `realworld name` extends the sample into a whole gazetteer: it proposes a name for every settlement in its realm's phonic style, so a realm's towns share a family sound — a Karon coast of Torvelras and Velkaeths, a Serai valley that sounds nothing like it — instead of the generic

placeholder names. They are proposals in the world's style; adopt one when you accept its Place, or, once you have realised the realm's tongue in the ConLang suite, name in the real language instead.

The bridge to the ConLang suite

To turn the profile into a real language you cross from the World system into Inkhaven's **ConLang suite** — its constructed-language workshop — with `inkhaven language`. That is where the profile stops being a sketch and becomes a tongue: a sound inventory, rules for how syllables combine, a lexicon that actually contains words, a grammar that inflects them. You hand the ConLang suite the profile as its starting brief — `SOV · agglutinative · tonal` — and it helps you build the phonology, the vocabulary, and the grammar that profile describes. Then, and only then, does the culture **speak** an invented tongue rather than merely gesture at one.



The world never writes into your book on its own. It proposes; you decide. The author always has the last word.

You do not have to carry the profile across by hand. Run `realworld propose-language` and the world reads every culture's profile and naming sample and proposes a **language** per people — waiting in the same queue as your Places and rulers. Accept one and the world scaffolds a fresh language book in the ConLang suite for you: its Meta, Phonology, Grammar, Dictionary, and Sample-texts chapters, with a `world-profile` brief already written in — the word order, the morphology, the sound, and the naming sample the world proposed. The handoff is done; what remains is the making, which was always yours.

This is the **World × ConLang bridge**, and it is the reason the two systems exist side by side rather than as one. The World system is very good at **where a language sits** — it can read the biome, the isolation of a mountain valley, the trade contacts of a river port, and propose a plausible typology for a people who grew up there. It is not in the business of **inventing the language**; that is craft of a different kind, and the ConLang suite is built for exactly it. So the world proposes the profile, and the ConLang suite realises it. Neither could do the whole job alone; together they

take you from "these people live in a cold forest" all the way to a named child in a spoken tongue.

> **INSIGHT**
>
> A people is three things at once: **where they live**, **what they believe**, and **how they speak** — and the world proposes all three. The ethos comes from the land you built, the belief from the ethos, and the language profile is a brief the ConLang suite turns into a real tongue. Build a people the way you built a climate: let each layer follow from the one beneath it.

Remember, too, that all of this obeys the same authority discipline as the rest of the world: `realworld culture` **proposes** an ethos, a belief, and a profile. You accept what rings true, rewrite what does not, and discard what belongs to some other people. The world is handing you a strong first draft of a nation's soul — never the last word. The last word, here as everywhere, is yours.

> **TRY IT**
>
> Run `realworld culture` and read one nation's card end to end — its ethos, its belief, its language profile, and the naming sample beneath it. Trace the ethos back to the capital's biome: does it fit the land you built? Then take the profile line — say `SOV · agglutinative · tonal` — and open `inkhaven language` with it as your brief. You have just crossed the bridge from a world that knows **where** a language lives to a suite that will help you **speak** it.

Pinning a culture

`realworld culture` generates a culture for every nation, and much of the time its proposal is the right one — the land it read is the land you built. But now and then you have a people already alive in your head, with a character no biome would have guessed, and you want to hold it fast against anything the generator might say. You can **pin** your own culture with a `cultures:` block, matched to a nation by name. Whatever you declare — the **ethos**, the **belief**, the **language** profile — overrides the generated one for that nation.

EDIT · world.hjson

```
cultures: [
  {
    nation: "Karon"
    ethos: "mercantile and open"
    belief: "the tide-mother"
    language: "SOV · agglutinative · tonal"
  }
]
```

You do not have to pin all three; declare only the fields you mean to fix, and the generator still fills the rest. A pinned culture is you taking the last word early — telling the world **this** people is settled, build the others around it.

NOTE

The world still checks a pinned culture for plausibility. It warns if a **seafaring** ethos is pinned to a dry, inland capital that touches no coast — a people of the tides with no tide to speak of — or if the `nation` you named does not exist among the polities you clustered. The warning does not overrule you; the author always wins. It only asks whether the people you pinned truly belongs to the land you gave them.

The realm's ruler, into your cast

A realm implies a person at its head, and that is the one individual the world can hand you without inventing a life out of nothing. `realworld propose-rulers` reads your polities and proposes, for each, a **ruler** — a Character named in the same style the world uses for its towns, rooted in that realm's ethos and belief:

```
inkhaven realworld propose-rulers
```

Each proposal waits in the same queue as your Places and Mythology entries. Accept one and it becomes a paragraph in the Characters book — a short, factual stub: who they rule, from what capital, over how many people, and what their people hold sacred. It is a starting point, not a character. The world will not give

you their wound, their want, or the choice that breaks them; those are the work only you can do. What it gives you is a name to rename and a throne to fill, so your cast begins already standing on the map you built.

> **NOTE**
>
> The world proposes **one** ruler per realm and nothing more — it does not model individuals, so it will not populate your book with courtiers, rivals, or kin. That restraint is deliberate: a generated crowd of empty names is a burden, not a gift. One rooted figure per realm, offered for you to accept or ignore, is the honest limit of what a deterministic world can know about people.

> **WHAT YOU LEARNED**
>
> - `realworld culture` gives each polity a **culture**: an **ethos** drawn from the capital's biome, a **belief** the people live by, and a **language profile**.
> - The **ethos** follows from the land — a desert people and a forest people differ for the same reason their climates do, because their homelands differed first.
> - A **language profile** (`SOV · agglutinative · tonal`) is a typological sketch — word order, morphology, and sound — plus a naming sample. It is a **proposal**, not a finished language.
> - You realise the profile in Inkhaven's **ConLang suite** with `inkhaven language`, which builds the real phonology, lexicon, and grammar — the **World × ConLang bridge**, and the reason the two systems exist together.
> - As always, the world **proposes** a people's character; the author decides. A people is where they live, what they believe, and how they speak.
> - `realworld propose-rulers` offers one ruler per realm as a **Character** stub, named in style and rooted in the culture — a name to rename and a throne to fill. The world models no one else; the rest of the cast is yours.

CHAPTER 11

11

Life

Stand back from your map for a moment and look at what you have. A sky that turns through real seasons, mountains raised by plates, rain that runs down them into rivers, cities where the land could feed them, a history that predates page one, and peoples with an ethos, a belief, and a tongue. It is a great deal. And yet, if you look closely at your forests, they are empty. The taiga is a colour. The savanna is a word. Nothing walks there.

That is what `realworld ecology` is for. It takes each land biome you grew back in the climate layer and fills it with **life** — plausible flora and fauna, and a keystone animal that holds the place together. It is the last thing the physical world needs before it stops being a diagram and starts being somewhere a story could happen.

TERM **Ecology**

The web of living things in a place, and how they hang together — what grows, what eats what, and which creatures the whole system leans on. In the World system, ecology is derived from the **biomes** you already built: the climate decides what could live there, and `realworld ecology` populates it. Life is not scattered at random over the map; it follows the climate, exactly as the rivers and the cities did.

Archetypes, not a field guide

Open the ecology and you will notice at once what it does **not** give you. There are no Latin names, no real species, no "red-tailed hawk" or "Scots pine." Instead you get **tall conifers** and **pack hunters**, **broad-leaved canopy trees** and **grazing herd-beasts**, **ambush predators** of the reeds. This is deliberate, and it is the right choice for an invented world.

TERM **Archetype**

A generic, evocative kind of living thing named by its role and form rather than by a real species — "tall conifers," "pack hunters," "burrowing grazers." An archetype tells you what a creature **is like** and what it **does** without pinning it to Earth. It is a slot in the ecology that you, or your story, can later fill with a named beast of your own.

Real species would be a mistake here. The moment you write "wolf" your reader imports the whole of Earth's wolf — its range, its myths, its documentary footage — into a world that is not Earth. An archetype keeps the door open: **pack hunters** of your northern forest can be whatever your world needs them to be, sung about in whatever way your peoples sing, feared in whatever way your winters teach. The archetype is evocative **because** it is generic. It gives you the shape of a living thing and leaves the naming to you.

PITFALL

Do not reach for a real Earth species to make an ecology feel richer. “Bison,” “cedar,” “jackal” each drag their whole real-world baggage onto your map and quietly make your world **less** your own. Keep the archetype — **grazing herd-beasts**, **tall aromatic evergreens**, **lean scavengers** — and name the particular creature only when your story actually needs one.

A keystone for each biome

For every land biome, `realworld ecology` names one **keystone species** — the animal the biome would fall apart without. This single choice does more to fix a place in the mind than a whole list of flora, because it tells you where the weight of the ecosystem sits.

TERM **Keystone species**

A creature whose presence holds a whole ecosystem in shape — remove it and the system reorganises or collapses. The grazer whose herds keep a grassland from turning to scrub; the great predator that keeps the grazers moving; the river-builder whose dams make the wetlands. Naming the keystone tells you, in one stroke, what a biome is **really about**.

The keystone is also a gift to your storytelling. It is the beast your herders follow, the predator your children are warned about, the animal on a realm’s banner, the migration your calendar quietly turns around. A biome with a named keystone is no longer scenery — it has a protagonist among its animals, and your human characters live in relation to it.

INSIGHT

Fill the biomes with life, or your map is just coloured regions. And you do not invent that life freely — the ecology **follows from the climate you already built**, the same way the rivers and the cities did. The cold forest gets its conifers and its pack hunters because it is a cold forest; change the climate and the life changes with it. Life is the last consequence of the physics you set at the very start.

What your people fear in the dark

An ecology is not a nature documentary running quietly behind the story. It is the world your characters move through — the meat on their table, the pelts on their backs, the tracks they read, the eyes they cannot see at the edge of the firelight. When you know what walks in your forests, you know what your hunters hunt, what your farmers fence against, what your myths are about, and what a traveller listens for on a night road. The ecology hands your peoples their fears and their harvests in the same breath.

ASK YOURSELF

What **walks in your forests** — and what do your people **fear in the dark**? Name the keystone of the biome your protagonist grew up in, and one archetype they would have been taught, as a child, to run from. If nothing in your world is dangerous and nothing is precious, your world has scenery but no **life** — no stakes an animal could raise. Give each land somewhere a thing worth fearing and a thing worth keeping.

Like everything the world proposes, the ecology is a draft in your hands. Accept the archetypes that fit, rename the keystone if a better beast comes to you, and let the ones you do not need stay quietly in the background where they belong.

TRY IT

Run `realworld ecology` and read the entry for the biome your main character calls home. Note its keystone animal and two or three archetypes. Now write one sentence of prose that puts your character in that place with one of those creatures in it — the herd on the horizon, the predator's cry at dusk. That sentence is the moment your map became a **world**.

Pinning a biome's life

`realworld ecology` fills every biome, and its archetypes are a generous first draft. But a biome can be the beating heart of your story — the desert your whole book crosses — and you may already know exactly what grows and hunts there. You can **pin** your own life for a biome with an `ecology:` block: give it a region keyed to a biome, and your **flora**, **fauna**, and **keystone** override the generated life for that biome wherever it occurs.

EDIT · world.hjson

```
ecology: {
  regions: [
    {
      biome: "hot_desert"
      flora: ["blood-thorn", "glass cactus"]
      fauna: ["sand-strider", "dune wyrm"]
      keystone: "dune wyrm"
    }
  ]
}
```

Notice the names are still your own inventions — **blood-thorn**, **dune wyrm** — not Earth species. Pinning the life does not mean abandoning the discipline of the archetype; it means you have already done the naming the generator left to you, and you want the world to keep your beasts instead of proposing its own.

NOTE

The world checks a pinned ecology against the climate it grew. It warns if **cold-adapted** life is pinned to a hot biome — or hot-country life to a frozen one — a creature living where its own body could not — or if the `biome` you named does not occur anywhere in the world you built. As ever the warning yields to you; it only asks whether the life you pinned could survive the weather you already set.

The end of Part IV

With that, the physical and human world is complete. Look back over what you have made across these four parts: you gave your world a **place** — a sky, land, climate, rivers, and cities grown from a single seed. You gave it a **past** — epochs and foundings and migrations, dated on its own calendar. You gave it a **people** — nations, cultures, beliefs, and the profile of a tongue. And now, with its ecology, you have filled that place and that people's world with **life**. A world now has a place, a past, and a people. What remains is to draw the line between what the world invented and what you declare by your own hand — and then to carry the whole living thing to your writing desk.

WHAT YOU LEARNED

- `realworld ecology` fills each land biome with plausible flora and fauna **archetypes** and names a **keystone species** — the last layer the physical world needs.
- Life is given as **archetypes** — "tall conifers," "pack hunters" — not real Earth species, because the world is invented and a real species drags Earth's baggage onto your map.
- A **keystone species** per biome fixes the place in the mind and gives your characters something to follow, fear, and revere.
- The ecology **follows from the climate you already built** — fill the biomes with life, or your map is just coloured regions.
- This closes Part IV: your world now has a **place**, a **past**, and a **people** — and life walking through all three.

PART V

The Author's Hand

CHAPTER 12

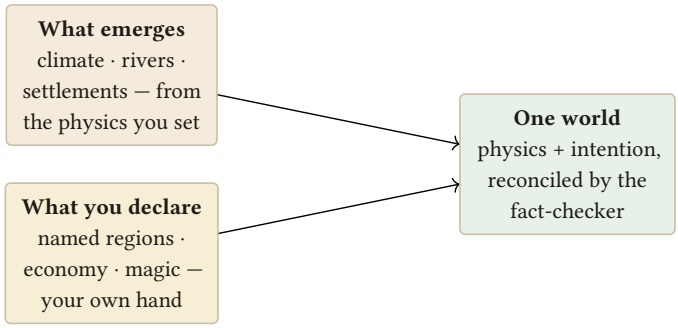
12

What You Declare

Everything so far has come to you by **emergence**. You set a star, a planet, a seed, and the world worked the rest out: the seasons fell out of the orbit, the mountains out of the seed, the rivers out of the mountains, the cities out of the rivers. You did not place any of it. That is the great economy of building a world as a system — most of it arrives on its own, already consistent, because each layer is a consequence of the ones before it.

But a world is not only its physics. A river the compiler carves is real, yet it has no name until you give it one. A fertile crescent the climate produces is a place, yet it is not **the Sunmark Vale** until you say so. Some of a world is emergent, and

some of it you **declare** — by intention, in your own hand — because no equation was ever going to invent the word your characters use for home.



A world has two hands: the physics that falls out on its own, and the names and rules you set by intention. Both are honoured.

> TERM **Declared**
>
> A fact you state outright in `world.hjson`, rather than one the compiler derives. Emergent facts answer **what the physics produces**; declared facts answer **what you have decided to call it and how you mean it to work**. A world is both hands at once — the physics that falls out on its own, and the names and rules you set by intention — and the compiler honours them together.

The declared blocks

Only a handful of things take a block in `world.hjson` at all. Three of them are for the world's named and social detail — `geography`, `hydrology`, and `economy` — and none is computed; each is read exactly as you wrote it. (You have already met the required `astronomy` block and the optional `geology`; `magic` gets its own chapter next.)

> **INSIGHT**
>
> There are three kinds of layer, not two. A few are **purely emergent**: you cannot declare `climate:` or `demographics:`, because they are pure consequences of the physics — to change them, change what they grow from. A few are **purely declared**: `economy` and `magic` are all intention. And most of the rest are **both**. `history`, `nations`, `cultures`, `ecology`, and a river's course all generate a complete answer on their own, but let you **pin your own** facts on top — and when you do, the world checks your hand for plausibility (a river that climbs, a polar beast in the tropics, a seafaring people in a landlocked desert) and tells you, without ever overruling you. Generate, declare, or both: the author always wins, and the world is a second pair of eyes. The rest of this chapter, and the ones before it, show how to declare each.

`geography` is where you name the land. You declare **regions** — a named stretch of the map — and **landmarks** — a single notable point. These are not decoration. Their names feed the **gazetteer** (below) and the fact-checker, so that when your prose mentions the Sunmark Vale, the world knows the place exists and roughly where it lies. A region you never declare is, to the fact-checker, a place that is not in the world.

`hydrology` you have already met as an emergent layer — the compiler runs the rivers downhill on its own. But the descriptive `hydrology` block lets you **name** the waters it found: this river is the Aldermere, that inland sea is the Bitter Gulf. The water flows by physics; the names are yours. You may set a `rainfall` note, and give a `name` and `description` to as many `rivers`, `lakes`, and `seas` as your story cares about — the unnamed ones still flow, they simply go unmarked.

EDIT · world.hjson

```
hydrology: {
  rainfall: "temperate"
  rivers: [
    { name: "The Aldermere", description: "The long river that
feeds the capital." }
  ]
  seas: [
    { name: "The Bitter Gulf", description: "The cold inland
sea north of the reach." }
  ]
}
```

`economy` is pure declaration — the compiler has no opinion on trade. Here you set `tech_level`, the `currency` people count in, the `trade_goods` that move between cities, and the `resources` the land is known for. These give the fact-checker a handle on economic sense (a bronze-age realm minting steel coin is worth a flag), and they give your scenes their texture.

EDIT · world.hjson

```
economy: {
  tech_level: "iron"
  currency: "the silver mark"
  trade_goods: ["salt", "amber", "wool"]
  resources: ["iron", "timber"]
}
```

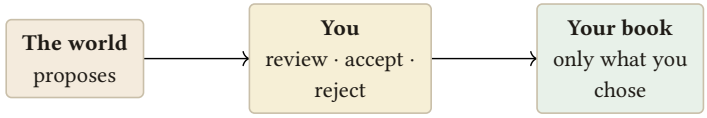
NOTE

All three blocks are optional. Leave them out and the world still compiles — you simply have an unnamed, un-priced world. Add them a line at a time, naming only what your story actually reaches for, exactly as Chapter 1 counselled.

The authority discipline

Here is the rule that governs everything the world produces, and it is worth stating as plainly as it can be stated: **the world proposes; the author decides**. Nothing the compiler generates is ever written into your manuscript on its own. The world's job is to offer; yours is to accept or refuse.

The clearest place to see this is settlements. The demographics layer produces dozens of them — but they are not yet Places in your book. `realworld propose` turns them into **proposals**: candidate Place entries, waiting. `realworld proposals` lets you walk the list and accept the ones you want, rejecting the rest. Only an accepted settlement becomes an entry in the Places book. The forty villages the compiler found but you never accept simply stay in the simulation, available if you need them, silent if you do not.



The world never writes into your book on its own. It proposes; you decide. The author always has the last word.

> TERM **Proposal**
>
> A fact the world **offers** for your manuscript — most often a settlement put forward as a candidate Place entry — that becomes canon only if you accept it. Proposals are how the world stays generous without ever being presumptuous: it shows you everything it found, and writes down only what you chose.

> TERM **Authority**
>
> The principle that the **author always wins**. Where a declared fact and a generated one disagree — you named a river the Aldermere, the generator would have called it something else — the declared name stands. The world is an advisor with an excellent memory, never a co-author with a vote.

INSIGHT

The world proposes; you dispose. A declared name always beats a generated one, and a generated fact enters your book only when you accept it. This is not a limitation of the tool — it is the whole point of it. A world that wrote itself into your manuscript would be a collaborator you did not hire. This one is a consultant you can always overrule.

The gazetteer

Once you have named the land and priced the economy, you will want it all in one place. `realworld gazetteer --output <file>` exports a **gazetteer** — a single consolidated Markdown reference gathering the calendar, the sky, the named regions and landmarks, the waters, the settlements, the economy, and the magic ledger into one document you can keep as an appendix to your manuscript.

TERM **Gazetteer**

A consolidated, human-readable reference to the whole world — every declared name and every headline fact in one file. Where the compiled World book is the world's inner workings, the gazetteer is its **index**: the page you hand a co-writer, or keep at your elbow, when you just need to know what everything is called.

PITFALL

Do not lean on the generator to keep a name you care about. If you love the name the compiler happened to give a river, and you do not **declare** it in `hydrology`, a later recompile with a changed seed or layer can quietly replace it — and your prose now points at a place the world no longer calls that. The cure is one line: declare the name, and authority makes it permanent. Anything you have written into your story, you should have declared into your world.

ASK YOURSELF

Which names in your story are **load-bearing** — the ones a reader would notice changing? Those are exactly the ones to declare, not to leave to the generator. List them, and you have written most of your `geography` and `hydrology` blocks already.

TRY IT

Add a `geography` block declaring a single region — a name and a rough location. Run `realworld compile`, then `realworld gazetteer --output world.md`, and open the file. Find your region sitting among the emergent ones. You have just put your own hand on the map, and the world has folded it in beside its own work.

EDIT · world.hjson

```
geography: {
  regions: [
    { name: "The Ashfall Reach", biome: "cold_desert",
climate: "harsh, dry", description: "A volcanic waste east of
the mountains." }
  ]
  landmarks: [
    { name: "Karrow", kind: "port", climate_zone: "temperate",
population: 18000, description: "The last deep harbour before
the ice." }
  ]
}
```

An AI reading of your world

The world checks itself as far as arithmetic can: `realworld validate` compiles every layer and runs the deterministic lints — a river that flows uphill, a seafaring people with no coast, a declared nation that sits on no land. Those checks are certain, but they are narrow. They cannot tell you that a 360-day calendar quietly

disagrees with a 365-day orbit, or that an Earth-like planet has no business freezing to a mean of five below. For the judgements that need a reader rather than a rule, there is one more pass, and it is the only place in the whole world system that asks a model to **think about your world as a whole**:

```
inkhaven realworld critique
```

It hands an AI your declared `world.hjson` together with a summary of everything it compiles to, and asks for three things: **inconsistencies** (a declared value that fights its own consequence), **implausibilities** (numbers that would not produce the world described), and **missed chances** to make the world more real. What comes back is a ranked list of concrete recommendations — each an aspect, an issue, and a fix. Add `--write-notes` and every recommendation is filed as a paragraph in your Notes book, waiting for the morning you sit down to deepen the world:

```
inkhaven realworld critique --write-notes
```

> **NOTE**
>
> The critique never edits `world.hjson`. It reads, it advises, it writes into Notes — and there it stops. Like the deterministic lints, its findings are advisory: you weigh each one and change the world by your own hand, or decide the oddity is exactly the world you meant to build. The call is cost-capped and the cap only informs; you can turn the pass off entirely with `world.critique_enabled = false`.

> **INSIGHT**
>
> The lints tell you what is **broken**; the critique tells you what is **unconvincing**. A world can pass every deterministic check and still read as thin — a tilt that gives no seasons anyone notices, a belief that fits no land, a population the land could never feed. The AI reading is the outside eye that catches the plausible-looking mistake, and turns it into a note you can act on.

WHAT YOU LEARNED

- A world has two hands: what **emerges** from the physics you set, and what you **declare** by intention. The `geography`, descriptive `hydrology`, and `economy` blocks are where your intentions live — names, waters, and trade the compiler would never invent.
- Declared facts feed the gazetteer and the fact-checker: a region you name is a place the world knows; a river you name keeps its name.
- **The world proposes; the author decides.** Settlements become Places only through `realworld propose` and `realworld proposals` — accept what you want, refuse the rest. Nothing is written into your manuscript without your say-so.
- Where declared and generated disagree, the **author always wins** — so declare any name your story leans on rather than trusting the generator to keep it. Export the lot with `realworld gazetteer` as a manuscript appendix.
- `realworld validate` runs the deterministic lints; `realworld critique` asks an AI to judge the world's consistency and realism and, with `--write-notes`, files its recommendations into the Notes book. Both advise; neither edits — the world changes only by your hand.

CHAPTER 13

13

Rules and Magic

The whole world you have built so far obeys its physics. Riders cross a continent at the pace of a horse, not a thought. Winter arrives when the orbit says it should. A character born a hundred years before the present is, by the calendar, a hundred years old. The fact-checker holds your prose to all of this, and that is exactly what makes the world worth having.

But you may be writing a world where some of that is **meant** to break. Messenger birds that fly day and night, never resting, carry word faster than any horse could run. A sorcerer keeps a garden in eternal summer while snow falls beyond its wall. An order of monks lives ten normal lifetimes. These are not mistakes. They are the

point of your world — and if the fact-checker does not know about them, it will flag every one as an error, and you will drown in false warnings.

The answer is not to switch the fact-checker off. A world with no laws catches no real contradictions either. The answer is to write your magic **down**, as a short list of precise exceptions the checker can consult.

The magic ledger

The `magic:` block is that list. It has two parts: `enabled`, which turns the whole system on, and `rules`, an array of individual exceptions. Each rule is a small, exact statement of **one** way your world departs from its physics.

> TERM **Magic ledger**
>
> The `magic:` block of `world.hjson` — a declared, enumerated list of every way your world's magic overrides its physics. It is called a **ledger** because that is what it is: an account, kept openly, of the exceptions you have chosen. A reader never sees it, but its discipline is felt on every page that stays consistent.

> TERM **Rule**
>
> One entry in the ledger — a single named exception. A rule carries a `kind` (what sort of magic it is), a `covers` list (which fact-check categories it may suppress), a `description` in plain words, and an `applicable_to` scope (the roles, regions, or seasons it applies to). One rule, one exception; the world's magic is the sum of them.

> TERM **Exception to physics**
>
> A place where your world **knowingly** breaks a law it otherwise keeps. The word that matters is **knowingly**: an exception is written down and bounded, so both the fact-checker and the reader can trust that the rest of the world still holds.

How a rule teaches the fact-checker

The mechanism is simpler than it sounds. Each fact-checker warning belongs to a **category** — `travel_time`, `climate_anomaly`, `character_age`, and the rest. A rule's `covers` list names the categories it is allowed to silence, and its `applicable_to` scope says where.

So you write a rule: **messenger birds fly day and night; their journeys break the normal travel limit.** You give it `covers: [travel_time]` and scope it to the courier role. From then on, when you fact-check a scene where a message crosses the realm overnight, the checker sees the rule, recognises that this exact exception was declared, and **stops flagging it**. A one-time act of declaration turns an endless stream of false warnings into silence — while every travel-time claim your rule does **not** cover is still checked as strictly as before.

EDIT · world.hjson

```
magic: {
  enabled: true
  rules: [
    {
      kind: "messenger_birds"
      covers: ["travel_time"]
      description: "Royal pelicans fly day and night, far
faster than any rider."
      applicable_to: { roles: ["royal_messenger"], regions:
["any"] }
    }
  ]
}
```

A second rule follows the same shape. Your order of long-lived monks breaks a different law — age, not travel — so it `covers` a different category and scopes itself to its own role:

EDIT · world.hjson

```
{
  kind: "long_lived_priests"
  covers: ["character_age"]
  description: "The monks of the Grey Cloister live ten normal
lifetimes."
  applicable_to: { roles: ["cloister_monk"], regions:
["any"] }
}
```

NOTE

A rule suppresses only the categories in its `covers` list, only within its `applicable_to` scope. Declaring that couriers outrun a horse does not excuse your foot-soldiers from the same physics. Magic narrows the checker exactly where you tell it to, and nowhere else.

Validating the ledger

`realworld magic` does two things: it **shows** the ledger, and it **validates** it. A ledger is a promise to the fact-checker, and a broken promise is worse than none, so the validator looks for the ways a ledger goes wrong. It flags a rule that `covers` nothing — an exception that suppresses no category is dead weight, probably a mistake. It flags an **unknown category** — a `covers` entry the checker does not recognise, which would silence nothing and likely hides a typo. And it flags a **duplicate** — two rules claiming the same ground, a sign your intentions have started to overlap and drift.

INSIGHT

Magic that means "anything can happen" is not worldbuilding — it is the absence of it, and a reader feels the floor drop out. A magic **system** is the opposite: a short, explicit list of consistent exceptions to laws that otherwise hold. The ledger is what makes the difference real rather than merely intended. The laws stay in force; the exceptions are named, bounded, and few. That is what lets a reader trust the impossible — because everything around it still obeys the rules.

PITFALL

The failure that hurts most is the **undeclared** exception: magic that quietly breaks a law you never wrote down. Your courier outruns a horse in chapter six because the plot needed it, but no rule says couriers can — so a reader who clocked the distances feels the cheat, even if they cannot name it. The fix is the ledger. If your world breaks a law, say so, once, in `magic:`. An exception you declared is wonder; an exception you smuggled is a plot hole.

ASK YOURSELF

What are your world's **laws**, and what are their **precise** exceptions? Not "there is magic" — that answers nothing. Which specific law does each piece of magic break, for whom, and where does the exception stop? Answer that for every supernatural thing in your story, and you have written your ledger.

TRY IT

Run `realworld magic`. Read your ledger back to yourself as the fact-checker reads it: each rule, the category it silences, the scope it silences it in. Then add a rule that covers a category not on the recognised list, run it again, and watch the validator catch you. That flag is the ledger keeping its promise.

WHAT YOU LEARNED

- Magic is not the suspension of your world's physics but a set of **declared, consistent exceptions** to it, kept in the `magic:` block — `enabled` plus a list of `rules`.
- Each rule carries a `kind`, a `covers` list of fact-check categories it may suppress, a `description`, and an `applicable_to` scope. A rule teaches the fact-checker to stop flagging exactly the exception you declared — and nothing else.
- `realworld magic` shows **and** validates the ledger, flagging a rule that covers nothing, an unknown category, or a duplicate.
- A magic **system** is a short list of bounded exceptions to laws that still hold; "anything goes" is not worldbuilding. An exception you declare is wonder; one you smuggle past a law you never wrote down is a cheat the reader feels.

CHAPTER 14

14

Myth and Belief

When you gave your peoples their cultures, the world handed each of them a **belief** — a single line. Ancestor veneration. A sky-pantheon. Nature spirits of river and stone. It was enough to know what a people held sacred, enough to give your characters something to swear by. But a belief on its own is a seed, not a mythology. It tells you **that** these people revere the sky; it does not yet carry the raven that means betrayal, the locked door that keeps returning, the herald who arrives to break the peace. Those are the working parts of a living mythology, and they live somewhere else: in a system book of their own, declared by your hand and watched over by a reader built for the purpose.

TERM **Mythology (the system book)**

A dedicated home for your story's **declared** symbolic vocabulary — its symbols, its recurring motifs, and its archetypal roles. Unlike the World system, which **generates** and **proposes**, the Mythology book holds only what you write into it. Its reader never discovers a symbol you did not name, never interprets one for you, and never touches your prose. It reads your declarations and tells you whether the manuscript keeps faith with them.

This is the same authority discipline as the magic ledger of the last chapter, pointed at a different target. There you declared what your world's powers cost and let Inkhaven flag the scene that spent nothing; here you declare what your symbols **mean** and let it flag the scene that betrays them. Both are the author's hand, written down so the machine can hold you to your own word.

From a belief to a mythology

A mythology, as the system understands it, is three kinds of thing, and it helps to name them before you build any. A **symbol** is a piece of vocabulary that carries a meaning: the word "raven", wherever it falls in your prose, drags a freight of betrayal behind it. A **motif** is a pattern that recurs: the locked door refused and then crossed, the drowned singer, the gift that curses the giver. An **archetype** is a role a character plays in the shape of the story: the mentor, the herald, the shadow. Declare these three well and you have told Inkhaven what your book **means to mean** — the standard it can then measure the prose against.

INSIGHT

A belief answers "what do these people hold sacred?" A mythology answers "and what, in my prose, will carry that reverence?" The first is a fact about the world; the second is a contract about the writing. The Mythology book is where the fact becomes the contract.

The three declarations

Each entry in the Mythology book is a paragraph you tag — `para:myth-symbol`, `para:myth-motif`, or `para:myth-archetype` — with a small HJSON block beneath it. A symbol names its **vocabulary** (the words that carry it), its **meaning**, a **valence** (whether it reads as positive, negative, or ambiguous), and the **traditions** that hold it:

EDIT · world.hjson

```
{ myth_symbol: {
  vocabulary: ["raven", "ravens"]
  meaning: "a herald of betrayal"
  valence: "negative"
  traditions: ["the northern clans"]
} }
```

A motif names itself and describes the pattern it stands for:

EDIT · world.hjson

```
{ myth_motif: {
  name: "the locked door"
  description: "a threshold refused, then crossed"
  valence: "ambiguous"
} }
```

And an archetype binds a **role** to one of your characters by name, saying what that character is **for** in the architecture of the story:

EDIT · world.hjson

```
{ myth_archetype: {
  role: "herald"
  character: "Seren"
  function: "announces the rupture that starts the story"
} }
```

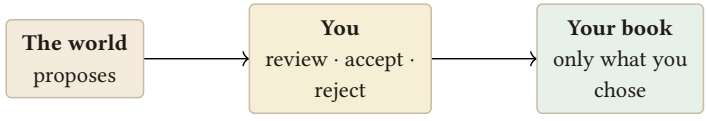
NOTE

Open the Mythology book in a fresh project and you will not find it empty. It ships with a short guide and one worked example symbol, so you can see the shape of a declaration before you write your own. Duplicate the example, change its words, and you have your first symbol; delete it when you no longer need the scaffold.

The bridge from the world

Here the two systems meet. Your cultures already carry beliefs, and a belief is the raw material of a symbol. So the world offers to make the first draft for you. Run `realworld propose-myth` and it reads every culture's belief and turns it into a Mythology proposal: a sky-pantheon becomes a symbol of sky and storm and thunder; nature spirits of river and stone become a symbol of the small gods bound to the land; a cult of the seasons becomes the motif of the turning year. Two peoples who share a belief are gathered into one entry, and the **traditions** field records which of them hold it — the thread that ties the symbol back to the world that suggested it.

Nothing commits on its own. Each proposal waits in the same queue as your Place proposals, and you work it the same way — `realworld proposals list` to read them, then accept the ones that ring true. An accepted proposal is written into the Mythology book as a real tagged entry, ready for you to sharpen.



The world never writes into your book on its own. It proposes; you decide. The author always has the last word.

The world is a generous but literal collaborator here. It can tell that a people who revere the sky will have a sky-symbol; it cannot tell you that in **your** book the sky-god is a tyrant whose worship the heroine will learn to refuse. That meaning is yours to write. Take the proposal as a rough-cut stone — the right material, roughly the right shape — and carve it into the symbol your story actually needs.

> **ASK YOURSELF**
>
> Read back the symbols the world proposed from your cultures. For each, ask: is this what the symbol means to **me**, or only what it would mean to the people who hold it? A drowned-god symbol a coastal people reads as protection might, in your protagonist's hands, mean grief. Declare the meaning your book will honour, not the one the culture would recite.

Archetypes and your characters

Symbols and motifs live in the words; archetypes live in your people. When you declare an archetype you name a character who already exists in your Characters book, and you say what role they play — herald, mentor, shadow, trickster, or a custom role of your own. This is the one declaration that reaches across into another system book, and it is worth the reach: it lets Inkhaven notice when a role you promised goes unfilled, or when the character you cast as the herald never actually heralds anything.

> **NOTE**
>
> An archetype needs a character to inhabit it. Declare the role before you have cast it and the Mythology reader will tell you the role stands vacant — a useful nudge when you have planned a mentor into your structure but not yet written the person who mentors.

Keeping the mythology honest

Once your declarations are in place, two commands hold the manuscript to them. `inkhaven myth scan` is the quiet one: it builds a density heatmap of where your symbols actually appear across the chapters — pure counting, no AI, no network — so you can see at a glance whether the motif you swore ran through the whole book in fact clusters in its first act and vanishes. `inkhaven myth check` is the searching one: it asks whether a symbol is ever used against its declared meaning, whether a motif you promised ever lands in the final act, whether an archetype's character does the thing their role requires. Its findings are advisory — it flags, it never edits — and every one can be suppressed when you have looked and decided the prose is right.

> **INSIGHT**
>
> The Mythology reader can only be as honest as your declarations are complete. It will never invent a symbol you did not name, which means it will never scold you for a pattern you never claimed. Declare what you mean your book to mean, and it becomes your most patient reader — the one who has memorised your intentions and quietly checks the pages against them.

> **TRY IT**
>
> Take a peopled world and run `realworld propose-myth`, then accept one symbol into the Mythology book. Open the book, sharpen the accepted symbol's meaning to fit your story, and add one motif of your own. Write a scene that uses the symbol's vocabulary, then run `inkhaven myth scan` and watch it appear in the heatmap. You have carried a people's belief all the way from a line in a culture card to a checked, living pattern in your prose.

WHAT YOU LEARNED

- The **Mythology** system book holds your **declared** symbolic vocabulary — **symbols** (words that carry a meaning), **motifs** (patterns that recur), and **archetypes** (roles bound to your characters). It reads only what you declare.
- A culture's **belief** is the seed of a mythology. `realworld propose-myth` reads your peoples' beliefs and proposes symbols and motifs, crediting the peoples who hold them in the **traditions** field — the World × Mythology bridge.
- Proposals wait in the same queue as Places; you accept the ones that ring true, and an accepted proposal becomes a real tagged entry you then sharpen. The world proposes the material; the meaning is yours.
- **Archetypes** reach across to the Characters book, so Inkhaven can notice a role left vacant or a herald who never heralds.
- `inkhaven myth scan` maps where your symbols fall (zero-AI); `inkhaven myth check` asks whether the prose keeps faith with their declared meanings. Both advise; neither edits. The author always has the last word.

PART VI

The World at the Desk

CHAPTER 15

15

Writing Against the World

Everything so far has been building. You raised a sky, grew a climate, ran the rivers down the mountains, settled people where the land would hold them, gave the whole thing a past and a people. It is a great deal of work to have done for a setting — and it is worth precisely nothing if, when you sit down to write the scene where your courier rides out of the capital in the first hard frost of the year, none of it is anywhere near your hand. A world kept in another window is a binder by another name. This part of the book is about the payoff: the world **at the desk**, present in the moment you are actually writing, answering the small questions a scene keeps asking.

INSIGHT

A world earns its keep only when it is **at your cursor**. However thorough the climate, however careful the history, a setting you have to stop and go look up is a setting you will stop looking up — and then quietly start contradicting. The whole point of building the world in the same tool you write in is that it can come to you, in the scene, without your leaving the page.

What season is it, where my character is standing?

This is the question a scene asks first, and the one a binder answers worst. You know the date — it is on the scene, on the Timeline — but the date alone does not tell you what your character **feels** when she steps outside. That depends on where she is standing on the planet, and your world already knows the answer, because it computed the insolation for every latitude when you grew the sky.

```
realworld weather --day 300 --lat 55
```

Give it a day of the year and a latitude, and it reports the local season and the weather you would expect there — and it is **hemisphere-correct**, which is the detail a hand-kept note almost always gets wrong. Day 300 at fifty-five degrees north is deep into the descent toward winter; the very same day at fifty-five degrees **south** is climbing toward high summer, because the tilt that leans one hemisphere toward the star leans the other away. The command knows this because it reads it off the astronomy layer you already built, not off a season table someone typed by hand.

TERM **Scene context**

The small bundle of world facts that a particular scene sits inside: **where** it takes place, **when** on the world's calendar, its **season and weather**, its **biome and climate**, and the **people** whose land it is. Scene context is the world narrowed to one moment and one spot — the answer to "what is true, right here, right now, for this character?"

Could they actually get there?

The second question a scene asks is about distance, and it is the one that most often catches a careful author out, because prose has no sense of scale. A sentence can carry a rider from the capital to the far coast in an afternoon without the slightest strain, and the reader who has been paying attention will feel the whole world shrink around it. Your world knows the **real** distance, because it knows how big the planet is and how the map grid maps onto it.

```
realworld travel --from Rivenmouth --to Caldwatch --days 4 --mode
horse
```

Name a starting place and a destination, say how many days you are giving the journey, and pick a mode — `foot`, `horse`, `cart`, or `ship` — and Inkhaven measures the true distance between the two points and checks it against that mode's honest pace. Four days on horseback covers only so much ground; if your draft asked for more, the command says so before your reader does. And crucially, it does not stop at physics: it consults the **magic ledger** first, so that if your world has declared some sanctioned way of crossing distance — a road that folds, a courier's spell, a season when the rivers run fast — the journey is judged against the rules you actually set, not against a bare horse.

> TERM **Continuity**
>
> The property of a story whose facts stay consistent with one another and with its world from one page to the next — the same journey taking the same time, the same city in the same place, winter falling when the calendar says it should. Continuity is what a built world exists to protect: not to make the writing fancier, but to keep it from quietly contradicting itself.

> NOTE
>
> If you know the map coordinates of a spot that is not a named place — a battle in an empty valley, a camp between towns — you can give `travel` a `--from-x` and `--from-y` instead of a named `--from`. The check is about distance on the real planet, so any point on the grid will do.

The whole scene, in one brief

Often you do not want three separate answers; you want the setting of a scene handed to you in one piece, the moment before you write it. That is what the scene brief is for.

```
realworld scene --place Rivenmouth --day 300
```

For a named place on a given day, Inkhaven assembles the **scene context** in a single reading: the season and weather at that place's latitude, the biome and climate it sits in, the culture of the nearest realm — the people whose land this is, the ethos and beliefs you gave them back when you grew the cultures — and the nearest neighbouring place, with the distance and the direction to it. It is the difference between remembering that Rivenmouth is "up north somewhere" and being told, before the first sentence, that it is a cold-coast town in the last grey week before the frost, in the lands of a people who hold the sea sacred, two days' ride south of Karsholt. You write a truer scene when the brief comes first.

> **NOTE**
>
> The neighbour can be any named feature with a position on the world grid — a Place (whether the world proposed it or you set it with `realworld set-coords`), a declared landmark given a `lat`/`lon`, or a named water with a source or mouth cell. So the moment you give any of them a location, it starts appearing in the briefs of its neighbours, and theirs in its. The map you build quietly makes every scene more grounded than the last.

The world while you type

The commands are there when you want to ask. But the deeper idea of writing against a world is that you should not always have to ask — that the setting should be quietly present as you work. Inkhaven does this two ways in the editor.

As you write a scene that is linked to a place and a date, an ambient **footer chip** sits at the bottom of the editor showing that scene's context in miniature: its place, its season, its people. You do not summon it; it is simply there, the way the word

count is there, so that the season your character is standing in is in the corner of your eye while you describe what she sees.

And when you want the fuller picture, `Ctrl+B W` opens the read-only World overview — every layer of the world laid out to read — and, when your cursor is inside a scene, it leads with a **This scene** header: the same context the brief would give you, pinned to the top, because that is the part of the whole world you need right now.

> **NOTE**
>
> The footer chip is **self-gating**. It appears only for a scene that is actually linked to a place and a date — the two facts it needs to compute a context. An unplaced, undated scene shows no chip, and rightly so: there is nothing true to say about a where and a when you have not set yet. The chip is a reward for having placed the scene, not a nag to do it.

> **INSIGHT**
>
> Notice what has quietly happened. The season your character feels, the distance her journey covers, the people whose land she crosses — none of these are things you now have to **remember**. They are things the world **tells you**, in the scene, as you write it. That is the whole arc of this book arriving at its point: you built the world once so that from here on it can carry the small facts for you.

> **TRY IT**
>
> Run `realworld weather --day 300 --lat 55`, note the season it reports, then run it again with `--lat -55` and watch the season flip to its opposite — the hemisphere correction, made visible. Then take a journey from your own draft that has always felt a little too easy, and put it to `realworld travel --from … --to … --days … --mode …`. If the world argues, believe it.

WHAT YOU LEARNED

- A built world pays off only when it is **at your cursor** — present in the scene you are writing, not filed in another window.
- `realworld weather --day <N> --lat <deg>` gives the local season and weather for a day and latitude, **hemisphere-correct** from the astronomy you built.
- `realworld travel --from … --to … --days … --mode <foot|horse|cart|ship>` checks a journey against the **real** distance and the mode's pace — and consults the magic ledger's `travel_time` rules first.
- `realworld scene --place … --day …` hands you the whole **scene context** in one brief: season, weather, biome, and the nearest realm's culture.
- In the editor an ambient **footer chip** shows the scene's place, season, and people, and `Ctrl+B W` opens the World overview with a **This scene** header — the chip appearing only for scenes linked to a place and a date.

CHAPTER 16

16

Keeping Your Prose True

The last chapter put the world at your cursor so it could hand you a season, a distance, a people, before you wrote. This chapter turns the same world around to face the other way — so that after you have written, it can read what you wrote and tell you where you contradicted it. This is the quiet, unglamorous dividend of all that building, and in truth it is the biggest one. A world you can consult is useful. A world that **checks you** is transformative, because the mistakes that wreck a setting are never the ones you look up — they are the ones you never thought to doubt.

INSIGHT

The real dividend of a built world is that it catches you contradicting it. You will not deliberately write snow into your tropics or send a cart three hundred miles in a morning; you will do it **by accident**, in a fast draft, and read straight past it a dozen times because it looked fine. The world argues back — it holds the facts you set and measures your prose against them — and that is a thing no binder has ever done.

Reading your prose against the world

The tool for this is the fact-checker. You hand it a passage — a sentence you are unsure of, a paragraph you just wrote — and it reads that prose against the world you compiled.

```
realworld fact-check --text "They rode from the capital to the
far coast in a single afternoon."
```

You can check loose text with `--text`, as above, or check a paragraph already in your manuscript by its identifier with `--paragraph <id>`. Either way the checker looks for the kinds of claim a world can adjudicate: **travel times** against the real distances and paces, **climate** against the biome a place sits in, **date coherence** against the calendar, and more besides. The afternoon ride above will come back flagged, because the world knows how far the far coast is and how long an afternoon lasts, and the two do not meet.

TERM **Fact-check (against the world)**

Reading a passage of prose against the compiled world to find claims that contradict it — a journey too fast for its distance, weather that fights the local climate, a date that does not fit the calendar. It is not a grammar or a style check; it takes no view on your sentences. It has exactly one question: **is this consistent with the world you built?**

> TERM **Continuity error**
>
> A place where the story contradicts itself or its world — the crossing that took four days on page ten and one day on page ninety, the harvest festival held in the dead of the local winter, the river that flows the wrong way down its own valley. A continuity error is rarely a failure of imagination; it is a failure of **bookkeeping**, which is exactly the sort of failure a machine is good at catching.

Two speeds of checking

The fact-checker runs on two tracks, and it is worth knowing which is doing the work. The first is a **fast, deterministic** track: it reads the structured claims it can measure directly — a named journey, a stated season, a date — and settles them against the world's own numbers, instantly and the same way every time. This is the track that catches the afternoon ride, because distance and pace are arithmetic.

The second is a slower **LLM track** that reads the prose more as a reader would, catching the softer contradictions that are not stated as tidy claims — an implied warmth in a scene the calendar places in deep winter, a detail that only reads as wrong once you understand what the sentence is describing. The deterministic track is your everyday check; the LLM track is there for the harder reading, when you want a second pair of eyes over a passage rather than a measurement of it.

> NOTE
>
> Because the two tracks differ in cost, they differ in when you reach for them. The deterministic track is cheap enough to run constantly, on a line you are unsure of, as often as you like. The LLM track is worth spending on a finished scene, a chapter you are about to call done — the moments where a careful, slower reading pays for itself.

Two commands sit either side of that line. `realworld co-location` is a deterministic check of a different kind: it reads your story Timeline and flags a **character in two places at overlapping times** — the oldest continuity slip in fiction — honouring any magic-ledger rule that would allow it. `realworld coherence <node>` is the LLM track pointed at a whole container: hand it a book or chapter

and it reads every paragraph under it **together**, for the contradictions that only show up between passages — a fact stated one way in chapter three and another in chapter nine. One is a free, constant guard; the other a considered, cost-capped pass over a finished stretch.

Why a checked world does not drown you

There is an obvious fear here, and it is worth meeting head-on: a checker strict enough to be useful sounds like a checker that will flag every deliberate wonder in your book. If your world has a witch who summons frost in high summer, a climate check that does not know about her will flag that frost every single time, and a tool that cries wolf on your own magic is a tool you will switch off inside a week.

This is exactly what the **magic ledger** is for. Back when you drew the line between what your world computes and what you declare, the magic you set down was recorded as a set of **declared exceptions** to physics — each rule saying what it covers, and whom and where and when it applies. The fact-checker consults that ledger before it flags anything, and **suppresses** any contradiction your ledger already sanctions. The witch's summer frost, covered by a rule with `covers` including `climate_anomaly` and an `applicable_to` that names her, is not a continuity error; it is a fact of your world, and the checker knows it.

> **INSIGHT**
>
> A one-time declaration buys you endless quiet. You wrote the magic rule once, deliberately, when you decided what your world's exceptions were. Because of that single act the checker can stay strict **everywhere else** without ever crying wolf on the one wonder you meant. This is the deep reason the ledger exists: not to permit magic, but to let the check around it stay honest.

NOTE

The magic ledger is precisely why a checked world does not bury you in false warnings. A checker with no notion of sanctioned exceptions must either be too loose to help or too shrill to keep on. The ledger gives the fact-checker a third option — strict about physics, silent about the magic you declared — and that is the only setting a working author can actually live with.

PITFALL

The commonest way to get a **true** flag you did not expect is to write weather that fights the biome — a snowfall in a place your climate layer made tropical, a drought in a rainforest, a hard freeze on a coast that never freezes. The checker will catch it. But note the condition buried in that sentence: it can only catch this **if you built the climate**. Skip the climate layer, or leave a place's biome undecided, and there is nothing for the weather to contradict — the check falls silent not because your prose is true but because the world has nothing to say. The check is only ever as good as the world beneath it.

TRY IT

Run `realworld fact-check --text "They rode from the capital to the far coast in a single afternoon."` and read the flag it returns — the distance, the pace, the verdict. Then take a real paragraph from your draft, one set somewhere with a decided climate, and check it with `--paragraph <id>`. If it comes back clean, you have earned a small, specific confidence; if it does not, you have caught a contradiction while it was still cheap to fix.

WHAT YOU LEARNED

- The **fact-checker** reads your prose against the compiled world — `realworld fact-check --text "…"` or `--paragraph <id>` — and flags travel times, climate, date coherence, and more.
- It runs on two tracks: a **fast deterministic** one for measurable claims, and a slower **LLM** one for the softer contradictions a reader would feel.
- The **magic ledger** suppresses declared exceptions, so a sanctioned wonder is never flagged — which is what lets the check stay strict without crying wolf.
- The checker only catches what the world can adjudicate: weather that fights the biome is flagged **only if you built the climate**.
- The genuine payoff of a built world is that it **argues back** — it catches the continuity errors you would otherwise read straight past.

CHAPTER 17

17

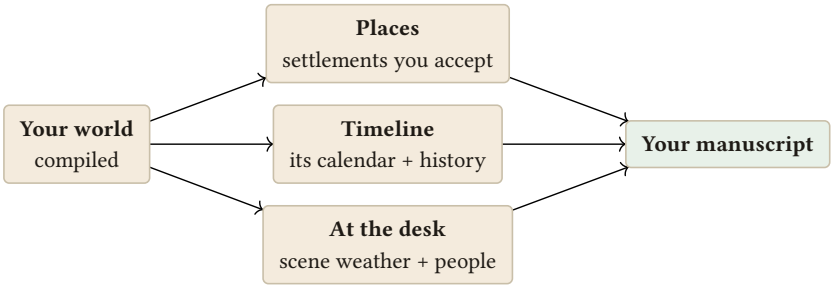
Into Your Book

You have a world that comes to your cursor and a world that checks your prose. The last step of the whole journey is the quietest and, in a way, the most important: letting the world flow **into the manuscript itself** — not as a separate reference you keep beside your book, but as the very Places you tag, the Timeline you date scenes on, the page you type. A world that only ever answers questions is still a world apart. The aim of this chapter is a world that is a **source**: material that pours into the rest of your writing rather than sitting in a silo next to it.

INSIGHT

The world is not a separate document. Its settlements become the same **Places** you tag in prose; its calendar and history become the same **Timeline** your scenes are dated on; its facts reach the page you are typing. A setting kept in its own file is a binder; a setting that flows into the tools you already write with is a source. The difference is the whole reason to have built it here.

Here are the bridges, drawn as one picture — every path the world takes into your book:



The world is not a separate document. It flows into the manuscript — as places, as a calendar and history, and as context at the cursor.

Settlements into the Places book

The cities and towns your demographics layer grew are not much use to a novelist as a list in a world file. They become useful when they are **Places** — entries you can tag in prose, search, and cite.

TERM **Places book**

Inkhaven's system book of settings — the towns, cities, regions, and landmarks your story uses. Tag a place in your prose and Inkhaven can gather every scene set there; a Place entry is the shared, canonical record of a spot your story keeps returning to.

The bridge here is `realworld propose`. The world does not write itself into your Places book — it **proposes** its settlements as Place entries, and you accept or reject

each one. Accept Rivenmouth and it becomes a real Place you can tag; leave the hundred villages you will never name unaccepted and they stay in the world file, out of your way. The authority runs one direction only: the world offers, you choose.

> NOTE
>
> Run `realworld propose` to have the world put its settlements forward as Place candidates, and `realworld proposals` to review what is waiting. Nothing enters the Places book until you accept it — the world never writes on its own.

The calendar and history into the Timeline

The second bridge carries time. Your world computed a calendar from its sky and a chronology from its founding, and both belong in the one place your story keeps time.

> TERM **Timeline**
>
> Inkhaven's system book of **when** — the calendar your story runs on and the dated events along it. Scenes are dated against the Timeline; it is what lets Inkhaven know that one scene comes before another, and what season a given date falls in.

The astronomy's calendar adopts into `timeline.calendar`: run `realworld calendar` and the world hands you the months, weekdays, and new-year alignment it derived from the sky, as lines you adopt so that your scene dates and your world's seasons share one root. The history adopts the same way — `realworld history` emits its epochs and events as ready-made `inkhaven event add` lines, one per founding and migration and the rise or fall of a realm, dated in years before the present. You paste in the ones your story reaches back to and leave the rest; the world's whole past is available, and only the part you use enters your Timeline.

INSIGHT

Notice the same discipline on every bridge: the world **proposes**, in a form you can adopt, and you decide what crosses over. Places come as proposals you accept; the calendar and history come as command lines you choose to run. The world's reach into your manuscript is real but never unilateral — the author always has the last word.

The world as a manuscript appendix

The last bridge is the simplest. Sometimes you do not want the world threaded through Places and Timeline — you want it **whole**, one readable reference bound in the back of the book, for yourself or your reader.

```
realworld gazetteer --output world-reference.md
```

The gazetteer gathers the entire compiled world — its calendar and sky, its regions and landmarks, its waters, its settlements, its economy and magic — into a single consolidated Markdown document. Written out with `--output`, it becomes a file you can keep as a series bible or fold into the manuscript as an appendix: the gazetteer at the back of the fantasy novel, made not by hand but from the same world every other bridge draws on, and so guaranteed to agree with them.

TRY IT

Run `realworld gazetteer --output world-reference.md` and open the file. Read it as a stranger to your own world would — this is the reference your reader might hold. Where it reads thin, you have found the part of the world worth deepening next; where it reads rich, you have found what your prose can safely lean on.

A rendered map, and the plakat round-trip

One bridge leads not into your prose but onto your wall: your compiled world can become a **labelled, painted map**. Back in the chapter on the land you met **plakat**,

Inkhaven's companion tool, and used it to grow a heightmap. The two tools pass a world back and forth, and it is worth seeing the whole loop, because each does what the other cannot: plakat draws shapes and paints pictures; Inkhaven runs the physics and grows the life. This section is the short version; the next chapter, **Drawing the Map with plakat**, walks the whole thing in full — with real maps, three styles, and every step for someone who has never done it.

Inkhaven → plakat: render your world as a map

The `realworld map` command compiles your world, assembles a plakat **MapSpec** from its geology, climate, rivers, and settlements — together with the Places you have accepted — and hands that spec to plakat, which draws a finished, labelled map:

```
inkhaven realworld map
```

This writes `maps/world.mapspec.json`, a rendered map image, and a GeoJSON of the features, all under your project. It runs plakat for you, so plakat must be installed and on your `PATH` (Inkhaven checks by calling `plakat --version`). One quiet extra happens on the way back: the map's resolved landmark positions are read in to **refine the coordinates** of your accepted Places, so the map and your Places agree — pass `--no-ingest` to skip that.

If you would rather drive plakat yourself — to choose a style, or paint the map with a model — ask Inkhaven for the spec alone and stop there:

```
inkhaven realworld map --spec-only
```

Then run plakat on the spec it wrote, exactly as in the map chapter of plakat's own guide:

```
plakat map --map-spec maps/world.mapspec.json --map-render
world.png --map-style parchment
```

`--map-style` takes `parchment`, `inked`, or `blueprint`; `--map-render-sd PATH` paints the map with a diffusion model instead of drawing it flat; and `--map-export-svg` / `--map-export-geojson` write vector versions. The spec is a pure function of your world, so the same compiled world always yields the same map.

> **NOTE**
>
> `realworld map` needs plakat on your `PATH`; `realworld map --spec-only` does not — it only writes `maps/world.mapspec.json`, which you can hand to plakat on another machine, or keep as the portable, byte-stable description of your world's map.

Placing a Place by hand

The map draws the settlements the world proposed and you accepted — each of those carries a grid position from the moment you accepted it. But some places on your map were never the world's idea: a hidden monastery, a ruined tower, a city you typed straight into the Places book because the story needed it. A Place authored by hand has no coordinates, so the map does not know where to draw it. You give it one with `realworld set-coords`:

```
inkhaven realworld set-coords "Skyhold" --lat 55 --lon -20
```

You may pass geographic degrees (`--lat`/`--lon`) or raw grid cells (`--x`/`--y`); Inkhaven fills in the biome under that cell from the compiled climate, so a hand-placed Place arrives knowing what land it stands on. From then on it is a first-class location: `realworld map` draws and labels it like any other, and the round-trip refines its position just the same. Move it later by running the command again with new coordinates.

> **NOTE**
>
> `set-coords` works on any Place in the Places book, whether the world proposed it or you wrote it. It does not invent a settlement or a population — it only answers the one question the map needs: **where on the world does this place stand?** Everything else about the Place remains yours.

A declared **landmark** can be placed the same way, without going through the Places book at all. Give any `geography.landmarks` entry a `lat`/`lon` (or raw grid `x`/`y`) and the map draws it where you put it:

EDIT · world.hjson

```
geography: {
  landmarks: [
    { name: "The Ashen Peak", kind: "mountain", lat: 40, lon:
-30 }
  ]
}
```

A landmark with no position stays a gazetteer entry the fact-checker knows by name; a landmark with one becomes a labelled marker on the map — and, like any positioned feature, it starts appearing in the scene briefs of the places near it.

plakat → inkhaven: bring a map in

The reverse direction you have already used once, and can use more fully:

- **The land.** A plakat heightmap (`plakat map … --map-dump-heightmap heightmap.png`) becomes your terrain through `geology.dem` — the recipe from the chapter on the land. This is how a shape you drew or generated in plakat becomes the ground Inkhaven grows climate, rivers, and cities over.
- **The named features.** A plakat map also gives names and positions — regions, rivers, notable places. Copy the ones that matter into Inkhaven's declared blocks: a plakat region becomes a `geography.regions` entry, a named river a `hydrology.rivers` entry, a labelled town a `geography.landmarks` entry. Now the same names live in both tools, and Inkhaven's fact-checker knows them.

INSIGHT

The full loop is worth holding in your head: **draw or generate a shape in plakat → its heightmap becomes Inkhaven's land → Inkhaven grows the living world (climate, rivers, cities, history, peoples) → `realworld map` hands that world back to plakat → plakat paints the finished, labelled map.** One world passes between the two tools, each turn adding what only it can. You can start from either end — a hand-drawn coastline or a simulated one — and arrive at the same place: a world that is both **true** and **beautiful**.

> **TRY IT**
>
> Run `realworld map --spec-only`, then open `maps/world.mapspec.json` — this is your whole world, distilled to the shape plakat draws from. If you have plakat installed, render it: `plakat map --map-spec maps/world.mapspec.json --map-render world.png`, and hang the result over your desk.

The world touches the page

Step back and look at what the bridges add up to. A settlement your climate and rivers decided becomes a Place you tag in a sentence. A calendar your sky computed becomes the date at the top of a scene. A founding from your world's deep past becomes an event on the Timeline your story is measured against. And while you write the scene itself, the season and the people of that place sit in the corner of the editor, and the fact-checker reads the finished paragraph back against all of it. There is no seam left between the world and the book. That was the promise made in the introduction — that a world is only worth the trouble if it **touches the page** — and here, on every bridge at once, it is kept.

WHAT YOU LEARNED

- The world is a **source**, not a silo: it flows into the same Places, Timeline, and page you already write with.
- Settlements cross into the **Places book** by `realworld propose` and your acceptance — the world proposes, you decide.
- The astronomy's calendar adopts into `timeline.calendar` via `realworld calendar`, and history's epochs and events adopt as `inkhaven event add` lines from `realworld history`.
- `realworld gazetteer --output <path>` writes the whole world as one Markdown reference — a series bible or manuscript appendix that agrees with every other bridge.
- The world round-trips with **plakat**: `realworld map` renders your compiled world (and refines your Places from the result), while a plakat heightmap and named features come back the other way into `geology.dem` and the declared blocks.
- Every bridge keeps the same discipline — the world offers, the author chooses — and together they close the seam between the world and the book.

CHAPTER 18

18

Drawing the Map with plakat

You have built a world that knows where its mountains rose, where its rivers reach the sea, and where its people settled. All of that is **shape** — coordinates and elevations and biomes held in the compiler. This chapter turns that shape into a picture: a labelled, painted map you can pin to the wall, drop into an appendix, or keep at your elbow while you write. Inkhaven does not draw the map itself. It hands the work to a companion tool built for exactly this — **plakat** — and the two pass a world back and forth. By the end of this chapter you will have rendered your own world three different ways, and you will understand every step in between.

If you met plakat back in the chapter on the land, where you used it to **grow** a heightmap, this is the other half of the partnership: there you brought terrain **in**; here you send a finished world **out** to be drawn.

> TERM **MapSpec**
>
> The bridge between the two tools — a single JSON file describing your world's cartography: its coastline and mountains, its rivers, its regions, its labelled places, and the roads between them. Inkhaven **emits** a MapSpec from your compiled layers; plakat **reads** it and draws. Because the spec is a pure function of your world and its seed, the same world always yields the same map, and you can keep the spec as a small, portable description of your map that renders identically on any machine.

Getting plakat

plakat is a separate program — a Rust binary you install once. If you have the Rust toolchain, the simplest route is:

```
cargo install plakat
```

That puts a `plakat` command on your `PATH`. Check it answers:

```
plakat --version
```

Inkhaven runs plakat **for** you when it needs to, and it always checks for it first with exactly that command — so if the map commands ever tell you plakat is missing, this is the line to run. Everything else in this chapter assumes `plakat` is installed and on your `PATH`.

> NOTE
>
> plakat is **optional**. Nothing in the rest of the book needs it — your world compiles, materialises, and fact-checks with Inkhaven alone. The map is a bonus at the end of the road, not a dependency along it. If you never install plakat, you simply never render a picture; the world is no less complete.

One command, one map

The fastest path from a compiled world to a drawn map is a single command. From a project with a `world.hjson`, run:

```
inkhaven realworld map
```

Inkhaven compiles every layer, assembles a MapSpec from the geology, climate, rivers, and settlements — together with the Places you have accepted and any realm capitals and trade routes — writes it to `assets/maps/world.mapspec.json`, and then calls plakat to draw it. Out come a rendered image and a GeoJSON of the features, all under your project. Here is a world called **Aeloria**, rendered exactly this way:

Aeloria, rendered by `realworld map` — coastline, mountains, rivers, and every accepted settlement, drawn from the compiled world. The compass, scale bar, and legend are plakat's; everything they point at is yours.

Every mark on that map came from a layer you have already met. The coastline is where geology's sea level cut the heightmap; the shaded relief is the heightmap itself; the blue threads are hydrology's rivers, running downhill to the sea; the labelled dots are the settlements demographics placed, sized by population. Nothing was invented for the picture — the picture is the world, seen from above.

Choosing a look

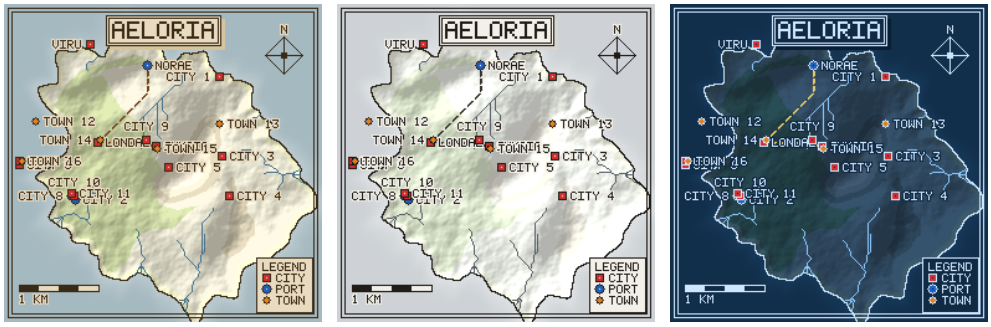
`realworld map` draws with plakat's default **parchment** style and sensible defaults. When you want to choose the look yourself — a different palette, a season, a printed edition — ask Inkhaven for the **spec alone** and drive plakat by hand:

```
inkhaven realworld map --spec-only
```

That writes `assets/maps/world.mapspec.json` and stops, without calling plakat. Then you render the spec yourself, choosing a style:

```
plakat map --map-spec assets/maps/world.mapspec.json --map-render
aeloria.png --map-style inked --seed 7
```

plakat ships three cartographic styles. The **same** Aeloria, the **same** spec and seed, drawn three ways:



`--map-style parchment` · `inked` · `blueprint`. *One world, three atmospheres: the aged chart, the clean line drawing, the surveyor's plan.*

The `--seed` is plakat's own drawing seed — it fixes the little cartographic choices (how a coastline wobbles, where a label sits) so the same command always draws the same picture. Change it and the terrain shifts slightly; keep it and your map is reproducible. A **season** recolours the land — the same Aeloria under `--map-season winter` trades its summer greens for snow:

`--map-season winter`. *The palette follows the season; the geography does not move.*

> **NOTE**
>
> A few more knobs worth knowing: `--map-render-sd <PATH>` paints the map with a diffusion model instead of drawing it flat; `--map-grid N` overlays an N×N tabletop coordinate grid (A1/B2…); `--map-tiles CxR` with `--map-render-tiles` slices the world into a grid of seamless tiles that stitch back together, for a large printed or virtual-tabletop map. `--map-export-svg` writes a vector version. Run `plakat map --help` for the full set.

What the map shows, and where it came from

It is worth naming every kind of mark, because each is a layer of your world made visible — and each is something you can **change** by changing the world:

- **Relief and coast** — the geology heightmap and its sea level. Grow a different DEM, or change `geology.generated.sea_level`, and the land itself redraws.
- **Rivers** — hydrology's flow model, running from source to sea. Declare a named course under `hydrology.rivers` and it is labelled.
- **Settlements** — the towns and cities demographics placed, and the Places you accepted, sized by population. A coastal city is drawn as a **port**.
- **Realm capitals and trade roads** — each realm's seat is a hub, and the routes from `realworld trade` are drawn as roads (land) or sea lanes between them.

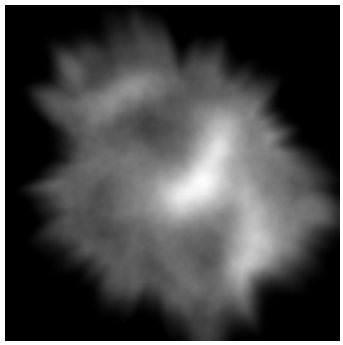
- **Declared landmarks** — any `geography.landmarks` entry you gave a `lat`/`lon` (or `x`/`y`) is drawn where you placed it, labelled by name.

So the map is not a separate artefact you maintain by hand — it is a **view**. Every time you recompile, the map can be redrawn to match, and it will always agree with the world your prose is checked against, because it is drawn from the same numbers.

The other direction: growing terrain from a heightmap

plakat can also hand a world **to** Inkhaven. Its most useful gift is a **heightmap** — a grayscale image where brightness is elevation, bright peaks down to black sea. plakat will dump one from any spec:

```
plakat map --map-spec assets/maps/world.mapspec.json --map-dump-
heightmap heightmap.png --seed 7
```



*A plakat heightmap (DEM): brightness is elevation. This is not a picture **of** a world — it is the **shape** a world grows from.*

To build your world's land **from** that image rather than from a seed, drop it into your project and point the `geology.dem` block at it:

EDIT · world.hjson

```
geology: {
  dem: {
    path: "maps/heightmap.png"
    scale_km_per_pixel: 5.0
    sea_level_pixel_value: 40
  }
}
```

Now `realworld compile --layer geology` reads the heightmap as your terrain, and every layer above it — climate, rivers, cities — grows over **that** shape instead of a generated one. This is how a coastline you drew, or generated in plakat, or lifted from real elevation data, becomes the ground your whole world stands on. The chapter on the land walks this path in full; here it completes the loop — a shape that left as a spec can come back as terrain.

PITFALL

A heightmap is only a shape — it has no names. Bringing one in gives you a coastline and mountains, but the rivers, regions, and cities are still grown by the layers above, and the **names** are still yours to declare or accept. Do not expect a plakat-drawn map's labels to survive the round-trip into `geology.dem`; the elevation survives, the lettering does not.

The round-trip, closed

There is one quiet exchange worth understanding, because it keeps your two pictures of the world in agreement. When `realworld map` renders, plakat resolves each labelled place to a precise position on the drawn map. Inkhaven reads those resolved positions back and uses them to **refine the coordinates** of your accepted Places — so the place on the map and the Place in your book sit at exactly the same spot. It happens automatically; pass `--no-ingest` if you would rather the map not touch your Places' coordinates.

INSIGHT

Hold the whole loop in your head, because it is the point of the partnership. **Inkhaven → plakat**: your compiled world becomes a MapSpec, and plakat draws it. **plakat → Inkhaven**: a heightmap becomes your `geology.dem`, and named features become declared regions, rivers, and landmarks. Each tool does what the other cannot — Inkhaven runs the physics and grows the life; plakat draws the shapes and paints the picture — and the world passes between them without ever losing its truth.

TRY IT

In a project with a compiled world, run `inkhaven realworld map` and open the image it wrote under `assets/maps/`. Then run `realworld map --spec-only` and render the spec yourself three times, once each with `--map-style parchment`, `inked`, and `blueprint`. Pin the one you like. You have just turned a definition a few lines long into a map of a whole world — and you can redraw it, exactly, any time the world changes.

WHAT YOU LEARNED

- Your compiled world can become a **drawn map**. Inkhaven emits a **MapSpec** (a pure function of the world + seed); the companion tool **plakat** reads it and draws. Install plakat with `cargo install plakat`; it is optional.
- `realworld map` does it in one command — compile, emit the spec, call plakat, write the image + GeoJSON under `assets/maps/`. `--spec-only` writes just the spec so you can drive plakat yourself.
- `plakat map --map-spec … --map-render … --map-style parchment|inked|blueprint` renders the spec; `--map-season`, `--map-grid`, `--map-tiles`, and `--map-render-sd` shape the look. A `--seed` keeps the drawing reproducible.
- Every mark on the map is a layer made visible — relief, rivers, settlements, trade roads, declared landmarks — so the map always agrees with the world your prose is checked against.
- The reverse direction: `--map-dump-heightmap` writes a DEM you feed back through `geology.dem` to grow terrain from a shape. And the round-trip refines your accepted Places' coordinates from the drawn map (`--no-ingest` to skip).

PART VII

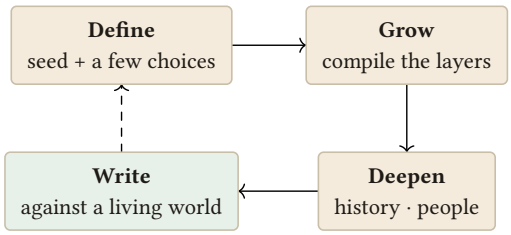
A Complete Walkthrough

CHAPTER 19

19

A World, End to End

Every chapter so far has taught one movement of the work in isolation — the sky on its own, the rivers on their own, the history on its own. That is how a craft is learned, but it is not how a world is built. When you sit down to make one, the movements run together: you set a few numbers, watch the land fall out of them, name a region because the land invited it, read the past that the cities imply, and only then start writing scenes that lean on all of it at once. This chapter is that single, continuous session. We will build one world, called **Tharn**, from an empty folder to a scene you can fact-check — and everything you learned in isolation will arrive, in order, exactly where a working writer would reach for it.



The world you build is not a decoration. You define it, grow it, deepen it — and then write against it, returning to refine as the story demands.

Follow along at your own keyboard if you can. Every command below is one you have met; the point here is not any single one of them but the **shape they make together** — define, grow, deepen, write, and loop back. Read Tharn as a worked example, then build your own the same way.

Define — the starter world

Everything begins with an empty definition. In a project folder, ask Inkhaven to scaffold one:

```
realworld new tharn
```

This writes a starter `world.hjson` — a small, readable file with Earth-like numbers already filled in. Open it and you will find the shape you know: a `name`, a `seed`, a `primary_language`, and an `astronomy` block holding a star, a planet, an orbit, one moon, and a calendar. Nothing exotic yet; a familiar sky you can trust while you decide what to change. Tharn's opening lines read:

```
name: "Tharn"
seed: 0x7a17
primary_language: "Tharnic"
astronomy: {
  star:   { class: "G2V", luminosity_solar: 1.0, age_gyr: 4.6 }
  planet: { mass_earth: 1.0, radius_earth: 1.0, axial_tilt_deg:
23.4,
            day_length_hours: 24.0, rotation_direction:
"prograde" }
  orbit:  { semi_major_axis_au: 1.0, eccentricity: 0.017,
year_length_days: 365.25 }
  moons:  [ { name: "Vael", mass_lunar: 1.0, period_days:
```

```
27.3 } ]
}
```

> **NOTE**
>
> `realworld new <name>` never overwrites an existing `world.hjson`. It gives you a clean, valid starting point every time — a real world you could compile as-is, not a stub full of blanks.

Grow — a sky of our own, then validate

A copy of Earth is a fine place to learn, but Tharn should feel like somewhere. Two small edits to the astronomy will do it. First, a slightly larger axial tilt — push `axial_tilt_deg` from `23.4` to `28.0`, for sharper seasons, a hotter summer and a longer-shadowed winter. Second, a second moon — Tharn will have two, so its nights and its tides have a layered rhythm:

```
axial_tilt_deg: 28.0
moons: [
  { name: "Vael",  mass_lunar: 1.0,  period_days: 27.3 }
  { name: "Corin", mass_lunar: 0.4,  period_days: 11.8 }
]
```

Before growing anything on top of this, make sure it holds together:

```
realworld validate
```

`validate` compiles every layer in turn and reports each one `ok` — it is your proof that the definition is sound before you build on it. Change a number to something impossible and this is what catches it. Tharn validates cleanly; the calendar-consistency check is quiet, the two moons resolve to two synodic periods. The sky is sound. Now let the rest of the world fall out of it.

> **TRY IT**
>
> Before you move on, run `realworld compile --layer astronomy` and read the two moons' synodic periods and the four season markers. You are looking at the raw rhythm — months and tides and solstices — that every later layer will inherit.

Grow — compile the world and read the layers

One command now grows the whole physical world and writes it down where you can read it:

```
realworld compile --materialize
```

The bare compile runs every layer in order; `--materialize` writes each one as a chapter into your World system book. Open it with `Ctrl+B W` and read what emerged, layer by layer — this is the moment the numbers become a place.

Geology came first, seeded from `0x7a17`: a handful of tectonic plates, two continents parted by a strait, a long mountain range thrown up where two plates met, minerals salted into the ranges, a sea level, a heightmap. None of it was placed by hand; all of it followed from the seed.

Climate came next, out of the sky and the land together. Tharn's sharper tilt shows here — the temperate band runs hot in summer, and the lee of that long mountain range is dry, a rain shadow where the wet winds have already spent themselves climbing. The biomes aggregate into zones: taiga in the north, temperate forest and grassland across the midlands, a hot desert behind the mountains, savanna toward the equator.

Hydrology came third, water finding its way downhill across the heightmap. Rivers gather off the mountains and run to the sea; two of them meet at a confluence in the midland forest; a great river reaches the coast at a broad mouth. And the layer marks its **settlement priors** — the river mouth, the confluence, a fertile valley — the sites people would reach for first.

Demographics came last, and put people on exactly those sites. A city at the great river's mouth, a town at the midland confluence, villages scattered where the land supports them, all arranged into a rank-size hierarchy. Tharn now has a

capital-in-waiting — the mouth-city, largest and best-sited — without your having chosen it. The land chose it.

> **NOTE**
>
> `Ctrl+B W` opens the World overview read-only — you are reading the compiled world, not editing it. Everything you see there was written by `--materialize`; to change it you change `world.hjson` and recompile, never the book directly.

Declare — a region and an economy

So far Tharn has grown entirely on its own. Now the author's hand enters. The mouth-city and its river valley deserve a name and a character, so declare a `geography` region over them, and give the world an `economy` to say how it makes its living:

```
geography: {
  regions: [
    { name: "The Vale of Enst", kind: "river_valley",
      center_lat: 34.0, center_lon: -12.0 }
  ]
}
economy: {
  base: "agrarian"
  trade_goods: ["grain", "river-fish", "mountain-iron"]
}
```

Then recompile so the declaration and the physics are reconciled into one world:

```
realworld compile --materialize
```

Nothing you declared overrode the land — the Vale of Enst still sits where the fertile valley was; the iron in the trade goods is the mineral the geology already salted into the range. You did not fight the world; you named what it offered. That is the whole discipline of declaring: the physics is the hand that emerges, your names and rules are the hand you set, and the compiler holds both.

Deepen — a past and a people

The place is complete; now read the time and the peoples folded into it. Four commands, in the order a working writer would run them.

```
realworld history
```

Tharn's history reads the founding order out of the settlements — the mouth-city oldest, the marginal villages youngest — divides it into the **Founding Age**, the **Age of Expansion**, and the **Present Age**, and dates every event in years before the present. A realm rose in the Vale during the Age of Expansion; a smaller inland realm fell two centuries before the present; a migration ran out of the dry lands behind the mountains toward the river forest during a long drought. None of it was decreed. It fell out of where the cities sit.

```
realworld polities
```

The settlements cluster into nations around their largest capitals. Tharn gets two: **Enst**, the river realm centred on the mouth-city, populous and agrarian; and **Kadur**, a lean upland realm along the mountains' dry flank. They are seeded as rivals — the drought migration left a grievance — which is precisely the kind of hook a plot reaches for.

```
realworld culture
```

Each polity gets one culture. Enst's ethos is drawn from its river-valley biome — settled, patient, water-minded — with a belief about the two moons' tides; Kadur's is harder, drawn from the desert's edge. Each culture carries a **language profile** — for Enst, say, `SVO · fusional · non-tonal` — a sketch you can realise in the ConLang suite with `inkhaven language`, plus a naming sample to write against.

```
realworld ecology
```

Finally the living world: flora and fauna archetypes per biome, and a keystone animal for each land biome — the river-forest's great heron, the desert's burrow lizard. Tharn is no longer a map with cities on it. It has a yesterday, two peoples with a quarrel between them, and animals a scene could put on the page.

> **TRY IT**
>
> Run all four — `history`, `polities`, `culture`, `ecology` — and read them as one chronicle. Notice how they agree: the rival polities, the drought migration, the desert ethos, and the burrow lizard are all the **same fact** seen from four angles. That agreement is the world holding together.

The author's hand — calendar and history events

The world has proposed a calendar and a past. Adopt the parts you want, and only those. First the calendar:

```
realworld calendar
```

This derives a story-Timeline calendar from Tharn's astronomy — its months, weekdays, the new year aligned to the spring equinox — and prints lines you adopt into `timeline.calendar`. Now your scenes' dates and your characters' seasons share one root and cannot drift apart.

Then a few history events. `realworld history` printed `inkhaven event add …` lines, one per event. Copy in only the ones your story needs — for Tharn, the founding of the mouth-city and the fall of the inland realm — and leave the rest on the cutting-room floor.

> **NOTE**
>
> The world proposes; you adopt. `calendar` and `history` hand you ready-made command lines and stop. Nothing reaches your Timeline until you run the lines you chose. The author always has the last word.

The author's hand — proposing Places

Settlements become writable Places the same deliberate way:

```
realworld propose
```

The world proposes its settlements as Place entries; `realworld proposals` lists them; you accept or reject each. Accept two for Tharn — **Enstmouth**, the capital at the great river's mouth, and **Karrow**, the midland confluence town — and reject the villages you will never set a scene in. Only the two you accepted become entries in the Places book, taggable in your prose.

At the desk — writing against the world

Now the payoff. Write a scene set in Enstmouth in early autumn, and put the world to work beneath it. Ask what the desk should know:

```
realworld scene --place Enstmouth --day 250
```

The scene brief answers in one breath: the season and weather at Enstmouth's latitude on day 250, its river-valley biome and climate, and the culture of the nearest realm — Enst, water-minded and patient. You are writing with the season, the land, and the people already in front of you.

```
realworld weather --day 250 --lat 34
```

The weather call sharpens it: the local season and the day's weather at latitude 34, so the light and the air in your paragraph are the world's, not a guess.

```
realworld travel --from Karrow --to Enstmouth --days 3 --mode
horse
```

And when your courier rides from Karrow to Enstmouth in three days, `travel` checks the real distance against a horse's pace — and consults the magic ledger's `travel_time` rules, if you declared any — and tells you whether the journey is plausible. For Tharn's midland distances, three days on horseback is comfortable; the world confirms it.

The loop closes — fact-check

Last, let the world read your prose back. Write a deliberately wrong sentence and check it:

```
realworld fact-check --text "In deep winter the courier crossed
the whole of Tharn on foot in a single day."
```

Two things are wrong, and the world flags both: no traveller crosses a continent on foot in a day — the distance and a walker's pace do not agree — and "deep winter" collides with the scene's dated early autumn. The fact-checker measures your sentence against the same compiled world every other command drew from. Fix the sentence, run it again, and it comes back clean.

> **NOTE**
>
> The magic ledger is how you silence a **deliberate** exception. If Tharn's couriers really do cross the world in a day by some declared road-magic, add a `travel_time` rule to the `magic` block; the fact-checker then suppresses that one warning and stops crying wolf. An exception you declared is not a mistake — and the world learns to tell the difference.

> **INSIGHT**
>
> This is the whole loop, and it is a loop on purpose. You **defined** a sky, **grew** a world out of it, **deepened** it with a past and a people, and **wrote** against it — and the fact-check at the end sent you back to the prose, or back to the definition, to reconcile them. A built world is never finished and filed away; it is a thing you return to, tightening the fit between the world and the page each time the story asks a new question of it. That returning — not any single command — is what it means to write inside a world that holds together.

WHAT YOU LEARNED

- Building a world is one continuous loop — **define, grow, deepen, write** — not a set of isolated commands; this chapter ran it end to end on the world **Tharn**.
- You **grow** with `realworld new`, edits to the astronomy, `validate`, and `compile --materialize`, then read the emergent layers in `Ctrl+B W`.
- You **deepen** by reading the past and peoples the land implies — `history`, `polities`, `culture`, `ecology` — and by **declaring** regions and an economy that name what the physics already offered.
- The author's hand is always deliberate: `calendar`, history events, and `propose`/accept adopt only what you choose into the Timeline and Places.
- At the desk, `scene`, `weather`, `travel`, and `fact-check` write and check your prose against the same compiled world — and the check loops you back to refine it.

PART VIII

The World at Hand

CHAPTER 20

20

The Interactive Worldbuilder

Everything so far in this book has passed through one file. You opened `world.hjson` in an editor, typed a block, saved, and ran a command to see what the compiler made of it. That loop is honest and it never lies to you — but it asks you to hold the whole world in your head between edits. This chapter introduces a second way in: a full-screen companion that keeps the world, its score, its facts, and its map all in front of you at once, and lets you shape the world by **asking** rather than by hand-editing HJSON.

It is called the **worldbuilder**, and you start it from your project like this:

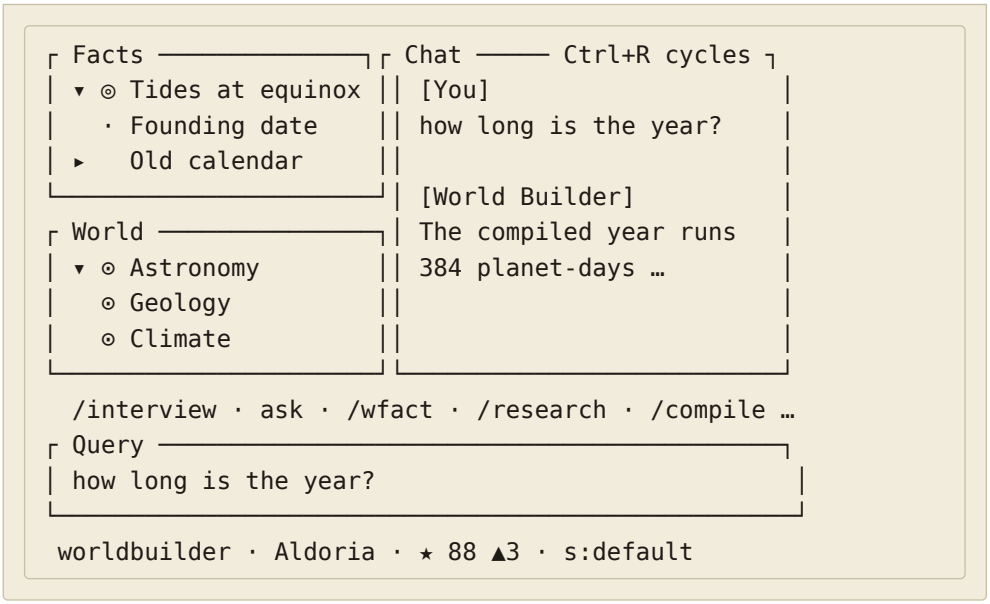
```
inkhaven worldbuilder
```

> **NOTE**
>
> The worldbuilder is a **front-end**, not a replacement. Every change it makes lands in the same `world.hjson` the rest of this book has taught you to read, and the same compiler turns it into a world. Nothing here is magic you cannot also do by hand — the worldbuilder simply keeps the loop tight and shows you the consequences as you go. Your existing `world.hjson` opens unchanged.

The shape of the screen

The worldbuilder fills the terminal with four regions. Down the left run two trees, one above the other: your **Facts** book on top and the **World** book below. To their right is a single wide pane that cycles between four views. Along the bottom is the **Query** prompt — one line where everything you type goes — and beneath it a status bar carrying the world's name and, once there is a world to score, its plausibility.

● ● ● *The worldbuilder, at rest*



Each glyph in the trees means something. In the **Facts** tree, ◎ marks a paragraph you have tied to the world (a *world fact*); a plain · is any other fact. In the **World**

tree, ◎ marks a chapter the compiler owns — you do not edit those by hand; they are re-derived from `world.hjson` on every compile. A fold arrow (▾ open, ▸ closed) sits before anything with children, and the row under the cursor is highlighted. A small pin marker before a row means you have pinned it into the AI's context.

> TERM **The Query prompt**
>
> The single point of entry. Plain words are a question to the worldbuilder; a line that begins with `/` is a *command*. You never leave this prompt to work — you ask, you shape, you record, all from the one line. `Esc` clears the line (and, in an interview, steps out of it).

Move between the panes with `Tab`, and back with `Shift+Tab`; the cycle runs Facts → World → Query → the right pane. The right pane cycles independently with `Ctrl+R` through four views — **Chat**, **Research**, **Map**, and **Ledger**. Resize to taste: `{` and `}` change how the two left trees share their height, `[` and `]` change how much width the left column takes from the right pane. Press `?` to toggle the one-line hint bar shown above the Query prompt; press `Ctrl+Q` to leave.

`Tab` / `Shift+Tab`	cycle panes (Facts → World → Query → Right)
`Ctrl+R`	cycle the right pane (Chat / Research / Map / Ledger)
`j k g G`	move within a tree (up / down / top / bottom)
`h l · Enter`	fold / unfold / step into a tree node
`Ctrl+P`	pin the selected node into the AI context
`Ctrl+T`	toggle the ◎ world-fact tag on a Facts paragraph
`Shift+F`	filter the Facts tree to world facts only
`z`	zoom the focused left tree to fill the column
`{ } · [ ]`	resize the tree split · the column ratio
`? · Ctrl+Q`	toggle hints · quit

Start by being interviewed

If you are beginning from nothing, do not reach for commands. Type `/interview` (or launch with `inkhaven worldbuilder --interview`) and answer plain questions. The worldbuilder walks five short stages — **Sky**, **Land**, **People**, **Rules**, and a closing review — asking one thing at a time.

Mid-interview — the Sky stage

```
┌ Chat ──────────────────────── Ctrl+R cycles ┐
│ [World Builder]                             │
│ Interview — I'll ask about the sky, land,   │
│ people, and rules. Answer in your own words │
│ (blank to skip, Esc to leave).              │
│                                             │
│ [World Builder]                             │
│ [Sky · 1/9] What kind of star? (G Sun-like  │
│ · K orange · M red dwarf)                   │
│                                             │
│ [You]  K                                    │
│ [World Builder]  recorded · star → K  (★ ▲2)│
│                                             │
│ [World Builder]                             │
│ [Sky · 2/9] Axial tilt in degrees? (Earth   │
│ 23.4 — higher means harsher seasons)        │
└─────────────────────────────────────────────┘
 interview — answer in the Query prompt · Esc to leave
```

Every answer becomes a **pending edit** — a proposed change to `world.hjson` that has not been committed yet — and you watch each one recorded in the conversation. A blank line skips a question; `Esc` leaves the interview at any point without losing what you have already answered.

> **INSIGHT**
>
> The plausibility score at the bottom of the screen moves as you answer, so you feel the shape of your world firming up before you have written a single line to disk. When the interview ends, review everything at once with `/diff`, then commit with `/write`.

Shaping by command

Once you know your way around, the shaping commands are faster than the interview. Each one proposes a precise edit and shows it to you for confirmation before it joins the pending delta.

`/set <path> <value>`	set any dotted key, e.g. `/set geology.gener 0.6`
`/star <class>`	the star's spectral class — `G`, `K`, `M`
`/tilt <degrees>`	axial tilt; higher means harsher seasons
`/moon <name> [days]`	add a moon, optionally with its period
`/nation <name> [era] [kind] [traits…]`	add a nation
`/magic on|off`	enable or disable the magic ledger
`/rule <kind> <cat,cat> [description]`	declare a magic rule (enables the ledger)

Type a shaping command and the worldbuilder opens a small confirmation over the screen, showing exactly what will change:

● ● ● *A shaping delta, awaiting confirmation*

```
┌ Confirm delta → world.hjson ──────────────┐
│ moon Lunara                               │
│                                           │
│   astronomy.moons[] += {"name":"Lunara",  │
│     "period":29.5}                        │
│                                           │
│ y accept (into pending) · n/Esc discard   │
│ · then /write to commit                   │
└───────────────────────────────────────────┘
```

Press `y` to fold the edit into the pending delta, or `n` to discard it. Nothing touches `world.hjson` until you say so.

TERM The pending delta

The stack of accepted-but-uncommitted edits. `/diff` lists it, `/undo` drops the last edit, `/reset` clears it, and `/write` folds all of it into `world.hjson` at once — atomically. The pending delta is *saved with your session*, so if you quit mid-thought, your uncommitted edits are waiting when you return.

Seeing what your choices imply

Declaring a world is only half of it; the point is the **consequences**. Two commands bring them to the surface. `/compile` runs the whole deterministic chain — astronomy, geology, climate, hydrology, demographics — over your world as it stands (disk plus pending edits) and reports the compiled state. From that moment the worldbuilder reasons over the *simulated* world, not merely what you declared.

`/validate` runs the plausibility lints and reports the score with every warning, graded high, medium, or low. This is the same score that rides in the status bar; the command spells out what is costing you points.

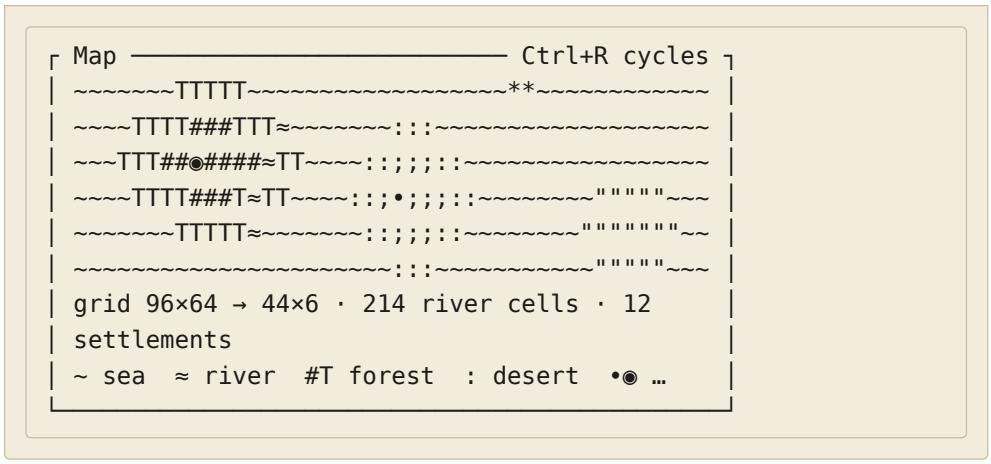
● ● ● */compile and /validate, reported into Chat*

```
┌ Chat ──────────────────────── Ctrl+R cycles ┐
│ [You]  /compile                             │
│ [World Builder]                             │
│ Compiled world state —                      │
│ World: Aldoria                              │
│ Astronomy: 0.82 solar-mass star · year 384  │
│   planet-days · axial tilt 23.4° · 1 moon   │
│ Geology: 96×64 · 3 continent(s) · 61% sea   │
│ Climate: mean land 12.4°C · 7 biome zone(s) │
│ Demographics: 4.1M people · 12 cities …     │
│                                             │
│ [You]  /validate                            │
│ [World Builder]  Plausibility 88/100 —      │
│   1 warning(s):                             │
│   [MED ] nations: capital is landlocked     │
└─────────────────────────────────────────────┘
```

TRY IT

Cycle the right pane to **Map** with `Ctrl+R` and run `/compile`. The pane fills with an ASCII minimap drawn straight from the compiled biomes — sea, forest, desert, ice — with rivers and settlements stamped over it. It needs no external tool and works on any terminal, so you always have a picture of the world your numbers describe.

The Map pane, after /compile



Recording what is true

Not everything about a world is physics. The name of a harbour, the reason two houses feud, the festival that moves with the second moon — these are **facts** you decide, and the worldbuilder records them without ever inventing them for you.

```
/wfact The tides run backwards at the autumn equinox
```

writes that statement into your Facts book, tagged so it shows with a ◎ in the Facts tree and feeds back into the worldbuilder's own context. To find related material you have already recorded, `/research <query>` retrieves matching Facts into the Research pane, where a ◎ marks the ones already tied to the world.

The Research pane after /research tides

```
┌ Research ──────────────────── Ctrl+R cycles ┐
│ query: tides                                │
│                                             │
│ ◎ Facts / Hydrology / Tides at equinox 0.71 │
│   The tides run backwards at the autumn     │
│   equinox, when both moons align.           │
│ · Facts / Calendar / Founding date     0.44 │
│   The realm was founded in the Year of …    │
└─────────────────────────────────────────────┘
```

PITFALL

The worldbuilder never edits your prose and never writes fiction for you. `/wfact` records *your* words verbatim; the AI in the Chat pane answers questions and points out contradictions, but the decisions — and the sentences — are always yours.

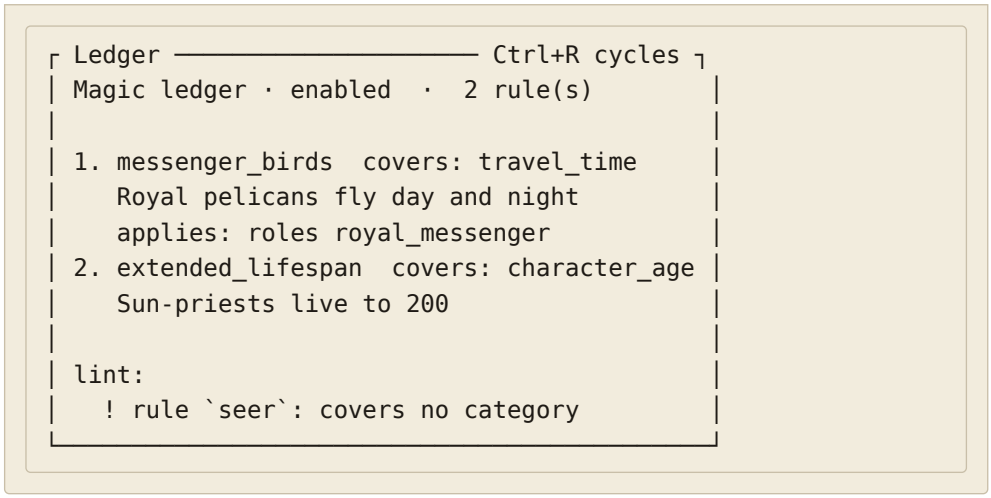
The magic ledger

If your world breaks physics on purpose, say so, and the fact-checker will stop flagging it. The **Ledger** pane (cycle to it with `Ctrl+R`) shows the world's declared exceptions and lints them live — a rule that covers no category, a ledger left disabled, a duplicate. Declare a rule from the Query prompt:

```
/rule messenger_birds travel_time Royal pelicans fly day and
night
```

That enables the ledger and adds a rule covering the `travel_time` category, exactly as Chapter 13 described — only now you watch it appear, and its lint, in the pane.

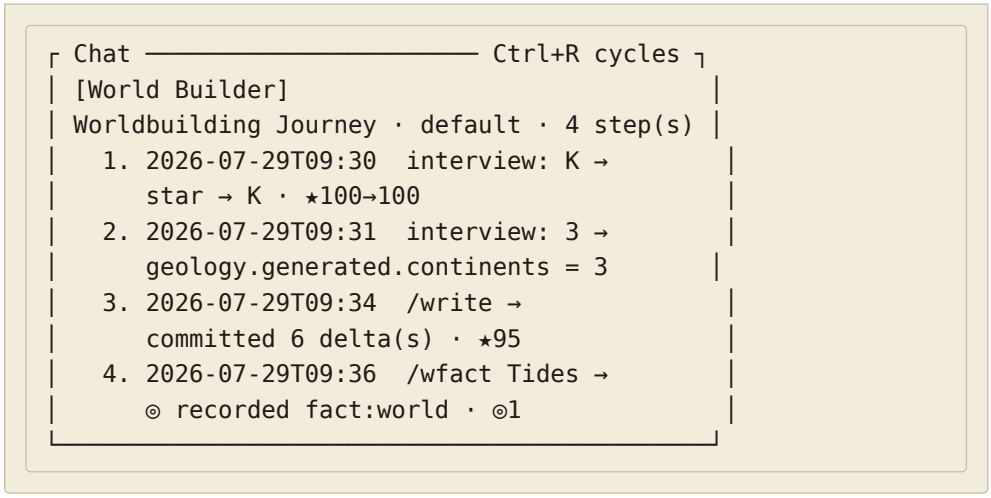
The Ledger pane



The journey, and taking it with you

Your worldbuilding is a **record**, not just a result. Every accepted edit, every committed write, every recorded fact is a step in the session's timeline. See it with `/journey`, which prints each step with its plausibility arc — how the score moved:

/journey — the session timeline



Sessions are named — `inkhaven worldbuilder --session aldoria-v2` — and `/sessions` lists them. When you want the world out of the tool and onto paper, `/export` assembles a single readable Markdown dossier — the compiled state, the plausibility report, the magic ledger, your recorded facts, and the whole journey — and writes it under `exports/` in your project. It is a record you can read, share, or drop into an appendix.

INSIGHT

The worldbuilder measures, validates, and records; you decide. That is the whole posture of this book, made interactive. The `world.hjson` it leaves behind is the same file the compiler, the materialiser, and the fact-checker have read all along — so everything you learned in the previous nineteen chapters is exactly what the worldbuilder is doing on your behalf, one question at a time.

WHAT YOU LEARNED

- `inkhaven worldbuilder` is a four-pane front-end to the `realworld` pipeline: Facts and World trees, a cycling right pane (Chat · Research · Map · Ledger), a Query prompt, and a live plausibility score. Every change lands in `world.hjson`.
- Build by **asking** — `/interview` walks Sky · Land · People · Rules; or shape directly with `/set`, `/star`, `/tilt`, `/moon`, `/nation`, `/magic`, `/rule`.
- Edits accumulate as a **pending delta** — previewed, `/diff`-able, `/undo`-able, saved with the session — and commit atomically with `/write`.
- `/compile` makes the AI reason over the simulated world; `/validate` grades the plausibility warnings; the Map pane draws the compiled biomes on any terminal.
- Record world facts with `/wfact` (◎, retrievable via `/research`), keep a `/journey`, and `/export` a Markdown dossier — the worldbuilder never writes prose for you.

CHAPTER 21

21

The Map Editor

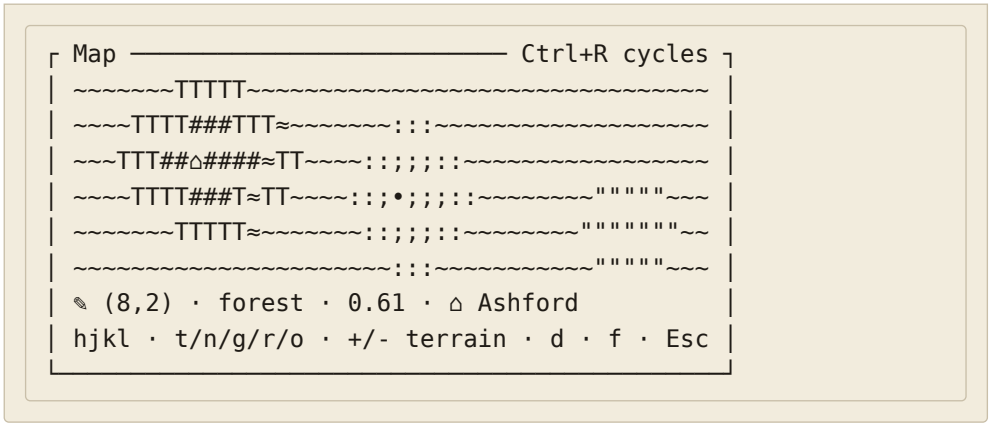
Chapter 18 sent your world **out** to plakat to be drawn; Chapter 20 gave you the worldbuilder, where the Map pane shows that drawing at your desk. This chapter joins the two: it turns the Map pane into a place you **draw** — set a town where you want a town, run a river to the sea, sculpt a coastline — and watch the world take your edits. Nothing here leaves the model you already know. Every mark you make is an ordinary `world.hjson` declaration; the map editor is a spatial front-end to the same `geography` and `hydrology` blocks you could type by hand, and the same compiler turns them into a world.

> **NOTE**
>
> The map editor lives inside `inkhaven worldbuilder`. Cycle the right pane to **Map** with `Ctrl+R`, then press `e` to edit. You need a compiled world first — run `/compile` (or `/map`) so there is terrain under the cursor. Every placement is a **pending edit**: review the batch with `/diff`, commit with `/write`.

The cursor

Press `e` and a crosshair appears on the map. Move it with `hjkl` or the arrow keys — a coarse step for travel, `Shift` for a single cell — and a readout tells you exactly where you are and what is under you: the grid coordinate, the biome, the elevation, and the name of any feature on that cell.

● ● ● *Edit mode — the cursor over a placed town*

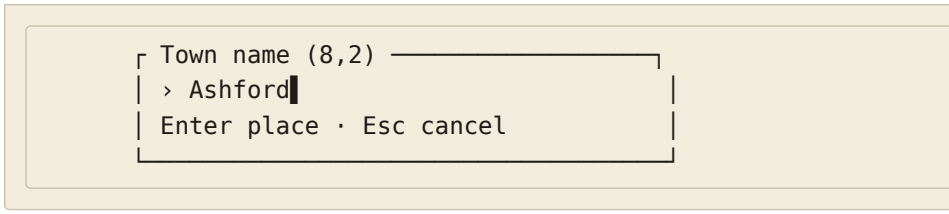


The glyphs read as a map: `~` sea, `T`/`#` forest, `:`/`;` desert, `"` grassland, `•` a procedural settlement, and `⌂` a landmark you placed. The single-key tools along the bottom are the whole editor; the rest of this chapter is those keys, one at a time.

Towns and landmarks

Put the cursor where you want a place and press `t` for a town or `n` for a named landmark. A small prompt opens; type a name and press `Enter`.

Naming a town at the cursor

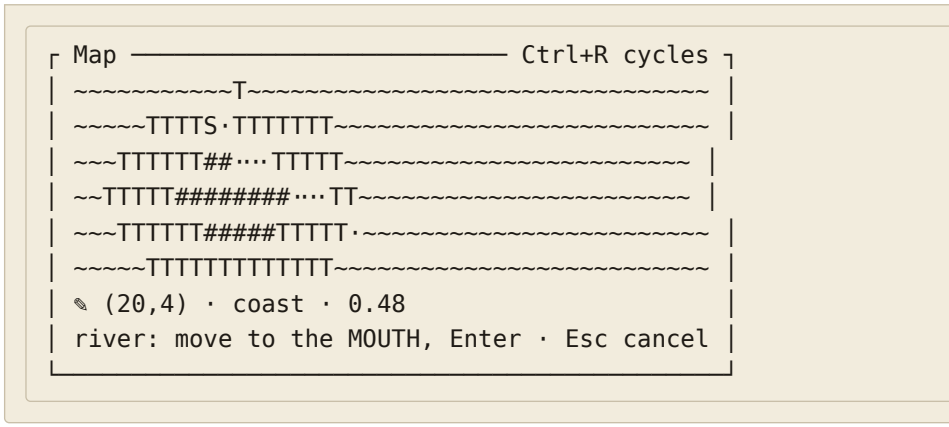


The town joins your `geography.landmarks[]` at the cursor's cell, drawn `⌂`, and — once you `/write` — appears on the plakat raster too. Press `d` to delete the feature under the cursor. Because a landmark carries a name, it also becomes a gazetteer entry the fact-checker can resolve when it reads your prose.

Rivers

A river is two points. Press `r`, move to the **source** and `Enter`, move to the **mouth** and `Enter`, then name it. As you go, a provisional course follows the cursor from a fixed `S`.

Drawing a river from source to mouth



The river lands in `hydrology.rivers[]` with its source and mouth cells, and the compiler **honours the declared course** — so after `/write` it carves its way to the sea and draws as `≈`.

> **INSIGHT**
>
> The river is **checked as you draw**. The worldbuilder runs the same `lint_rivers` the physics uses, so the moment you place a course that runs uphill, or ends its mouth above the shoreline, it tells you — "river runs uphill: its source sits below its mouth". You fix the map against the world, at the desk, before you commit.

Regions and roads

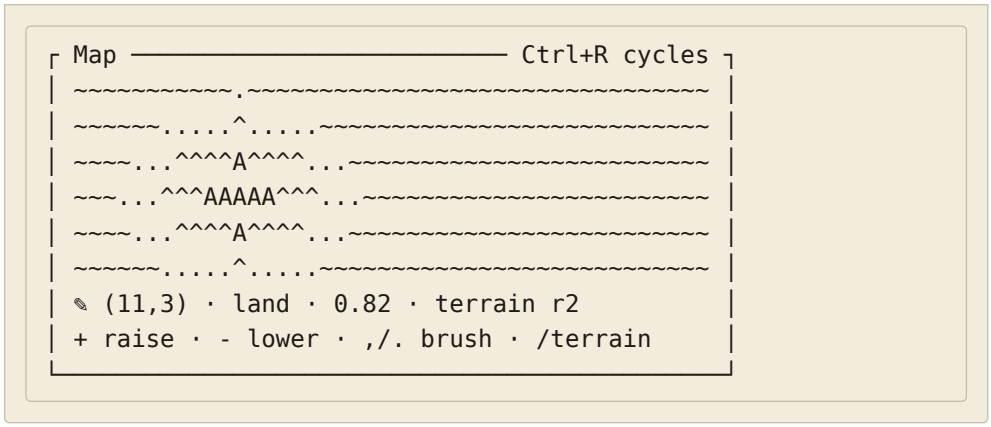
Two more tools round out the human layer. Press `g` to drop a **region** at the cursor; its biome is filled in from the cell under you, so a region you place in the forest is a forest region. It joins `geography.regions[]`, drawn `§`.

Press `o` to route a **road**. A road connects two of your named landmarks, so move to the first and `Enter`, then to the second and `Enter`. It lands in `geography.roads[]` and, because it references landmarks the map already knows, draws between them as `=` on the worldbuilder map **and** on the plakat raster.

Sculpting the land

The tools so far annotate the world; this one **changes** it. Press `+` to raise the land under a soft brush and `-` to lower it (`,` and `.` size the brush). The map re-shades live as you sculpt — `~` sea, `.` lowland, `^` hills, `A` peaks — so you can raise a mountain range or carve a bay and watch it appear.

Raising a mountain range with the brush



When the shape is right, `/terrain` writes the sculpted heightmap as a grayscale **DEM** under `assets/maps/` and points `geology.dem` at it (a pending edit). The `realworld` compiler reads DEMs, so a `realworld compile` rebuilds the **whole** world — climate, rivers, settlements — from your terrain.

> **PITFALL**
>
> Terrain is the one edit the worldbuilder's own `/compile` does not yet re-simulate: it previews your sculpt and writes the DEM, but its live climate/biomes stay procedural until you compile with `realworld` (which is DEM-aware). Sculpt, `/terrain`, `/write`, then `inkhaven realworld compile` to see the land reshape everything.

Checking the map

Placing things by eye, you will sometimes drop a harbour on the wrong side of a coastline. `/mapcheck` reads the whole map layer against the compiled world and flags the mistakes — a town in open ocean, a coordinate off the map, a region in the sea — right on the cells where they sit. Press `f` to jump the cursor from one to the next.

● ● ● *A town in the sea, flagged by /mapcheck*

```
┌ Map ──────────────────────────── Ctrl+R cycles ┐
│ ~~~~~~~TTTTT~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~ │
│ ~~~~TTTT###TTT≈~~~~~~~:::~~~~~!~~~~~~~~~~~~~ │
│ ~~~TTT#######≈TT~~~~::;;;;::~~~~~~~~~~~~~~~~ │
│ ~~~~TTTT###T≈TT~~~~::;;;;;;::~~~~~~~"""""~~~ │
│ ~~~~~~~TTTTT≈~~~~~~~::;;;;::~~~~~~~"""""""~~ │
│ ~~~~~~~~~~~~~~~~~~~~~:::~~~~~~~~~~"""""~~~ │
│ ! settlement 'Sunkport' in open ocean (30,1) │
│ f: jump to next issue                        │
└──────────────────────────────────────────────┘
```

The mouse

If your terminal supports it, the map takes the mouse: a left-click in the pane focuses it, enters edit mode, and drops the cursor on the cell you clicked. Placement still happens with the single keys — click to point, then `t`, `r`, `+`, and the rest — so a world can be laid out as fast as you can aim.

WHAT YOU LEARNED

- In the worldbuilder's Map pane, `e` enters edit mode: a cursor with a readout of coordinate, biome, and elevation. Every tool writes an ordinary `world.hjson` edit into the pending delta — `/diff`, then `/write`.
- `t`/`n` place towns and landmarks (`geography.landmarks[]`, ⌂); `r` draws a river (`hydrology.rivers[]`, ≈) checked as you go; `g` a region (§); `o` a road between landmarks (=); `d` deletes.
- `+`/`-` sculpt terrain under a brush; `/terrain` writes a DEM and points `geology.dem` at it, so `realworld compile` rebuilds the world from your land.
- `/mapcheck` flags map-layer mistakes — a town in the sea, an off-map coordinate — on the cells where they sit; `f` jumps between them. A left-click positions the cursor.

APPENDIX A

A

Command Reference

You never have to memorise these. Every command in this appendix was taught, in context, in the chapter where you needed it — this page exists only for browsing, for the moment at the keyboard when you remember **what** you want to do but not the exact word for it. The commands are grouped by the part of the process they belong to, in the same order the book teaches them: first defining and growing the physical world, then deepening it with a past and a people, then the author's deliberate hand, then working at the desk, and finally the bridges that carry the world into your manuscript.

Every command below is a subcommand of Inkhaven: prefix each with `inkhaven`. When a row reads `realworld compile`, the command you type

is `inkhaven realworld compile`. The `realworld` group is Inkhaven's world builder; the few rows outside it (`Ctrl+B W`, `inkhaven event add`) are marked.

Define & grow

`realworld new <name>`	Scaffold a starter world.hjson with Earth-like defaults; never overwrites an existing one.
`realworld validate`	Compile every layer in turn and report each one ok — your proof the definition is sound before you build on it.
`realworld compile`	Compile the whole world — every layer, in order: astronomy → geology → climate → hydrology → demographics. `--materialize` writes it all into the World book, the human half (nations, cultures, ecology) included.
`… --layer <name>`	Compile just one named layer (or all) and read what it found on its own.
`… --materialize`	Write the compiled layers as chapters into the World system book.
`… --json`	Emit the result as structured data, for tools and scripts.
`realworld variants --count N`	Propose N candidate worlds from consecutive seeds (a one-line summary each) so you can pick a seed — the world proposes, you choose.
`realworld show`	Print the world definition; --json for structured form.
`Ctrl+B W`	Open the read-only World overview — every compiled layer, plus a "This scene" header when the cursor is in a scene.

Deepen — a past and a people

`realworld history`	Infer a founding chronology, three epochs (Founding / Expansion / Present Age), and events (realm rise & fall, migrations), dated in years before the present.

realworld chronicle	The same past as a state trajectory: for each epoch, how far the world had grown by then — settlements, settled population, realms standing — beside its events.
… --materialize	Write the chronology as a History chapter in the World book.
… --json	Emit the whole chronology as structured data.
realworld polities	Cluster settlements into nations around their largest capitals — names, populations, seeded relations (allied / rival / neutral).
realworld culture	Give each polity a culture — an ethos from its capital's biome, a belief, a language profile for the ConLang suite, a naming sample, and the world's common social roles in that realm's own terms.
realworld name	Propose a name for each settlement in its realm's own phonic style, so a realm's towns share a family sound instead of the generic placeholders. A naming aid you adopt on accept.
realworld ecology	Generate flora and fauna archetypes, with a keystone animal per land biome.
realworld trade	The trade network: each realm links to its nearest non-rival neighbours, by land road or sea lane. Connectivity, not simulated economics — drawn on the map as roads.

Declare & the author's hand

realworld calendar	Derive a story-Timeline calendar from the astronomy; prints lines you adopt into timeline.calendar.
realworld magic	Show and validate the magic ledger — the declared exceptions to physics.
realworld propose	The world proposes its settlements as Place entries for you to accept or reject.
realworld propose-myth	The world reads your cultures' beliefs and proposes Mythology symbols and motifs for you to accept into the Mythology book.
realworld propose-rulers	Propose one ruler per realm — a Character stub named in style and rooted in the culture — for you to accept into the Characters book.

realworld propose-language	Propose one language per culture (from its profile + naming sample); accept to scaffold a language book in the ConLang suite, seeded with the world's brief.
realworld proposals	List the pending proposals — Places, Mythology entries, and rulers alike — awaiting your decision.
realworld critique	An AI reading of the whole world: consistency + realism recommendations. `--write-notes` files them into the Notes book. `--lints-only` skips the AI.

At the desk

realworld scene --place <name> --day <N>	A scene brief: season and weather at the place's latitude, its biome and climate, the nearest realm's culture, and the nearest named feature — Place, declared landmark, or named water (distance + bearing).
realworld weather --day <N> --lat <deg>	The local season and weather at a day-of-year and latitude.
realworld travel --from <p> --to <p>	Is a journey plausible? Checks real distance against the mode's pace; consults the magic ledger's travel_time rules.
… --days <D> --mode <m>	Set the days allowed and the mode: foot, horse, cart, or ship. (Also --from-x/--from-y coordinates.)
realworld fact-check --text "…"	Check prose against the world — travel time, climate, date coherence — with declared magic exceptions suppressed.
… --paragraph <id>	Fact-check a manuscript paragraph by its id instead of literal text.
realworld co-location	Flag co-location conflicts — a character in two places at overlapping times — from the story Timeline; respects the magic ledger. Deterministic, zero-AI.
realworld coherence <node>	An LLM pass over every paragraph under a node (book / chapter), looking for contradictions between them. Cost-capped (`--max-cost`, `--force`).

Bridges to your book

realworld gazetteer	A consolidated Markdown world reference — calendar, sky, regions, landmarks, waters, settlements, economy, magic.
… --output <path>	Write the gazetteer to a chosen path.
realworld map	Render a map with the external plakat tool (optional). Draws your settlements, coordinate-bearing Places, declared `geography.landmarks` with a position, realm-capital hubs, and the trade routes between them as roads.
realworld set-coords <name> --lat <d> --lon <d>	Give a Place (compiler-born or hand-authored) a location on the world grid so plakat draws it. Grid `--x`/`--y` also accepted.
realworld places	List the Place ↔ World cross-references — each accepted Place with its climate zone, biome, hydrology basis, and grid coordinates.
inkhaven event add …	Adopt a chosen history event onto the story Timeline — the lines realworld history prints for you.
inkhaven language	Realise a culture's language profile in the ConLang suite.
inkhaven myth scan	Map where your declared symbols fall across the chapters — a density heatmap, zero-AI.
inkhaven myth check	Ask whether the prose keeps faith with your declared symbols, motifs, and archetypes — advisory, never edits.

APPENDIX B

B

Glossary

Every term this book defined, gathered in one place and in alphabetical order — both the worldbuilding ideas and the Inkhaven ones — so you can settle any word without hunting for the chapter that first introduced it.

Archetype
A role a generated element plays rather than a named individual — a settlement's role archetype, or a flora, fauna, or keystone-animal archetype in the ecology. The world sketches the type; you write the particular.

Authority
The discipline that the world proposes and the author decides. Nothing the world generates is written into your manuscript without you accepting it, and you may always override what it inferred. The author has the last word.

Axial tilt
The angle between a planet's spin axis and the line straight up from its orbit — Earth's is about 23.4°. It is why there are seasons at all, and the single most consequential number in the sky.

Biome
A large-scale kind of living landscape defined by its climate — the world recognises twelve, from ice_cap and tundra through the forests, grasslands, and deserts to tropical_rainforest and ocean. Biomes aggregate into climate zones and decide what can live and grow where.

Canon
What is true in your world — the settled body of facts your prose must answer to. The compiled world is the source of it; the fact-checker measures your sentences against it.

Capital
The largest, best-sited settlement of a polity, around which its cluster of cities is gathered. Because prime sites are claimed first, a capital is usually also among the oldest cities.

Carrying capacity
How large a population a patch of land can support, given its climate, water, and fertility. It is why cities grow where they do — river mouths and fertile valleys carry more people than marginal ground.

Chronology
The ordering of events along time — what came before what, and how long ago. Your world's chronology is inferred from where its cities sit and how large they grew, and dated on the world's own calendar.

Compile
To run the world's layers, in order, turning the small world.hjson definition into a full world — plates, climate, rivers, settlements, and more. Compiling is pure and repeatable: the same definition always yields the same world.

Consistency
The property that the pieces of a world hold together and never contradict one another. Good worldbuilding is measured less by how much you invent than by how well it stays consistent; the compiler and fact-checker exist to protect it.

Continent
A large landmass raised by the geology layer where tectonic plates meet and part. Continents, their coastlines, and the seas between them all follow from the seed, not from a hand-drawn map.

Continuity
Consistency held across time and across your manuscript — the same tower the same age in chapter three and chapter twelve, the same season the same date wherever it recurs. The shared root of calendar and climate is what keeps continuity from drifting.

Continuity error
A place where the manuscript contradicts the world or itself — a journey too fast for the distance, a season wrong for the date, a tower two different ages. The fact-checker exists to catch them before a reader does.

Culture
The way of life of a people — one per polity — with an ethos drawn from its capital's biome, a belief, a language profile, and a naming sample. Cultures give a world its peoples, not just its populations.

Declared
A fact you set by intention in world.hjson — a name, an economy, a magic rule, a river's course — as opposed to one the compiler emerges from the seed. The world honours what you declare and grows the rest around it.

DEM
A digital elevation model — a real heightmap image. If geology.dem is set, the geology layer builds its terrain from that image instead of from the seed, letting you ground a world on real or hand-made relief.

Demographics
The last physical layer: where people settle and how many. Derived from climate and hydrology, it places settlements, assigns populations and role archetypes, and arranges them into a rank-size hierarchy.

Deterministic
Producing the same result every time from the same inputs. The world compiler is fully deterministic and runs offline — a given definition and seed always grow exactly the same world, so your world is reproducible and shareable.

Ecology
The living layer of the world — flora and fauna archetypes per biome, with a keystone animal for each land biome — read out by realworld ecology so a scene has plants and creatures consistent with its climate.

Emergence
The way each layer arises as a consequence of the ones before it — the sun shaping the climate, the climate carving the rivers, the rivers deciding where cities stand. You set initial conditions; the world emerges from them rather than being placed by hand.

Epoch
A named stretch of a world's history with its own character. The history command produces three, oldest to most recent: the Founding Age, the Age of Expansion, and the Present Age.

Ethos
The character of a culture, drawn from its capital's biome — settled and water-minded for a river valley, harder for a desert's edge. The ethos is the seed of how a people think and behave.

Exception to physics
A rule you deliberately declare in the magic ledger that overrides what the physical world would otherwise insist on — a road that folds distance, a season that does not behave. Once declared, the fact-checker honours it and stops warning about it.

Fact-check
Checking your prose against the compiled world — travel time, climate, date coherence, and more — with realworld fact-check. Declared magic exceptions are suppressed, so a one-time setup does not produce endless false warnings.

Gazetteer
A consolidated Markdown world reference — calendar, sky, regions, landmarks, waters, settlements, economy, and magic in one document — produced by realworld gazetteer for reading or sharing.

Insolation
The amount of a star's light and heat falling on a patch of ground over time. It varies with latitude and season, and is the raw energy budget the climate layer spends to make temperature and rain.

Keystone species
The one animal per land biome that anchors its ecology — the creature a scene in that biome would most naturally put on the page. The ecology layer names one for each land biome.

Language typology
A description of a language by its structural type rather than its words — word order (SVO, SOV, …), morphology (isolating, agglutinative, fusional), and sound. A culture's language profile is a typology sketch the ConLang suite realises into a real tongue.

Layer
One stage of the compiled world — astronomy, geology, climate, hydrology, or demographics — each computed in order and each a consequence of the ones before it. The layers are the physical world, grown one at a time.

Magic ledger
The magic block of world.hjson: declared exceptions to physics, each a rule naming what it covers and whom it applies to. The fact-checker consults it so a deliberate departure from physics is honoured rather than flagged.

MapSpec
The JSON bridge between Inkhaven and plakat — a description of a world's cartography (coast, mountains, rivers, regions, labelled places, roads) emitted from the compiled layers and drawn by plakat. A pure function of the world and its seed, so the same world always yields the same map.

Materialize
To write the compiled world down as readable chapters in the World system book, with compile –materialize (or history –materialize). Materialising records the world's proposal; it does not touch your manuscript.

Mythology
The system book of an author's declared symbols, motifs, and archetypes. Inkhaven reads only what you declare there and checks the manuscript keeps faith with it; realworld propose-myth seeds it from your cultures' beliefs.

Places book
The system book of your settings — towns, cities, regions, landmarks. The world proposes its settlements into it; you author the rest, and give any of them a position on the map with realworld set-coords.

plakat
Inkhaven's companion cartography tool — a separate binary that reads a MapSpec and draws a finished, labelled map, and that can grow or dump a heightmap. Optional, installed with cargo install plakat.

Polity
A nation — a cluster of settlements gathered around a capital, with a name, a population, and seeded relations (allied, rival, or neutral) with its neighbours. Polities are read out of the settled world by realworld polities.

Proposal
Something the world offers for you to accept or reject — a settlement as a Place, a calendar, a history event. The world proposes; you decide; only what you accept reaches your book.

Rain shadow
The dry region in the lee of a mountain range, where winds arrive having already spent their moisture climbing the windward side. It is why the far side of a range can be desert while the near side is green.

Rank-size hierarchy
The orderly spread of settlement sizes — a few large cities, more towns, many villages — that the demographics layer arranges, mirroring how real settlement sizes distribute rather than clumping at one scale.

Rule
One entry in the magic ledger — a named exception to physics that says which fact-check categories it may cover and whom it applies to. A rule is how you teach the world that a departure from physics is intended, not an error.

Scene context
What the desk knows about the scene at your cursor — its place, season, and people — surfaced by realworld scene, the ambient footer chip, and the "This scene" header in the World overview. The world present while you write.

Seed
A single number that fixes all the "random" choices a world involves — where coastlines fall, which valley grows the first city. The same seed always produces the same world, so your world is reproducible.

Settlement
A place people live — classed as a city, town, or village — placed by the demographics layer where the land supports it, on the good sites the hydrology marked: river mouths, confluences, fertile valleys.

Solstice / Equinox
The four turning points of the year. At an equinox the tilt favours neither hemisphere, so day and night are equal; at a solstice one hemisphere leans most toward the star (its summer) and the other away (its winter).

Sphere of influence
The reach of a polity around its capital — the settlements it gathers and the ground it claims. Where two spheres meet is where a contested border belongs.

Synodic period
The time a moon takes to return to the same phase as seen from the ground — new moon to new moon. It differs from the raw orbital period because the planet is itself moving, and it is the rhythm a lunar month keeps.

System book
A dedicated project-wide book Inkhaven keeps beside your manuscript — Places, Characters, the Timeline, and the World book — readable, searchable, and citable from your prose.

Tectonic plate
A section of the planet's crust whose movements the geology layer simulates. Where plates meet and part, the world raises mountains, opens seas, and shapes continents.

The present (year 0)
The moment your story is set, from which the world's history is dated. Events before it carry negative years; the compiled chronology runs up to it.

The World book
The system book that holds your compiled world — every materialised layer written as readable chapters, opened read-only with Ctrl+B W. It is where the world you grew is written down.

Timeline
The system book that holds your story's dated events. The world's calendar drives it, and history events can be adopted onto it with inkhaven event add.

Trade route
A link in the trade network between two non-rival realms — a land road or a sea lane, by whether their capitals sit on a coast. Read out by realworld trade and drawn on the map. Connectivity, not simulated economics.

Watershed
The area of land that drains to a single river or lake — the catchment whose rainfall a river gathers. The hydrology layer traces watersheds as it runs water downhill across the terrain.

world.hjson
The single file that defines a world: its name, seed, primary language, and the blocks — astronomy (required), geology, geography, hydrology, economy, magic. Everything the compiler grows begins here.

Worldbuilding
The craft of inventing a setting — its geography, history, peoples, and rules — thoroughly and consistently enough that no reader ever catches it contradicting itself. Less about how much you invent than about how well the pieces hold together.

APPENDIX C

C

The world.hjson Keys

Every value you can set in `world.hjson`, block by block. Only `name` and the `astronomy` block are required; every other block is optional, and every layer that lacks an explicit block is generated from the `seed`. Types are given as `number`, `text`, `list`, or `block`; where a value has a natural default (the Earth-like starter), it is noted.

Top level

name
`text` — the world's name. Required.

seed
`number` — the single value that fixes every generated choice (coastlines, which valley grows the first city). Accepts a decimal integer or a `0x…` hex value. The same seed always grows the same world. Defaults to `0`.

primary_language
`text` — the world's common tongue, used when labelling materialised output. Defaults to `"en"`.

astronomy — the sky (required)

Holds four sub-blocks and a calendar. Everything downstream — climate, rivers, settlement — depends on it.

star

class
`text` — spectral class, e.g. `"G2V"` (a sun like ours) or `"K"`, `"M"` for cooler, redder stars.

luminosity_solar
`number`, required — brightness in units of our Sun. `1.0` is Sun-like; raise it for a hotter, wetter world, lower it for a colder one.

mass_solar
`number`, optional — mass in units of our Sun. Omit it and the sky derives a plausible mass from the luminosity; set it to override.

age_gyr
`number` — the star's age in billions of years. Descriptive.

planet

mass_earth
`number` — planet mass in Earth masses (`1.0` = Earth).

radius_earth
`number` — planet radius in Earth radii; sets the world's physical size, and so the real distance a rider or ship must cross.

axial_tilt_deg
`number` — the tilt of the axis in degrees. Earth is `23.4`. Tilt drives the **strength** of the seasons: near `0`, seasons all but vanish; large tilts give violent ones.

day_length_hours
`number` — the length of one rotation, in hours.

rotation_direction
`text` — `"prograde"` (like Earth) or `"retrograde"`, which flips the prevailing winds.

orbit

semi_major_axis_au
`number` — orbital distance in astronomical units (`1.0` = Earth's distance from the Sun).

eccentricity
`number` — how elliptical the orbit is (`0` is a circle; Earth is `0.017`).

year_length_days
`number` — the declared length of the year in days. The sky computes the **true** length from the orbit and flags a year that contradicts it (see the calendar check).

moons
A `list` of moons, each a `block`:

name
`text` — the moon's name.

mass_lunar
`number` — mass in units of our Moon; larger moons raise bigger tides.

period_days
`number` — orbital period in days, from which the synodic period (the visible month) is computed.

eccentricity
`number`, optional — how elliptical the moon's orbit is (`0` is a circle). Descriptive.

calendar

months
`number` — months in the year.

month_length_days
`number` — days in a month.

weekdays
`number` — days in a week.

month_names
`list` of `text` — optional names for the months; used when the calendar is adopted into the story Timeline.

day_names
`list` of `text` — optional names for the days of the week.

new_year_aligns_to
`text` — the season marker the new year begins on, e.g. `"winter_solstice"`, `"vernal_equinox"`.

geology — the land (optional)

Omit this block and the land is generated from the `seed`. Include it to **steer** that generation with a `generated` sub-block, or to supply your own heightmap with a `dem` sub-block. If both are present, the `dem` wins.

generated

Knobs on the seed-grown land. Every key is optional; the values below are the defaults.

plates
`number` — how many tectonic plates the world starts with (default `7`). More plates → more, smaller landmasses and coastlines.

continents
`number` — the target number of continents (default `4`).

mountain_orogeny
`text` — `"active"` (default, tall young ranges), `"quiet"`, or `"ancient"` (low, worn-down ranges). Drives peak elevation.

sea_level
`number` `0.0`–`1.0` — the land/sea threshold on the height scale (default `0.4`). Raise it to drown the map, lower it to expose more land.

volcanism
`text` — `"quiet"` / `"moderate"` / `"active"`. Descriptive.

mineral_richness
`text` — `"sparse"` / `"normal"` / `"rich"`. Descriptive.

notable_minerals
`list` of `text` — named minerals the land holds; fed to the economy fact-checker (alongside `economy.resources`) so a claim about a metal the world does not have is flagged.

dem

dem
`block` — bring-your-own-map. `dem.path` (`text`) is the heightmap image, relative to the project root; `dem.scale_km_per_pixel` (`number`) sets its real scale;

`dem.sea_level_pixel_value` (`number`) marks the pixel level at or below which land is sea.

geography — named places (optional)

Author-declared regions and landmarks. These feed the gazetteer and let the fact-checker resolve places you name in prose.

regions
`list` of `block`, each: `name` (`text`), `biome` (`text`), `climate` (`text`), `description` (`text`).

landmarks
`list` of `block`, each: `name` (`text`), `kind` (`text`, e.g. `"city"`, `"port"`, `"mountain"`), `climate_zone` (`text`), `population` (`number`), `description` (`text`). Optional position: `lat` + `lon` (`number`, degrees) or raw grid `x` + `y` (`number`, cells). A landmark with a `climate_zone` becomes a gazetteer entry the fact-checker knows by name; a landmark with a position is also drawn on the plakat map.

hydrology — named waters (optional)

Descriptive names laid over the procedural rivers, which still run.

rainfall
`text` — a note: `"arid"`, `"temperate"`, or `"wet"`.

rivers
`list` of `block`, each `name` (`text`) + `description` (`text`), and optionally a declared course: `from` and `to` (each a `[x, y]` cell). When both are set, the world checks the course runs downhill to water and warns if it does not.

lakes
`list` — same shape as `rivers`.

seas
`list` — same shape as `rivers`.

economy — trade and technology (optional)

tech_level
`text` — e.g. `"bronze"`, `"iron"`, `"medieval"`, `"industrial"`.

currency
`text` — the coin of the realm.

trade_goods
`list` of `text` — what moves along the roads.

resources
`list` of `text` — what the land yields; these are added to the fact-checker's known minerals, so trading a declared resource is not flagged.

magic — declared exceptions to physics (optional)

enabled
`text`/`number` — `true` turns the ledger on; `false` gates it entirely.

rules
`list` of `block`. Each rule: `kind` (`text`, your own label, e.g. `"messenger_birds"`); `covers` (`list` of `text` — which fact-check categories it may suppress: `astronomy`, `climate`, `climate_anomaly`, `date_coherence`, `demographics`, `economy`, `travel_time`, `character_age`); `description` (`text`); and `applicable_to` (`block` with optional `roles`, `regions`, `seasons` lists — an unset facet means "any"). Kind-specific parameters (e.g. `speed_kph_override`) may be added as extra keys on the rule.

history — declared events (optional)

Author events merged into the generated chronology; the world checks them and adopts place-linked ones onto the story Timeline.

events
`list` of `block`, each: `year` (`number` — years before the present, negative is the past); `title` (`text`); `epoch` (`text`, optional — inferred from the year when omitted); `places` (`list` of `text`, optional — accepted Place names, for Timeline links); `description` (`text`, optional). The world warns if a year is after the present or before recorded history, or if a declared `epoch` does not contain the year.

nations — declared realms (optional)

A `list` of nations; each pins a named realm, and the remaining settlements cluster into generated realms around them.

name
`text` — the realm's name.

capital
`[x, y]` — the capital's map cell; the nearest settlement becomes its seat. The world warns if it sits far from any settlement (in the wilderness).

relations
`list` of `block`, each `with` (`text` — another nation's name)
1. `stance` (`text` — `"allied"`, `"rival"`, `"neutral"`). Override the seeded relations.

cultures — pinned cultures (optional)

A `list` that overrides the generated culture of a nation, matched by name.

nation
`text` — the nation this culture belongs to.

ethos
`text`, optional — overrides the generated ethos. The world warns if a seafaring ethos is pinned to a dry inland capital.

belief
`text`, optional — the culture's belief system.

language
`text`, optional — a conlang typology profile, e.g. `"SOV · agglutinative · tonal"`.

ecology — pinned life (optional)

regions
`list` of `block`, each: `biome` (`text` — one of the twelve biome names, e.g. `"hot_desert"`); `flora` (`list` of `text`); `fauna` (`list` of `text`); `keystone` (`text`). Overrides the generated life for that biome. The world warns if cold-adapted life is pinned to a hot biome (or the reverse), or if the biome does not occur in the world.

AFTERWORD

About the author

Vladimir Ulogov.

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

Observability, in the end, is a discipline of **coherence** — of never reporting a state the system cannot account for, of insisting that every signal follow from something real. It is not an accident that a tool he built for writers carries the same instinct into the making of worlds.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not in the sense of small utilities, but in the sense of programming languages. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistack`, `rust_multistackvm`, `bundcore`, `zbus_universal_data_gateway` — each is a building block that exists because the off-the-shelf options didn't fit the shape of the work.

Why the World Simulation exists

Inkhaven is Vladimir's personal reflection on how a literary tool can help the people who write books. The World Simulation is one answer to a specific frustration: that the world a writer invents lives in scattered notes, a map in one program and a wiki in another, and a binder that quietly begins to contradict itself somewhere around chapter twelve — and that none of it is anywhere near the page where the story is actually written.

A novelist keeps a map open in one window, a timeline in a spreadsheet, and a folder of half-remembered decisions about how magic works; by the third act the sun rises in the wrong place and a rider crosses a continent in an afternoon. The World Simulation was built to close that gap: to make the world a **room inside the writing tool**, where the physics is consistent because it is computed rather than remembered, where the same seed always grows the same world, and where that world flows straight back into the manuscript — as places, as a calendar, as the weather of the very scene under your cursor.

A work of love

Inkhaven is open source. The licence is permissive — you can read it, fork it, study it, modify it, and pass it on. Strictly speaking, the licence also lets you sell it. The author would, gently but firmly, disagree with you doing that. Inkhaven was not designed as a *for sale* project. It is a work of love made for the authors who can least afford to pay for software, and turning it into a commercial product would betray the reason it exists. So please — don't.

It carries no analytics, no telemetry, no upsell. The binary will never phone home; the project will never have an "Enterprise" tier you have to escape from.

This was a deliberate choice. There are excellent commercial tools for world-building and writing. They cost money — which is fine for many writers, and a barrier for many more. Inkhaven exists for the second group: for the novelist building a trilogy on a battered laptop, for the game-master who shouldn't have to pick between rent and software, for the engineer drafting in the same terminal where they already write code, for anyone who would benefit from a tool that respects their work without asking for a credit-card number.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet.

If Inkhaven helps you finish your book — that is enough. If it gives you a chord pattern you adapt into your own tool — that's a gift back to the larger project of making software writers can love. As a society, we achieve the greatest things when we help each other rather than compete with each other.

Where to find more

GitHub	@vulogov — the source for Inkhaven, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and PRs welcome.
LinkedIn	/in/vladimirulogov — posts on observability, the occasional long-form essay.
YouTube	@vulogov — talks and walkthroughs from the conference trail.
X / Twitter	@vladimir_ulogov

If a river ever runs uphill in your world, or a season falls in the wrong month and you can't find the answer in this book — open an issue on GitHub. The author reads them.

END OF THE BOOK · INKHAVEN 3.0.0